Examples of Plots Generated by the blsplotGG Package

Brian C. Monsell

2026-02-19

Contents

1 Introduction 2
2 Series Plots 3
2.1 plot_date_span 3
2.2 plot_matrix 6
2.3 plot_ratio 9
2.4 plot_ratio_two 11
2.5 plot_series 16
2.6 plot_table 17
2.7 plot_year_over_year 21
3 regARIMA Modeling Diagnostic Plots 23
3.1 plot_acf 23
3.2 plot_acf_and_pacf 26
3.3 plot_acf_and_pacf_identify 27
3.4 plot_acf_matrix 30
3.5 plot_fcst 32
3.6 plot_fcst_two 33
3.7 plot_first_difference 35
3.8 plot_fts 36
3.9 plot_resid 38
4 Seasonal Adjustment Plots 41
4.1 plot_component_facet 41
4.2 plot_irregular_boxplot 42
4.3 plot_ratio_facet 45
4.4 plot_sadj_and_ori 47
4.5 plot_sadj_and_trend 49
4.6 plot_sadj_and_trend_facet 52
4.7 plot_sf_mean 54

4.8 plot_sf_series 57
4.9 plot_sf_two 58
4.10 plot_two_sa 60
4.11 plot_two_sa_comparison 64
4.12 plot_two_sa_facet 66
5 SEATS Plots 68
5.1 plot_fully_differenced_transformed 68
5.2 plot_fully_differenced_transformed_all 70
5.3 plot_seasonal_sums 71
5.4 plot_seats_filter 73
5.5 plot_seats_filter_all 75
5.6 plot_squared_gain 76
5.7 plot_time_shift 78
6 Spectral Plots 80
6.1 plot_double_spectrum 80
6.2 plot_spectrum 82
7 History Plots 86
7.1 plot_all_trend_lags 87
7.2 plot_changes_history 88
7.3 plot_fcst_history 91
7.4 plot_sa_history 93
8 Sliding Spans Plots 95
8.1 plot_maximum_percent_difference 96
8.2 plot_maximum_percent_difference_by_period 97
8.3 plot_sliding_spans 100
8.4 plot_sliding_spans_two 101
9 References 103

1 Introduction

The `blsplotGG` R package (Monsell 2025) was developed to provide seasonal adjustment users and researchers with quality data visualizations for seasonal adjustment within the X-13ARIMA-SEATS framework (U. S. Census Bureau 2025). The package uses the `ggplot2` package (Wickham 2016) to generate the plots and relies heavily on the `seasonal`

package (Christoph Sax and Eddelbuettel 2018) to generate the seasonal adjustments within the R environment (R Core Team 2024).

Many of the plotting functions use `seas` objects generated by the `seasonal` package to provide actual adjustments, as well as modeling and diagnostic information.

This document will use several series contained within base R as well as series within the `blsplotGG` package to illustrate what the package can do in terms of providing useful plots for researchers and analysts.

Many of the functions in the `blsplotGG` package take `ts` objects as input. A `ts` object is a vector or matrix of a series collected over equally spaced points in time, and is considered to be an object with class `ts`. This object is a basic building block for time series analysis in R. There are other types of time series objects defined in R - the `tsbox` package (Christoph Sax 2021) allows users to convert objects to and from different types of time series objects within R.

Many of the plots in this package were inspired by plots that appear in the `X-13-Graph` and `X-13-Graph Java` programs from the US Census Bureau. `X-13-Graph` was originally developed by Catherine Hood (Hood 2002) in SAS and is now maintained by Demetra Lytras (Lytras 2022a). X-13-Graph Java is an implementation of X-13-Graph in Java and was developed by Demetra Lytras (Lytras 2022b).

The remaining sections of this report will show the results of plotting functions grouped by type of plot. Within these groups, the functions are listed in alphabetical order. The R code that generated each plot is given before each plot.

2 Series Plots

This set of functions shows different plots of individual series. Only one plot requires a `seas` object generated by the `seasonal` package (`plot_table` from Section 2.6); many of the others take a `ts` object as their input.

2.1 plot_date_span

The `plot_date_span` function takes a `ggplot` object and returns another `ggplot` object with a user specified date range. This allows the user to quickly view spans of a given plot.

Figure 1 shows the full plot of the Airline Passengers series as used in examples from Box, Jenkins, and Reinsel (1976). This plot shows a trend component from an X-11 seasonal adjustment with the original series.

```r
# generate seas object with seasonal adjustment of Airline
# Passenger series
air_seas <-
   seasonal::seas(AirPassengers,
      arima.model = "(0 1 1)(0 1 1)",
      x11 = "")


# Create data frame with components of the Airline Passenger
# seasonal adjustment
air_df   <-
   data.frame(date = tsbox::ts_df(AirPassengers)$time,
              ori = as.double(seasonal::original(air_seas)),
              sa = as.double(seasonal::final(air_seas)),
              trend = as.double(seasonal::trend(air_seas)))

# Use ggplot2 to generate plot of original series (grey) and
# trend component (dark red) on the same axis, and store in
# air_p
air_p <-
   ggplot2::ggplot(air_df, ggplot2::aes(x=date)) +
   ggplot2::geom_line(ggplot2::aes(y = ori),
                      color = "grey") +
   ggplot2::geom_line(ggplot2::aes(y = trend),
                      color="darkred",
                      linetype="twodash") +
   ggplot2::labs(
      title = "Airline Passenger X-11 Trend Component",
      subtitle = NULL,
      x = "Time",
      y = "Airline Passengers")

# Display ggplot2 object
print(air_p)
```

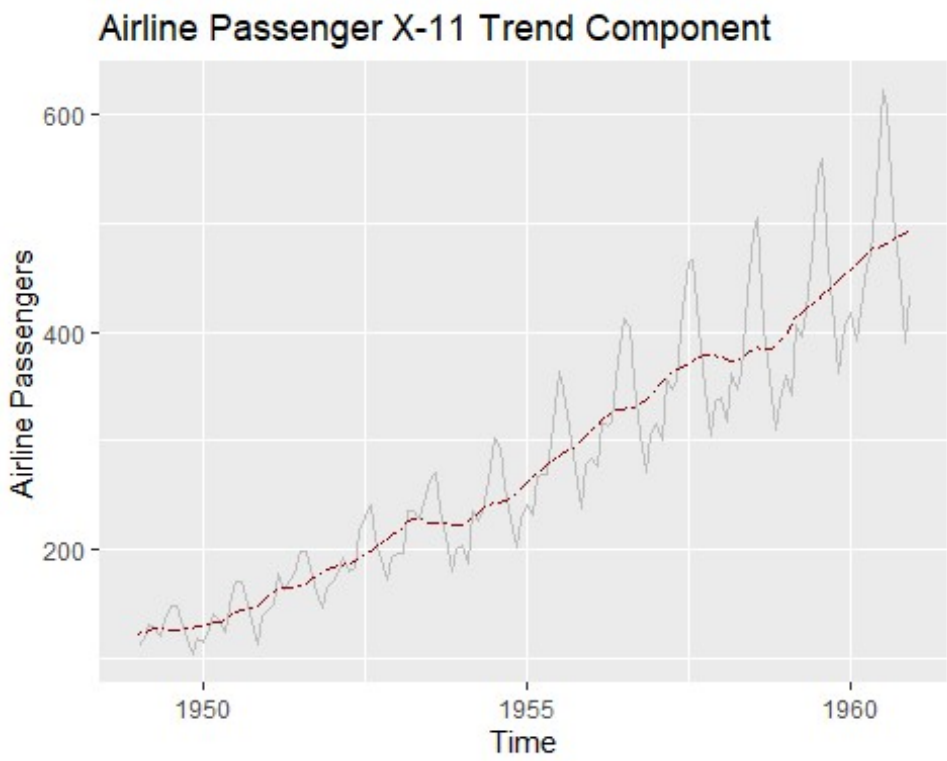

Figure 1: Original Plot Example

Figure 2 shows a shortened version of this plot, limiting the span of data from January of 1956 to December to 1960. These dates are supplied by the user.

```
# Use plot_date_span function to produce plot of span
air_short_p <-
   blsplotGG::plot_date_span(air_p,
      this_start_span = "1-1-1956",
      this_end_span = "12-31-1960",
      reset_y_limit = TRUE)

# Display ggplot2 object, using the suppressWarnings function
# to suppress warning messages
suppressWarnings(print(air_short_p))
```

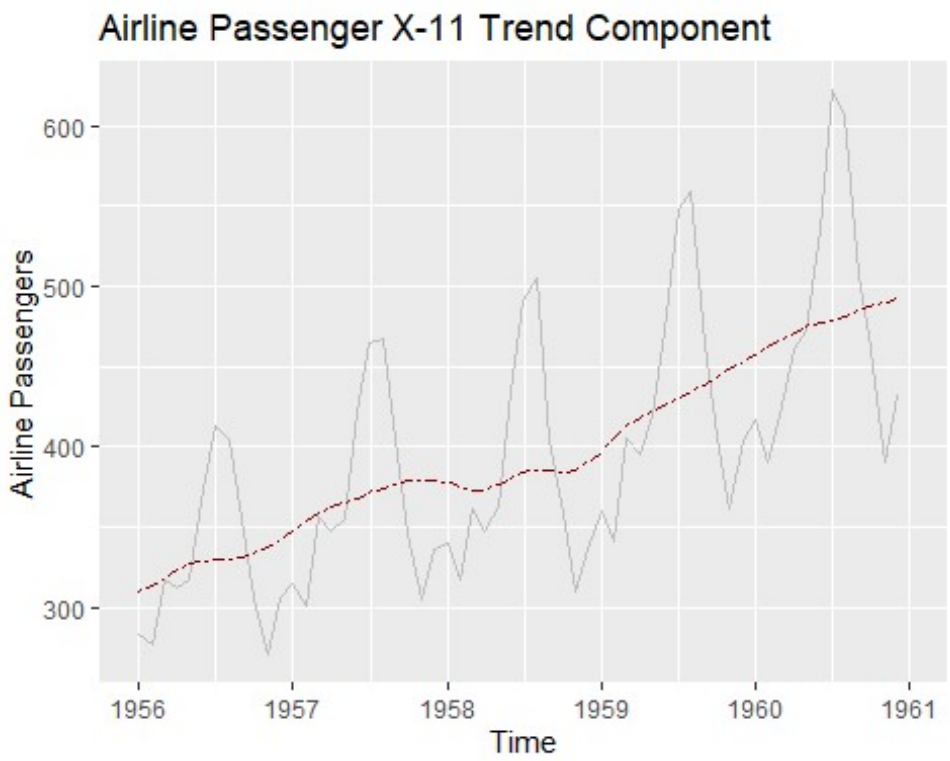

Figure 2: Shortened Plot Example

Note that using this function will likely generate a warning message like the one below:

```
message("Warning messages:
Removed 84 rows containing missing values or values outside the
scale range (`geom_line()`).")
```

If you want to suppress this warning message, you can use the `suppressWarnings` function from base R.

2.2 plot_matrix

The `plot_matrix` function produces plots of a `ts` matrix object. This object is a matrix where the columns are individual `ts` objects.

Figure 3 is an example of plotting the regional components of US Building Permits for One Family Units. This data is part of the `blsplotGG` package.

```r
# generate matrix of regional Building Permit series
BP_Region_Matrix <-
   cbind(blsplotGG::xt_data_list$mw1u,
         blsplotGG::xt_data_list$ne1u,
         blsplotGG::xt_data_list$so1u,
         blsplotGG::xt_data_list$we1u)
colnames(BP_Region_Matrix) <- names(blsplotGG::xt_data_list)

# generate matrix plot, save in p_BP
p_BP <-
   blsplotGG::plot_matrix(BP_Region_Matrix,
      this_y_label = 'Building Permits',
      main_title = "US Building Permits, 1 Family Units",
      do_grid = TRUE,
      this_line_type = rep("solid", 4),
      line_color =
         c("orange", "steelblue", "forestgreen", "brown"))

# Display ggplot2 object
print(p_BP)
```

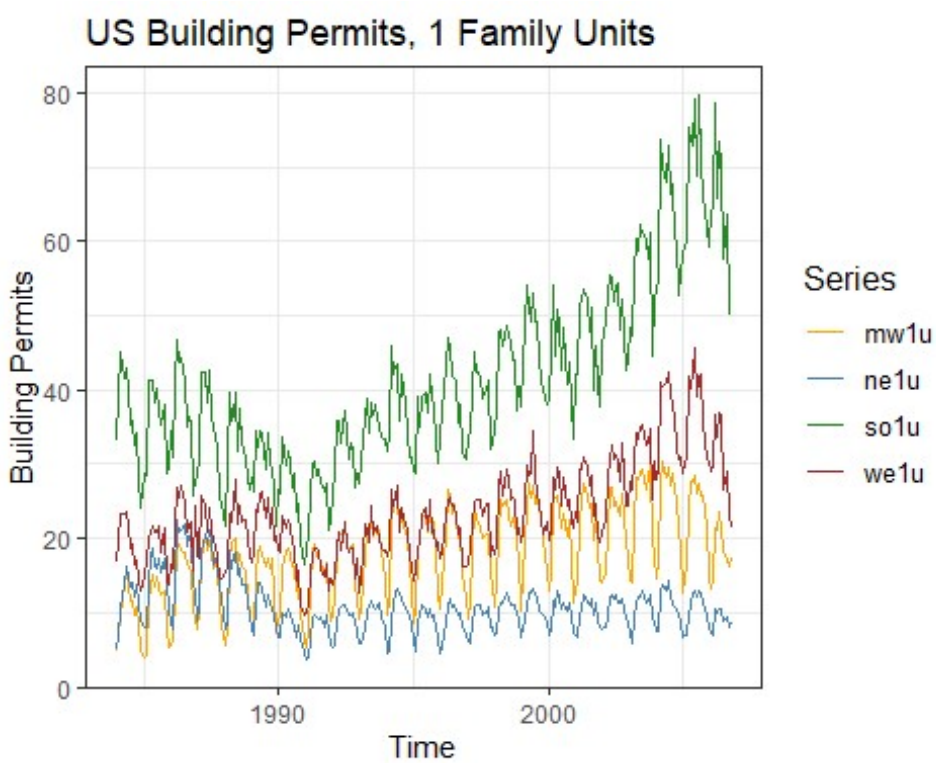

Figure 3: Matrix Plot Example

Figure 4 shows a facet plot with the same data. A facet plot shows each series plotted in its own panel, so instead of four lines in one plot, there are four panels.

```
# generate facet plot of matrix columns, save in p_BP_facet
p_BP_facet <-
  blsplotGG::plot_matrix(BP_Region_Matrix,
     this_y_label = 'Building Permits',
     main_title = "US Building Permits, 1 Family Units",
     do_grid = FALSE,
     do_facet = TRUE,
     line_color =
        c("orange", "steelblue", "forestgreen", "brown"))

print(p_BP_facet)
```

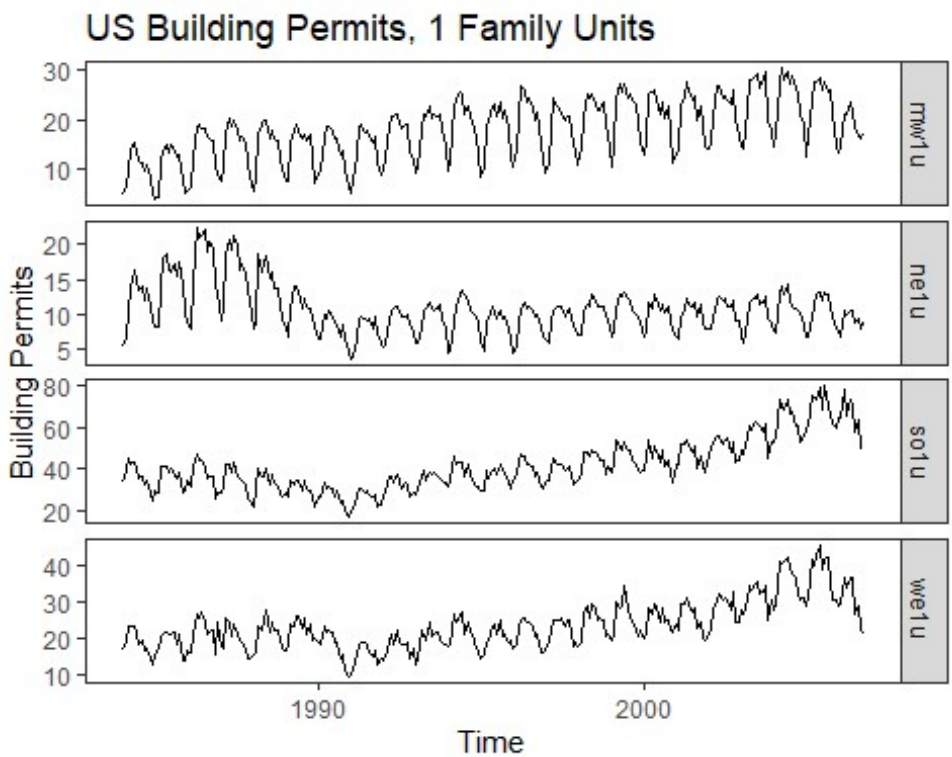

Figure 4: Matrix Facet Plot Example

Note that this function uses the `autoplot` function of the `forecast` package (R. J. Hyndman and Khandakar 2008; R. Hyndman et al. 2025) to generate the base plot for both the regular and facet versions of the matrix plot generated by `plot_matrix`.

2.3 plot_ratio

The `plot_ratio` function generates a high-density plot of a factor, usually generated from an X-13ARIMA-SEATS seasonal adjustment run. The plot is structured as line segments for each observation from an assumed mean. High density plots are very useful for factor tables like irregulars and seasonals and output from regARIMA modeling like calendar factors and outlier factors.

For multiplicative adjustments, the assumed mean is assumed to be one (1.0). This is the default and is shown in Figure 5, which shows a ratio plot of the seasonal factors from an X-11 run of the `UKgas` time series from the base R datasets. This is quarterly UK gas consumption from the first quarter of 1960 to the fourth quarter of 1986, shown in Durbin and Koopman (2012).

```
# seasonally adjust UKgas with X-11
ukgas_x11_seas <-
   seasonal::seas(UKgas,
      series.period = 4,
      arima.model = "(0 1 1)(0 1 1)",
      transform.function = "log",
      forecast.maxlead = 20,
      series.appendfcst = "no",
      x11.seasonalma = "s3x5",
      check.print = c( 'pacf', 'pacfplot' ))

# extract X-11 seasonal factors
ukgas_x11_sf     <- seasonal::series(ukgas_x11_seas, "d10")

# generate ratio plot of seasonal factors
ukgas_sf_ratio   <-
   bslplotGG::plot_ratio(ukgas_x11_sf,
      main_title = "UK Gas X-11 Seasonal Factors",
      sub_title = "Seasonal Filter: 3x5")

# display ggplot2 object
print(ukgas_sf_ratio)
```

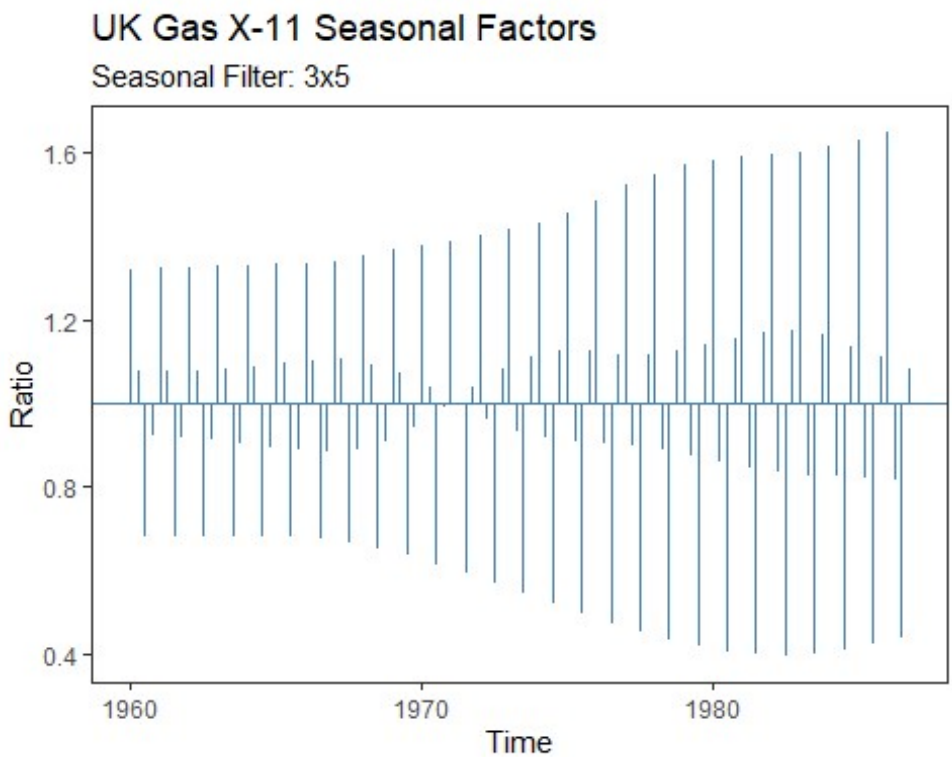

Figure 5: Ratio Plot Example

Figure 6 shows a ratio plot of the additive seasonal factors from an X-11 run of the `UKgas` time series. The assumed mean is set to zero by setting the `ratio_mean` argument to 0.

```
# Seasonally adjust UKgas with X-11 additive adjustment
ukgas_x11_add_seas <-
   seasonal::seas(UKgas, series.period = 4,
      arima.model = "(0 1 1)(0 1 1)",
      transform.function = "none",
      forecast.maxlead = 20,
      series.appendfcst = "no",
      x11.seasonalma = "s3x5",
      check.print = c( 'pacf', 'pacfplot' ))

# Extract additive seasonal factors
ukgas_x11_add_sf <-
   seasonal::series(ukgas_x11_add_seas, "d10")
```

```r
# Generate ratio plot of additive seasonal factors
ukgas_add_sf_ratio   <-
    plot_ratio(ukgas_x11_add_sf,
        main_title = "UK Gas X-11 Additive Seasonal Factors",
        sub_title = "Seasonal Filter: 3x5",
        ratio_mean = 0.0,
        ratio_color = 'darkblue')

# Display ggplot2 object
print(ukgas_add_sf_ratio)
```

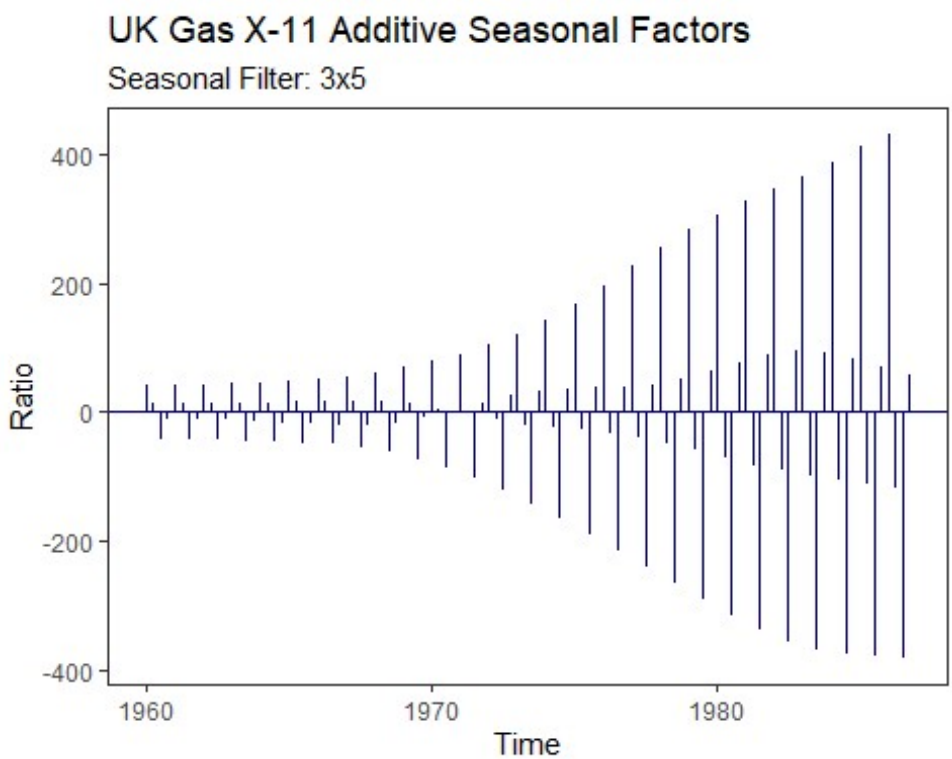

Figure 6: Ratio Plot Example, Additive Factors

2.4 plot_ratio_two

The `plot_ratio_two` function generates a facet plot of two time series (`ts`) objects. This allows for more easy comparison of two factors.

Figure 7 shows a comparison between the seasonal factors for the UK Gas series from an X-11 seasonal adjustment and a SEATS seasonal adjustment.

```r
# Seasonally adjust UKgas with X-11 multiplicative adjustment
ukgas_x11_seas   <-
   seasonal::seas(UKgas, series.period = 4,
      arima.model = "(0 1 1)(0 1 1)",
      transform.function = "log", forecast.maxlead = 20,
      x11.seasonalma = "s3x5", x11.save = c("d10", "d11"),
      check.print = c( 'pacf', 'pacfplot' ),
      series.appendfcst = "no")

# Seasonally adjust UKgas with SEATS adjustment
ukgas_seats_seas <-
   seasonal::seas(UKgas, series.period = 4,
      arima.model = "(0 1 1)(0 1 1)",
      transform.function = "log", forecast.maxlead = 20,
      seats.save = c("s10", "s11"), series.appendfcst = "no",
      check.print = c( 'pacf', 'pacfplot' ))

# Extract seasonal factors
ukgas_x11_sf      <- seasonal::series(ukgas_x11_seas, "d10")
ukgas_seats_sf    <- seasonal::series(ukgas_seats_seas, "s10")

# Produce seasonal factor ratio plot, with facets
ukgas_sf_two_plot <-
     blsplotGG::plot_ratio_two(ukgas_x11_sf, ukgas_seats_sf,
        overall_title = "UK Gas Production",
        ratio_one_title = "X-11 Seasonal Factors",
        ratio_two_title = "SEATS Seasonal Factors",
        do_comparison_plot = FALSE,
        ratio_color = "darkgreen")

# Display ggplot2 object
print(ukgas_sf_two_plot)
```

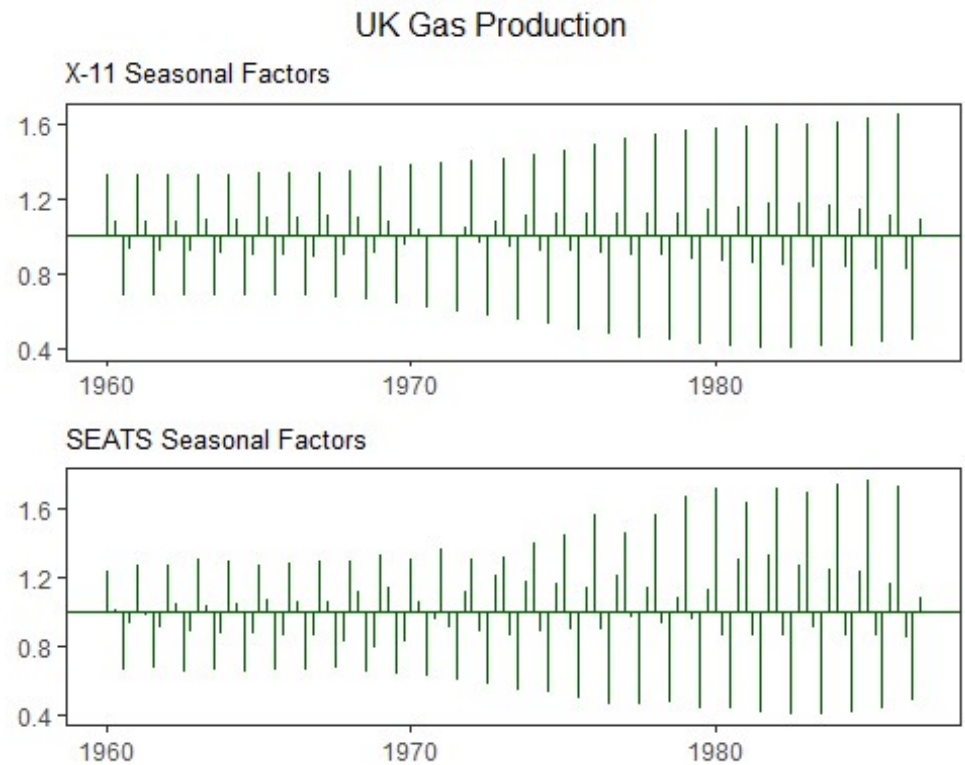

Figure 7: Ratio Comparison Plot Example

Figure 8 adds a comparison panel to Figure 7, showing a direct comparison between the two sets of factors. This is done by setting the `do_comparison_plot` argument to TRUE; this is the default.

Since these are multiplicative factors, the ratio between the two sets of factors is shown in the third panel of Figure 8.

```
# Produce seasonal factor ratio plot, with comparison panel
ukgas_sf_two_plot <-
   blsplotGG::plot_ratio_two(ukgas_x11_sf, ukgas_seats_sf,
      overall_title = "UK Gas Production",
      ratio_one_title = "X-11 Seasonal Factors",
      ratio_two_title = "SEATS Seasonal Factors",
      do_comparison_plot = TRUE,
      comparison_title =
         "Ratio of Seasonal Factors (X11 / SEATS)",
      ratio_color = "darkgreen")

# Display ggplot2 object
print(ukgas_sf_two_plot)
```

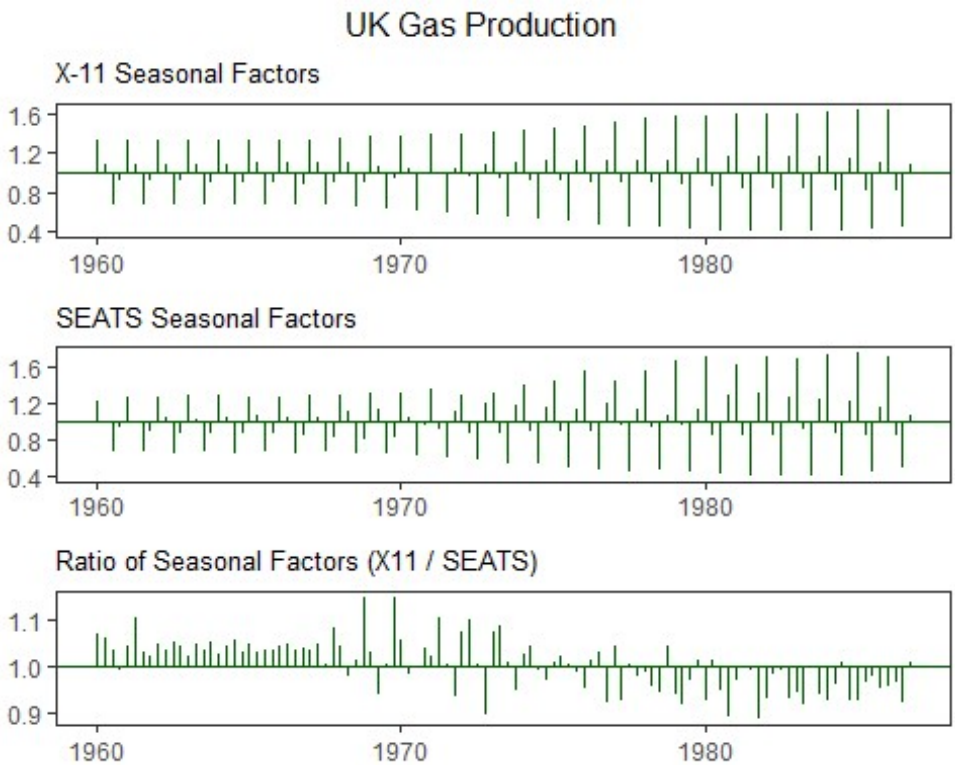

Figure 8: Ratio Comparison Plot Example with Comparison Panel

Figure 9 compares the seasonal factors from an additive and multiplicative SEATS seasonal adjustment.

There is no comparison panel as the adjustment mode needs to be the same for both time series for there to be a comparison panel.

```
# Seasonally adjust UKgas with SEATS additive adjustment
ukgas_seats_add_seas <-
   seasonal::seas(UKgas, series.period = 4,
      arima.model = "(0 1 1)(0 1 1)",
      transform.function = "none", forecast.maxlead = 20,
      seats.save = c("s10", "s11"), series.appendfcst = "no",
      check.print = c( 'pacf', 'pacfplot' ))

# Extract seasonal factors
ukgas_seats_add_seas_sf    <-
   seasonal::series(ukgas_seats_add_seas, "s10")
```

```r
# Produce seasonal factor ratio plot, additive vs multiplicative
# SEATS adjustments
ukgas_sf_two_plot_add_mult <-
   blsplotGG::plot_ratio_two(ukgas_seats_add_seas_sf,
      ukgas_seats_sf,
      overall_title = "UK Gas Production",
      ratio_one_title = "SEATS Additive Seasonal Factors",
      ratio_two_title = "SEATS Seasonal Factors",
      do_comparison_plot = FALSE,
      ratio_color = "darkgreen")

# Display ggplot2 object
print(ukgas_sf_two_plot_add_mult)
```

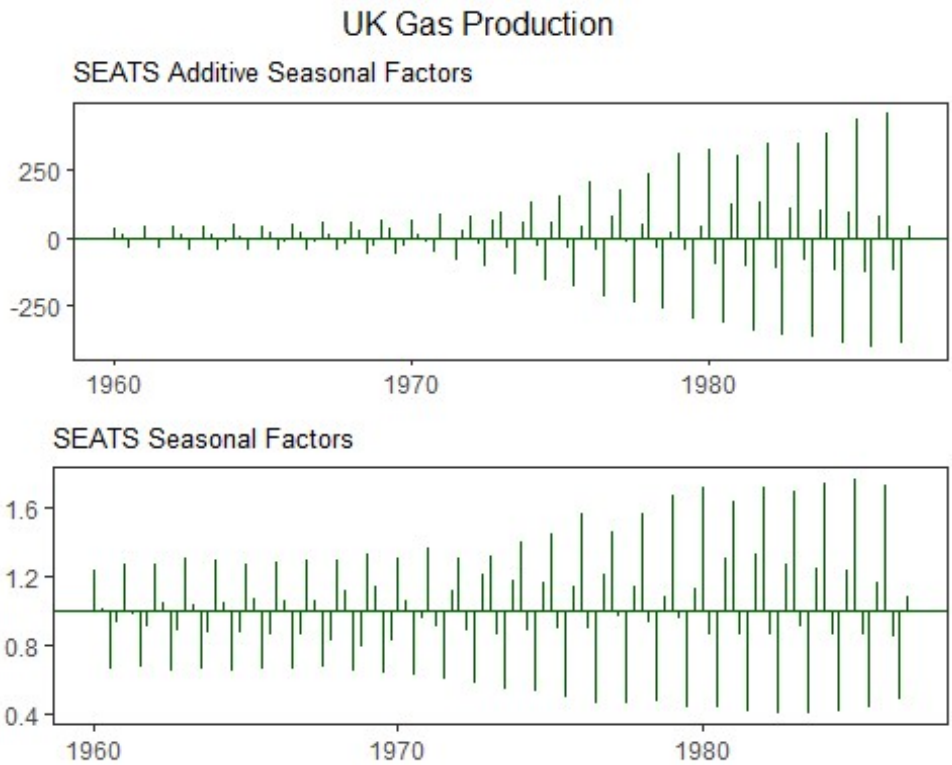

Figure 9: Ratio Comparison Plot Example (Additive vs Multiplicative)

The individual ratio plots are generated using calls to `plot_ratio`. They are put in facet format using the `ggarrange` function of the `ggpubr` package (Kassambara 2025).

2.5 plot_series

The `plot_series` function generates a time series plot for an R `ts` object. This can be a series generated by the `seasonal` package, or any series that can be expressed as a `ts` object.

Figure 10 shows a simple plot of the seasonally adjusted Airline Passenger series. In this plot, the `draw_recess` argument has been set to TRUE, which uses grey shading to mark US recessionary periods. Also, the `do_grid` argument is set to TRUE, setting grid lines for the plot.

```
# Seasonally adjust Airline Passenger series with X-11
air_seas <-
   seasonal::seas(AirPassengers,
      arima.model = "(0 1 1)(0 1 1)",
      x11="",
      transform.function = "log")

# Extract seasonally adjusted series
air_seas_d11 <- seasonal::final(air_seas)

# Generate plot of seasonally adjusted series
air_d11_p <-
   blsplotGG::plot_series(air_seas_d11,
      this_series_name = "AirPassengers",
      main_title =
         'X-11 Seasonal Adjustment of Airline Passengers',
      sub_title = 'Box-Jenkins Airline series',
      do_grid = TRUE,
      draw_recess = TRUE,
      line_color = "darkblue")

# Display ggplot2 object
print(air_d11_p)
```

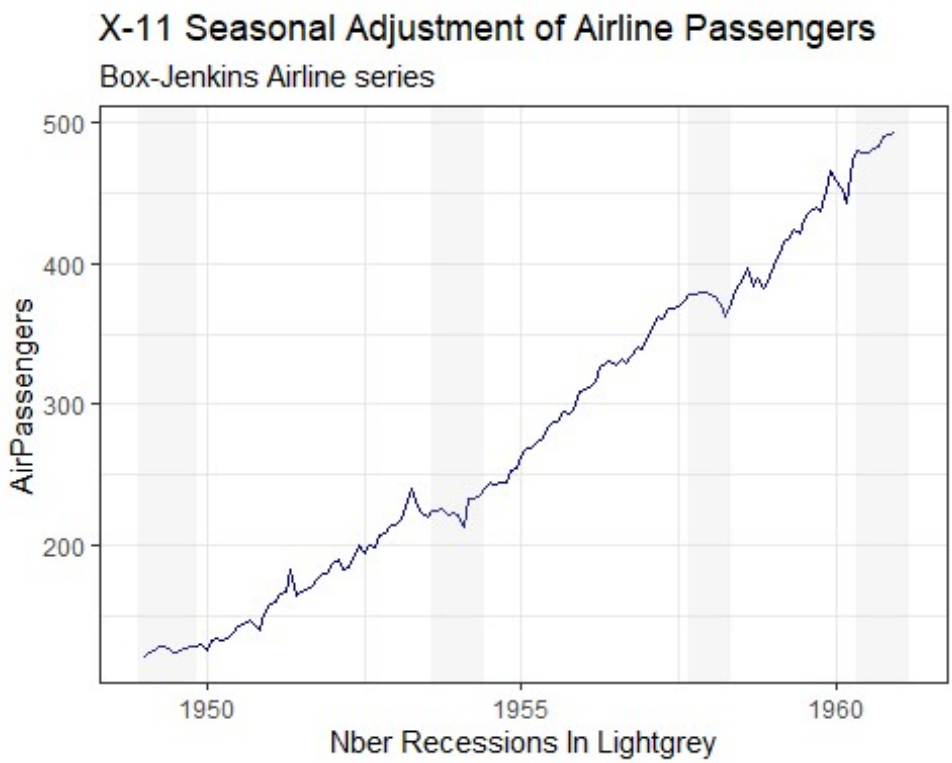

Figure 10: Series Plot Example

2.6 plot_table

The `plot_table` function generates a plot of a table from the X-13ARIMA-SEATS output specified by the user using either the full table name (example: `seasadj` for the seasonally adjusted series) or the abbreviation of the table (example: `d11`).

Valid table and abbreviation names for this function can be found in the documentation for the `series` function of the `seasonal` package (Christophe Sax 2018).

Figure 11 shows a simple plot of the seasonally adjusted Airline Passenger series (table D11), which is like Figure 10. One difference is that a line has been added for an outlier (`AO1951.May`, a point outlier in May of 1951) that is found in the regARIMA model identified for the Airline Passenger series.

```
# Seasonally adjust Airline Passenger series with X-11
air_seas <-
   seasonal::seas(AirPassengers,
      arima.model = "(0 1 1)(0 1 1)", x11="",
      series.save = 'b1',
      transform.function = "log",
      x11.save = "e3")
```

```r
# Generate plot of the seasonally adjusted series with
# annotation for outliers
air_d11_p <-
   blsplotGG::plot_table(air_seas, "d11",
      this_y_label = "AirPassengers",
      main_title =
         "X-11 Seasonal Adjustment of Airline Passengers",
      sub_title = "Seasonal Adjustment (D11)",
      do_grid = TRUE,
      draw_recess = TRUE,
      use_ratio = FALSE,
      add_outlier = TRUE,
      line_color = "darkblue")

# Display ggplot2 object
print(air_d11_p)
```

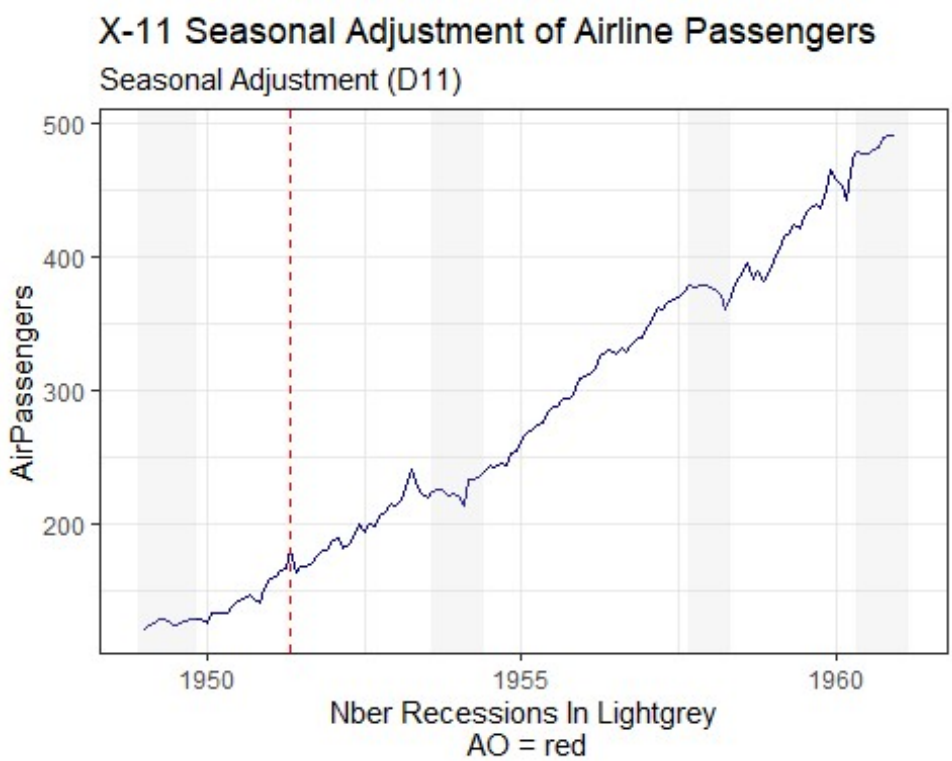

Figure 11: Table Plot (D11) Example

Figure 12 is a line plot of the combined adjustment factors (table D16). In this plot, the shading for the recessionary periods has been kept, and the grid lines have been removed.

```r
# Generate plot of the combined adjustment factors (Table D16)
# with annotation for outliers, recession shading, no grid lines
air_d16_p <-
   blsplotGG::plot_table(air_seas, "d16",
      this_y_label = "AirPassengers",
      main_title =
         "X-11 Seasonal Adjustment of Airline Passengers",
      sub_title = "Combined Factors (D16)",
      do_grid = FALSE,
      draw_recess = TRUE,
      add_outlier = TRUE,
      line_color = "steelblue")

# Display ggplot2 object
print(air_d16_p)
```

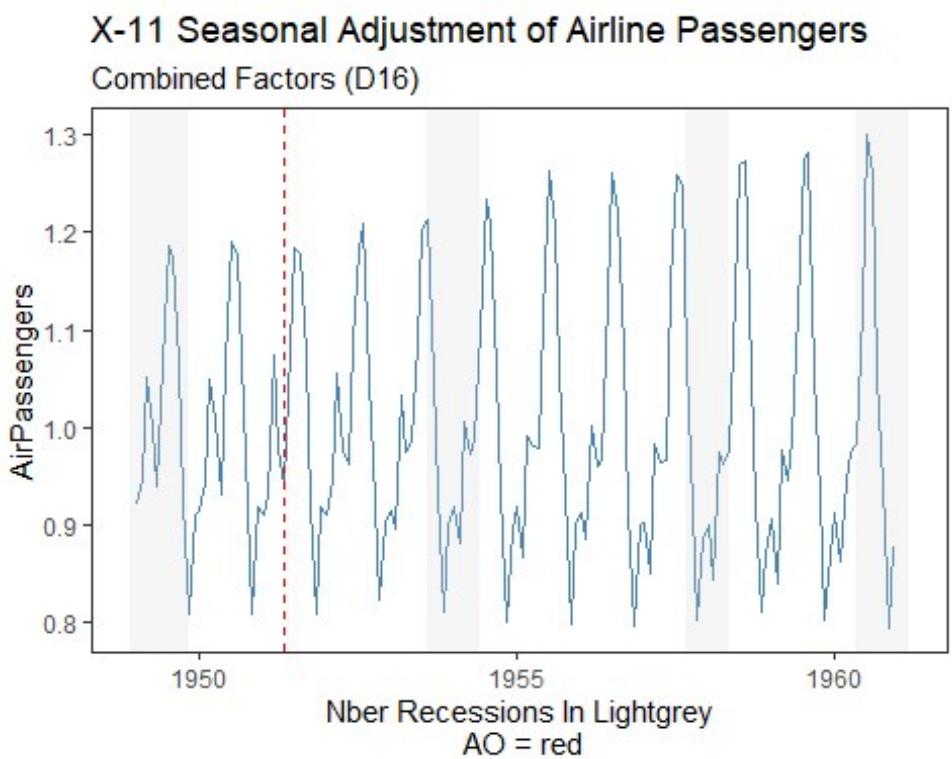

Figure 12: Table Plot (D16) Example

Figure 13 is a high-density plot of the extreme modified irregular factors (table E3). This is the same as the `plot_ratio` plot in Section 2.3.

```r
# Generate ratio plot of the X-11 extreme adjusted irregular
# (Table E3)
air_e3_p <-
  blsplotGG::plot_table(air_seas, "e3",
    this_y_label = "AirPassengers",
    main_title =
      "X-11 Irregular (Extreme Adjusted) of Airline Passengers",
    sub_title = "Box-Jenkins Airline series",
    do_grid = FALSE,
    draw_recess = TRUE,
    use_ratio = TRUE,
    add_outlier = TRUE,
    line_color = "steelblue")

# Display ggplot2 object
print(air_e3_p)
```

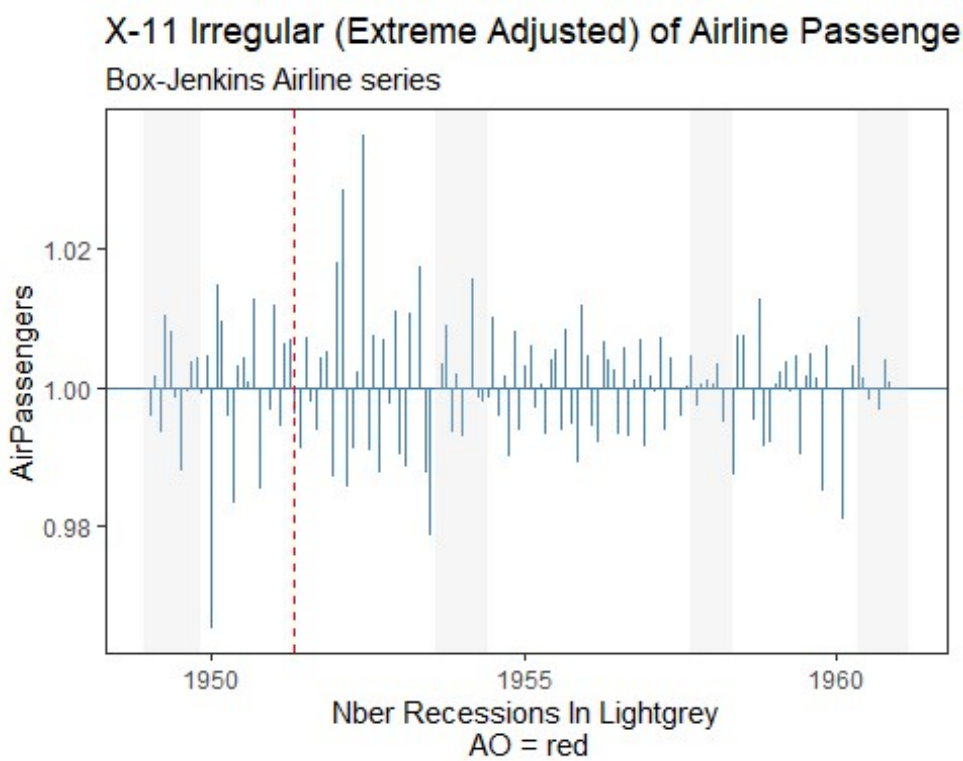

Figure 13: Table Plot (E3) Example

In this plot, the shading for the recessionary periods has been kept, and the grid lines have been removed. The vertical line for the identified outlier is also present.

2.7 plot_year_over_year

The `plot_year_over_year` function generates a plot where each year of the time series is plotted on the same axis to look for seasonal patterns in the data. This type of plot first appeared in W. S. Cleveland and Terpenning (1982).

Figure 14 shows an example of a year-over-year plot for the Airline Passenger series. Each year has a different color, controlled by default by the `RColorBrewer` palette `"Paired"` (Neuwirth 2022).

```
# Generate year over year plot for Airline Passenger Series
this_yyplot <-
    blsplotGG::plot_year_over_year(AirPassengers,
        this_y_label = "Air",
        this_palette = "Dark2",
        this_legend_size = 0.25,
        this_legend_space = 0)

# Display ggplot2 object, using the suppressWarnings function
# to suppress warning messages
suppressWarnings(print(this_yyplot))
```

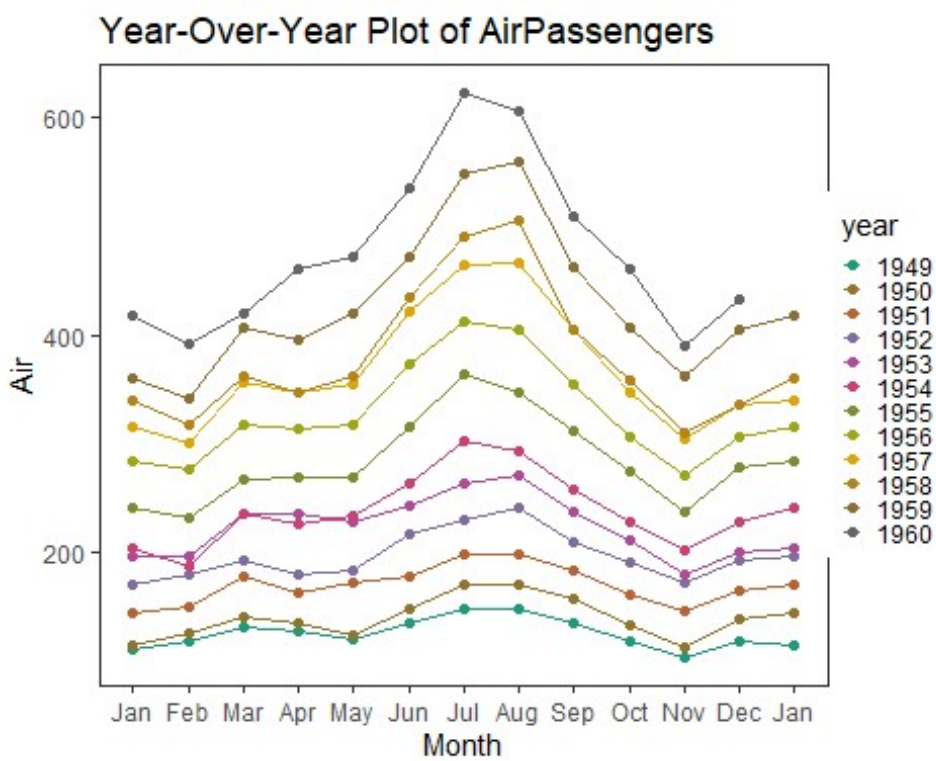

Figure 14: Year-to-Year Plot Example

Figure 15 shows an example of a year-over-year plot of the detrended series. In this case, the series is detrended using a Loess trend (R. B. Cleveland et al. 1990). This is specified through setting `detrend_series = TRUE` and `detrend_lowess = TRUE` in the call to `plot_year_over_year`; the detrending itself is done using the `detrend` function of the `astsa` package (Stoffer 2025).

```
# Set order of legend elements
legend_order <-
   as.character(as.vector(matrix(1949:1960, ncol = 4,
                                 byrow = TRUE)))

# Generate year over year plot for Loess detrended
# Airline Passenger Series, moving the legend to the bottom
# of the graph
this_yyplot_detrend <-
    blsplotGG::plot_year_over_year(AirPassengers,
        this_y_label = "Air",
        this_palette = "Dark2",
        main_title =
            "YY Plot of Loess Detrended Air Passenger Series",
        detrend_series = TRUE,
        detrend_lowess = TRUE,
        this_legend_position = "bottom",
        this_legend_size = 0.35,
        this_legend_order = legend_order,
        this_legend_space = 0)

# Display ggplot2 object, using the suppressWarnings function
# to suppress warning messages
suppressWarnings(print(this_yyplot_detrend))
```

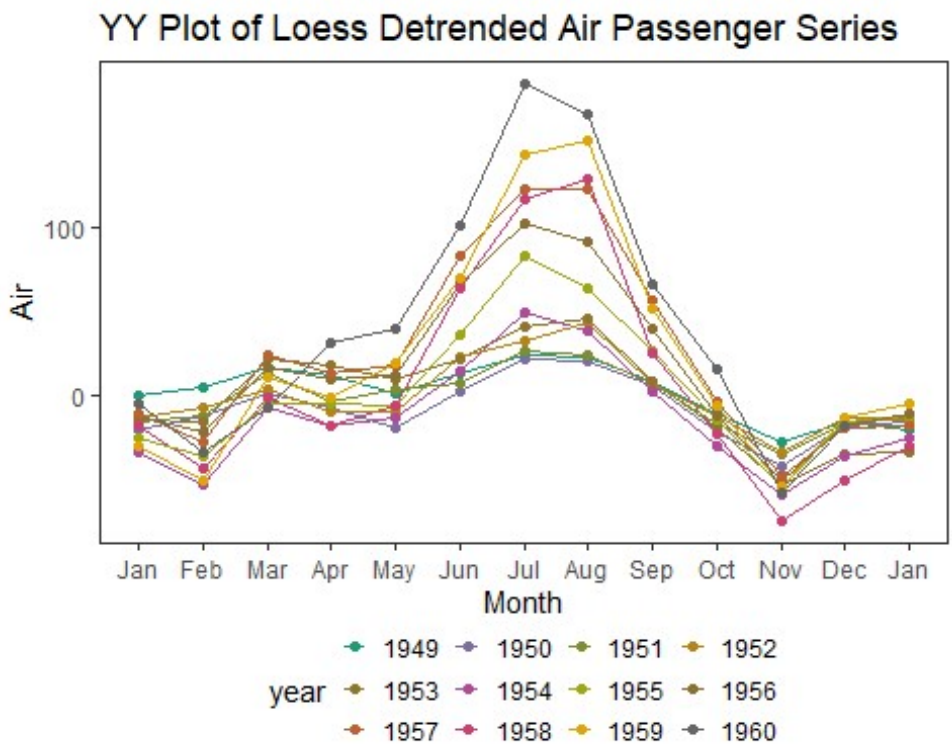

Figure 15: Detrended Year-to-Year Plot Example

Note that this plot the legend appears in the bottom of the plot. This is done by setting the `this_legend_position` argument to `"bottom"`; the default is having the legend to the right of the plot, as in Figure 14.

3 regARIMA Modeling Diagnostic Plots

There are several diagnostic plots generated by the `blsplotGG` package that are helpful to analysts fitting regARIMA models to time series. RegARIMA models are models that have a time-varying mean function modelled via linear regression effects with ARIMA errors.

RegARIMA models are described in Chapter 4 of U. S. Census Bureau (2025), and their uses in seasonal adjustment are detailed in Findley et al. (1998).

There are four ACF-related functions (Section 3.1 to 3.4), two forecast plotting functions (Section 3.5 and 3.6) a function that shows outlier identification t-statistics (Section 3.8), and a function that plots the regARIMA residuals (Section 3.9).

3.1 plot_acf

The `plot_acf` function can be used to generate plots of the autocorrelation and partial autocorrelation functions of the regARIMA residuals. The analyst can also specify a plot of the autocorrelation of the squared residuals. This plot will have upper and lower confidence intervals for the autocorrelations and partial autocorrelations plotted.

Figure 16 shows an ACF plot for the UK Gas time series. The autocorrelations and standard errors for the autocorrelations of the regARIMA model residuals are extracted from a `seas` object generated by the `seas` function of the `seasonal` package.

```
# Seasonally adjust UKGas series
ukgas_x11_seas <-
   seasonal::seas(UKgas, series.period = 4,
      arima.model = "(0 1 1)(0 1 1)",
      transform.function = "log",
      forecast.maxlead = 20,
      x11.seasonalma = "s3x5",
      check.print = c( 'pacf', 'pacfplot' ), check.maxlag = 12,
      check.save = c("acf", "pcf", "ac2"))

# Produce Residual ACF plot of UK Gas series
p_ukgas_acf   <-
   blsplotGG::plot_acf(ukgas_x11_seas, this_plot = "acf",
      main_title = "UK Gas Model Residual ACF",
      acf_color = "darkblue")

# Display ggplot2 object
print(p_ukgas_acf)
```

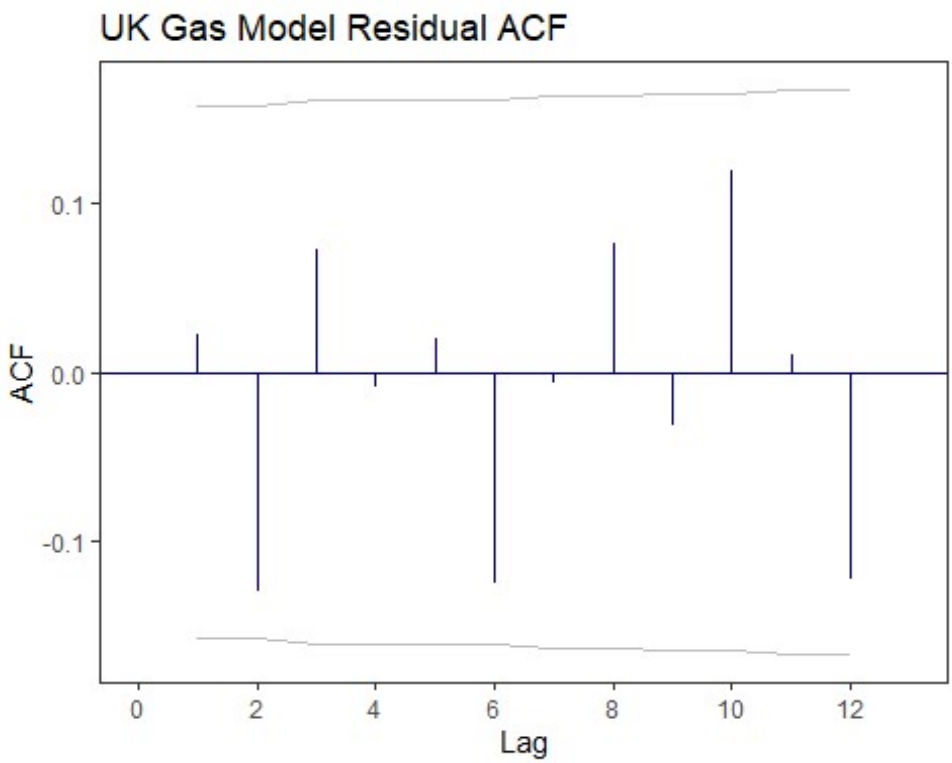

Figure 16: ACF Plot Example

ACF plots are useful to determine if the residuals have a significant amount of autocorrelation. We expect the residuals to be normally distributed white noise; eliminating autocorrelations from the residuals is a primary concern for the time series modeler.

Figure 17 is an example of a PACF plot of the UK Gas series. Similar to the ACF plot, there should be no significant lags, and the user may want to change the model with more AR terms if significant lags are found in the PACF plot.

```
# Produce Residual PACF plot of UK Gas series
p_ukgas_pacf   <-
   blsplotGG::plot_acf(ukgas_x11_seas, this_plot = "pcf",
            main_title = "UK Gas Model Residual PACF",
            acf_color = "darkblue")

# Display ggplot2 object
print(p_ukgas_pacf)
```

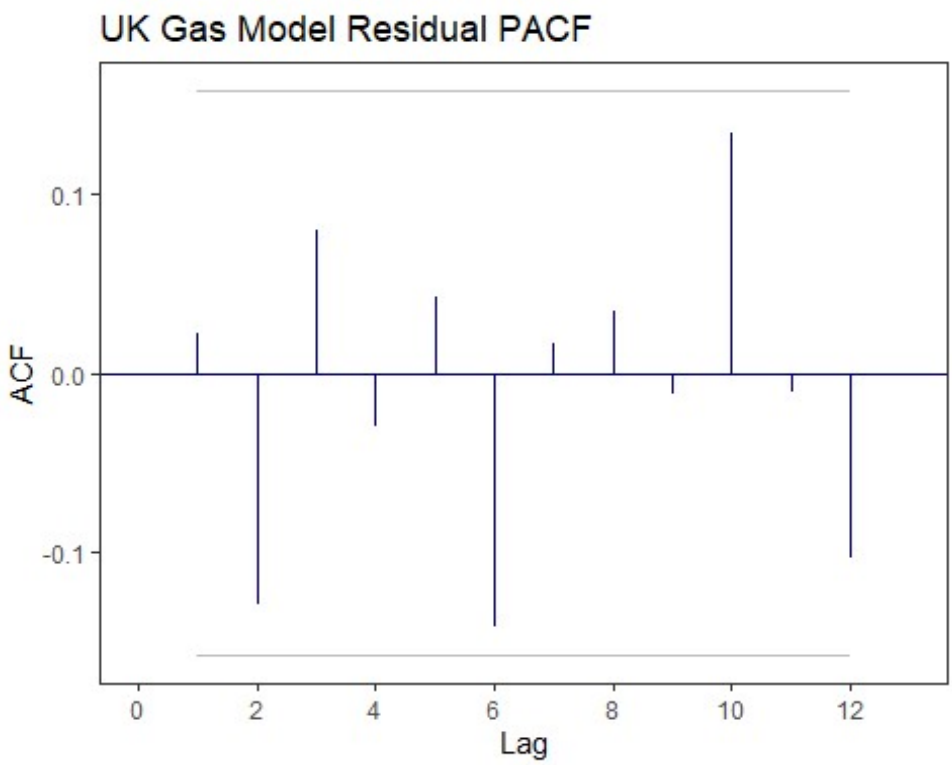

Figure 17: PACF Plot Example

Figure 18 is an example of an ACF plot for the squared residuals of the UK Gas series. This type of plot is helpful in indicating whether there is time-varying volatility in the residuals.

```
# Produce residual squared ACF plot of UK Gas series
p_ukgas_acf2 <-
   blsplotGG::plot_acf(ukgas_x11_seas, this_plot = "ac2",
            main_title = "UK Gas Model Residual Squared ACF",
            acf_color = "darkblue")

# Display ggplot2 object
print(p_ukgas_acf2)
```

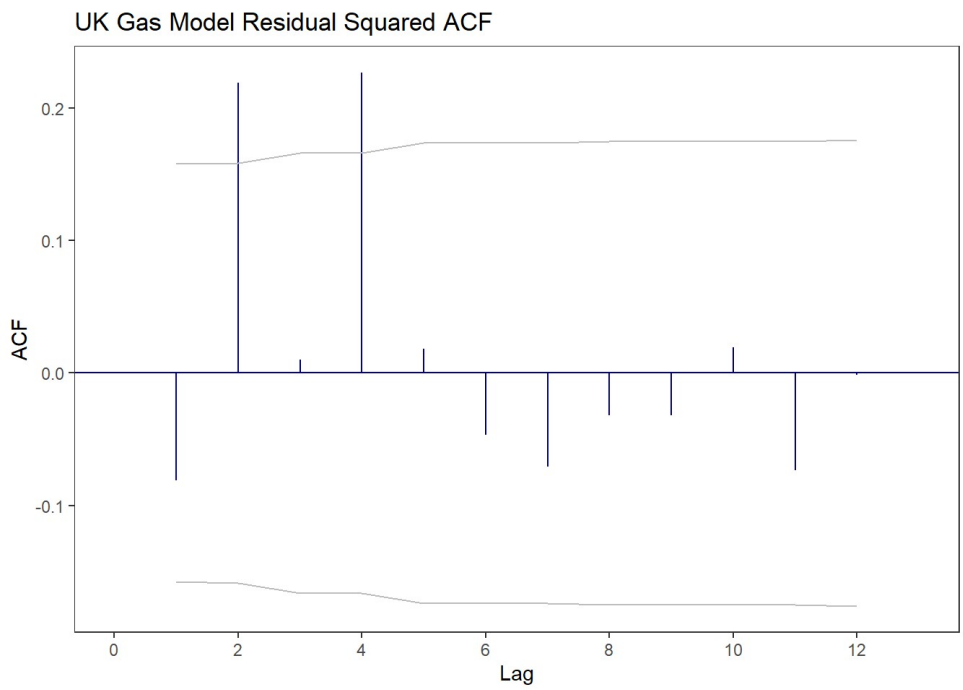

Figure 18: Squared ACF Plot Example

Note that `plot_acf` calls another function, `plot_acf_matrix`, to handle the actual plotting of the autocorrelation and partial autocorrelation functions. That function is described in Section 3.4.

3.2 plot_acf_and_pacf

The function `plot_acf_and_pacf` generates a plot of the regARIMA residual autocorrelations and partial autocorrelation, both extracted from a seas object provided by the analyst. The ACF and PACF plots are formatted as a facet plot - each plot is located in its own panel.

Figure 19 shows an example of this for the UK Gas series. Having the ACF and PACF in the same plot allows the analyst to compare the plots for the different lags.

```r
# Produce residual ACF and PACF plot of UK Gas series
p_ukgas_acf_and_pacf   <-
   blsplotGG::plot_acf_and_pacf(ukgas_x11_seas,
      overall_title = "UK Gas Model",
      acf_color = "darkblue")

# Display ggplot2 object
print(p_ukgas_acf_and_pacf)
```

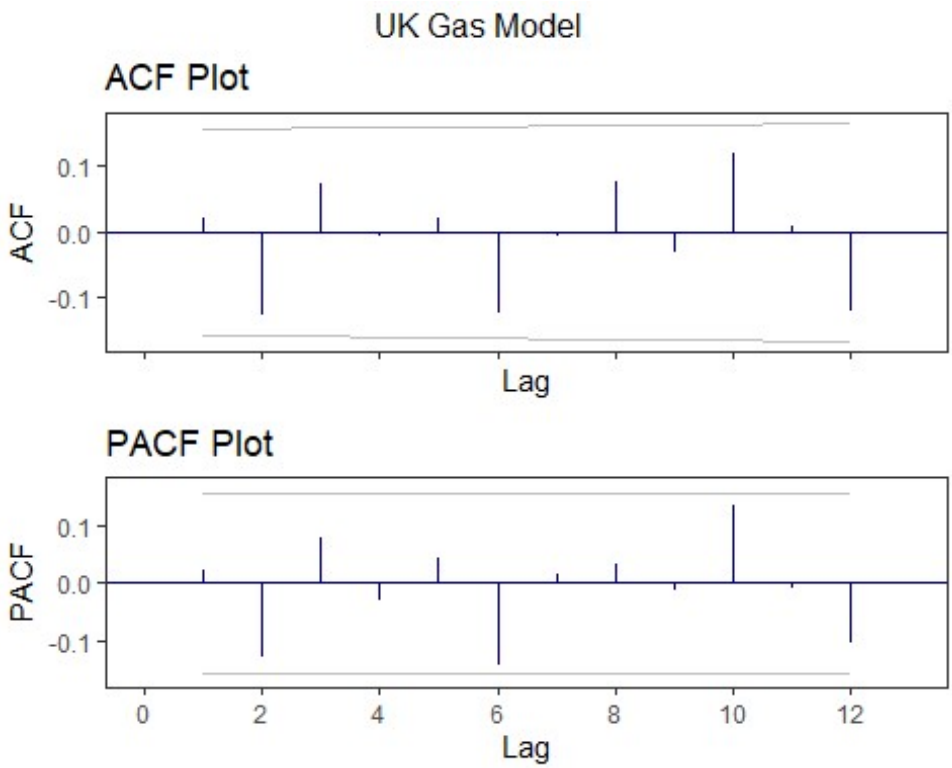

Figure 19: ACF and PACF Plot Example

As with `plot_acf`, the individual plots are generated using the `plot_acf_matrix` function. They are put in facet format using the `ggarrange` function of the `ggpubr` package (Kassambara 2025).

3.3 plot_acf_and_pacf_identify

The `plot_acf_and_pacf_identify` generates ACF and PACF plots of the original, transformed series from a `seas` object where options from the identify spec of X-13ARIMA-SEATS have been specified.

The user must specify options from the `identify` spec for this function to work; more information on the identify spec can be found in Chapter 7.9 of U. S. Census Bureau (2025).

Figure 20 generates an ACF and PACF plot for the transformed original series with no regular or seasonal differencing for the US retail shoe store series.

In the call to the `seas` function, the differencing orders for the ACF and PACF plots generated in the output are set by the `identify.diff` for regular differences and the `identify.sdiff` arguments for seasonal differenes, and setting `identify.save = c("iac", "ipc")` makes the autocorrelations and partial autocorrelations for every combination available to the `series` function from the `seasonal` package.

Each call of `plot_acf_and_pacf_identify` plots the ACF and PACF for one combination of the regular and seasonal difference applied to the transformed original series. In this case, no regular and seasonal differencing is specified for the `plot_acf_and_pacf_identify` function.

```
# Generate X-13 seas object with identify spec options
shoes_identify_seas <-
   seasonal::seas(blsplotGG::shoes2008,
                  identify.diff = c(0, 1),
                  identify.sdiff = c(0, 1),
                  identify.save = c("iac", "ipc"),
                  arima.model = "(0 1 1)(0 1 1)",
                  transform.function = "log",
                  forecast.maxlead = 36,
                  check.maxlag = 36,
                  check.acflimit = 1.96,
                  check.qlimit = 0.01,
                  check.print = c( 'pacf', 'pacfplot' ))

# Produce ACF and PSCF plot for no differencing
p_shoes_acf_and_pacf_identify_d0_sd0   <-
   blsplotGG::plot_acf_and_pacf_identify(
      seas_obj = shoes_identify_seas,
      overall_title = "US Shoe Sales - No Differencing",
      acf_color = "darkblue")

# Display ggplot2 object
print(p_shoes_acf_and_pacf_identify_d0_sd0)
```

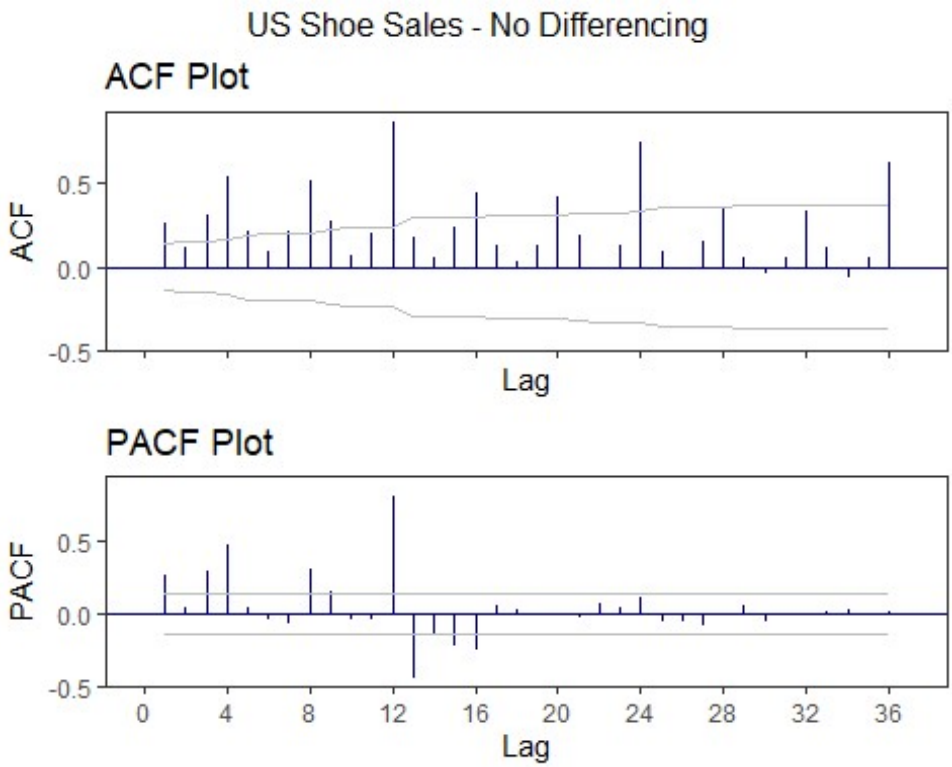

Figure 20: ACF and PACF Plot, No Differencing

Figure 21 shows the output for regular and seasonal differencing of the transformed original series. By examining this type of output for several combinations of regular and seasonal differencing, analysts can determine the optimal orders of differencing and ARMA model parameters.

```
# Produce ACF and PACF plot for regular and seasonal
# differencing, US Shoe Sales Series
p_shoes_acf_and_pacf_identify_d1_sd1   <-
   blsplotGG::plot_acf_and_pacf_identify(shoes_identify_seas,
      this_diff = 1,
      this_sdiff = 1,
      overall_title =
         "US Shoe Sales - Regular and Seasonal Differencing",
      acf_color = "darkblue",
      remove_acf_x_labels = FALSE)

# Display ggplot2 object
print(p_shoes_acf_and_pacf_identify_d1_sd1)
```

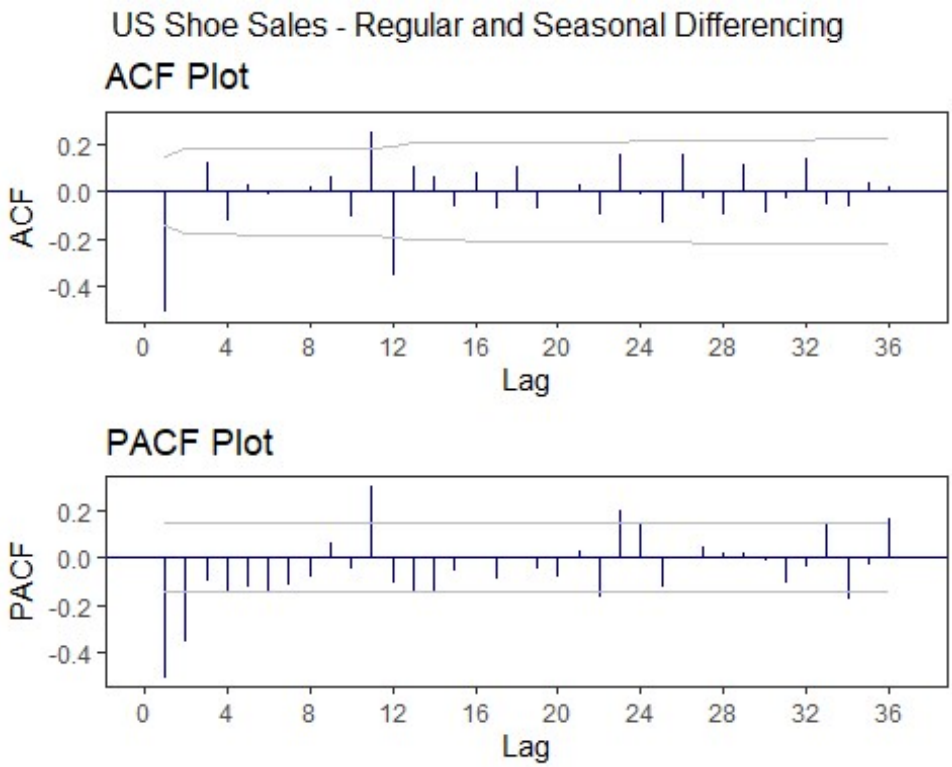

Figure 21: ACF and PACF Plot, Regular and Seasonal Differencing

Again, the individual plots are generated using the `plot_acf_matrix` function. They are put in facet format using the `ggarrange` function of the `ggpubr` package (Kassambara 2025).

3.4 plot_acf_matrix

The `plot_acf_matrix` function is the base function for all ACF and PACF plotting for functions described in Section 3.1 to 3.3. While this function is called from other ACF generating functions, it can be used on its own if the ACF or PACF matrix is extracted from a `seas` object using the `series` function in the `seasonal` package.

Figure 22 shows an ACF plot generated from the residuals extracted from a seas file provided as input. The plot is the same as Figure 16, except that the grid and background shading is present in Figure 22. Those options are applied within the `plot_acf` function.

```r
# Generate seas object for UKgas, saving acf plots
ukgas_x11_seas <-
   seasonal::seas(UKgas, series.period = 4,
                  arima.model = "(0 1 1)(0 1 1)",
                  transform.function = "log",
                  forecast.maxlead = 20,
                  x11.seasonalma = "s3x5",
                  check.print = c( 'pacf', 'pacfplot' ),
                  check.maxlag = 12,
                  check.save = c("acf", "pcf", "ac2"))

# Extract ACF matrix
ukgas_acf_matrix <- seasonal::series(ukgas_x11_seas, "acf")

# Generate ACF plot from extracted matrix
p_ukgas_acf_matrix   <-
   blsplotGG::plot_acf_matrix(ukgas_acf_matrix,
            main_title = "UK Gas ACF", this_frequency = 4,
            acf_color = "darkblue")

# Display ggplot2 object
print(p_ukgas_acf_matrix)
```

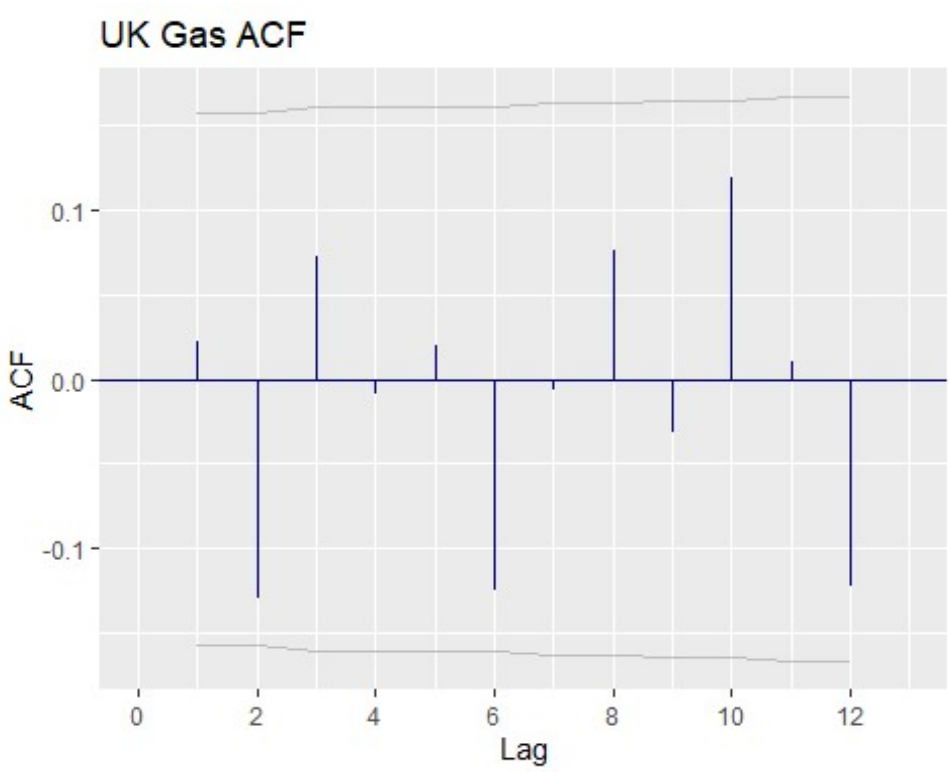

Figure 22: ACF Plot from Saved ACFs Example

3.5 plot_fcst

The `plot_fcst` function generates a plot of the forecasts generated from the regARIMA model. The plot will show the forecasts along with the upper and lower confidence intervals that appear in the X-13ARIMA-SEATS output generated by the `seasonal` package.

Figure 23 shows forecasts and confidence intervals for the Airline Passengers series from Box, Jenkins, and Reinsel (1976). The model used is the `ARIMA (0 1 1)(0 1 1)`. Since a subtitle was not specified, the function generates a subtitle containing the average absolute percentage error in within-sample forecasts (AAPE) for the last three years.

Note that using this function will likely generate a warning message like the one below:

```
Warning messages:
Removed 135 rows containing missing values or values outside the
scale range (`geom_line()`).
```

If you want to suppress this warning message, you can use the `suppressWarnings` function from base R.

```
# Generate seas object for Airline Passengers, saving forecasts
air_seas <-
   seasonal::seas(AirPassengers,
      arima.model = "(0 1 1)(0 1 1)",
      forecast.maxlead = 60, forecast.save = "fct",
      series.save = "a1")

# Generate forecast plot, with confidence intervals
air_fcst_p <-
   blsplotGG::plot_fcst(air_seas,
      main_title = "Forecasts for Airline Passengers",
      do_grid = TRUE)

# Display ggplot2 object, using the suppressWarnings function
# to suppress warning messages
suppressWarnings(print(air_fcst_p))
```

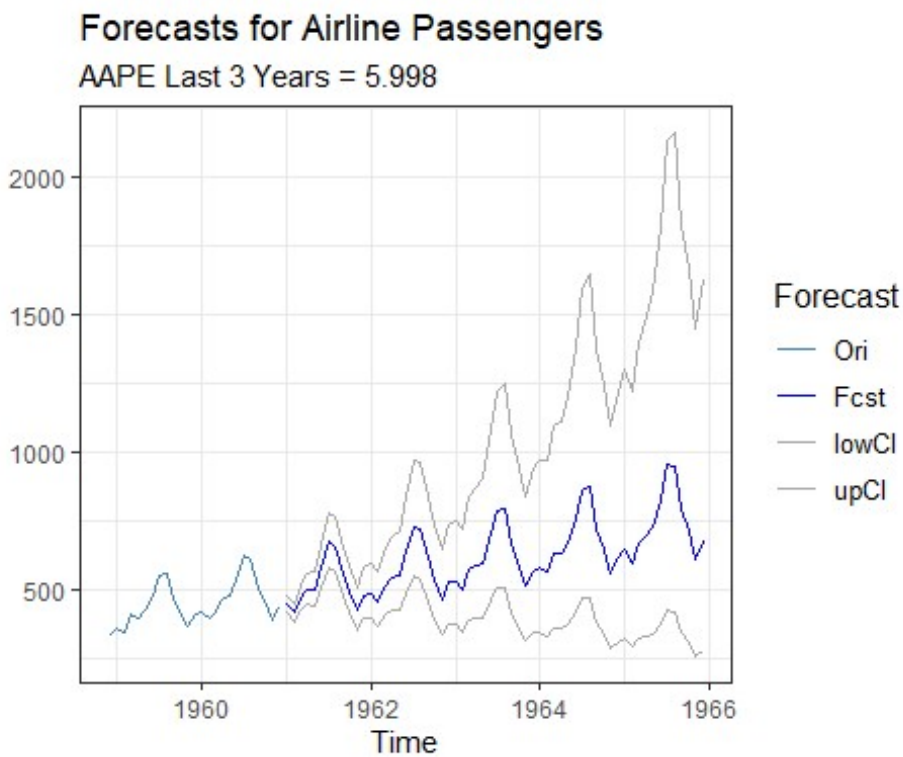

Figure 23: Forecast Plot Example

3.6 plot_fcst_two

The `plot_fcst_two` function generates a plot comparing forecasts from two regARIMA models. The upper and lower confidence interval of the two sets of forecasts are not shown.

Figure 24 compares the forecasts for two models fit to the Airline Passenger series - one with an ARIMA model of `(0 1 1)(0 1 1)`, and another with the same ARIMA model and trading day regressors.

Note that using this function will likely generate a warning message like the one below:

```
Warning messages:
Removed 110 rows containing missing values or values outside the
scale range (`geom_line()`).
```

If you want to suppress this warning message, you can use the `suppressWarnings` function from base R.

```r
# Generate seas object for US Shoe Sales, no regressors,
# saving forecasts
shoes_seas_one <-
   seasonal::seas(blsplotGG::shoes2008, x11="",
      slidingspans = "",
      transform.function = "log",
      arima.model = "(0 1 1)(0 1 1)",
      regression.aictest = NULL,
      outlier = NULL,
      forecast.maxlead = 60, forecast.save = "fct",
      check.print = c( 'pacf', 'pacfplot' ),
      series.save = "a1")

# Generate seas object for US Shoe Sales, trading day
# regressors and outlier identification, saving forecasts
shoes_seas_two <-
   seasonal::seas(blsplotGG::shoes2008, x11="",
      slidingspans = "",
      transform.function = "log",
      arima.model = "(0 1 1)(0 1 1)",
      regression.variables = c("td"),
      forecast.maxlead = 60, forecast.save = "fct",
      check.print = c( 'pacf', 'pacfplot' ),
      series.save = "a1")

# Generate forecast comparison plot
shoes_fcst_two_p <-
   blsplotGG::plot_fcst_two(shoes_seas_one, shoes_seas_two,
      main_title = "Forecast Comparison Plot",
      name_mdl1 = 'Airline',
      name_mdl2 = 'Airline + reg',
      do_grid = TRUE)

# Display ggplot2 object, using the suppressWarnings function
# to suppress warning messages
suppressWarnings(print(shoes_fcst_two_p))
```

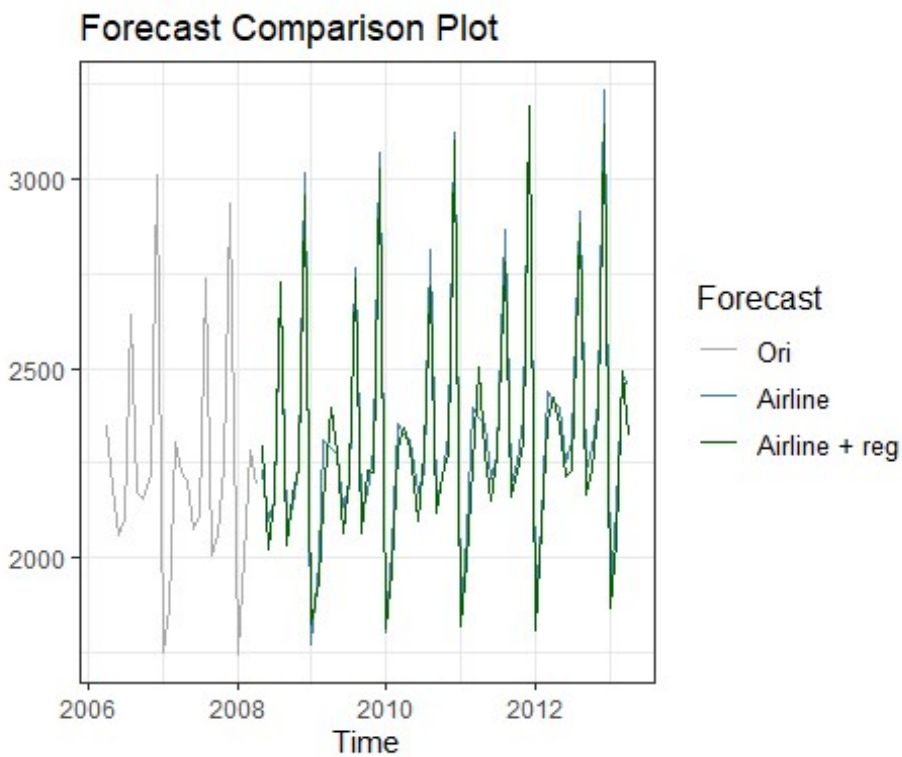

Figure 24: Forecast Comparison Plot Example

3.7 plot_first_difference

The `plot_first_difference` function shows a plot of the first differences of a given time series, with the differences grouped by month or quarter.

Figure 25 shows a first difference plot for the US Shoe Sales series. Each observation is plotted as the final digit in the year of the series.

This form of the first difference graphs was developed by Stuart Scott at the Bureau of Labor Statistics, where it has been used to detect outliers in the original time series. See Scott (1987) and Buszuwski and Scott (1988) for examples of using first difference graphs to identify different types of outliers.

```r
# Generate first difference plot of Shoe Sales series
p_shoes_sa_diff <-
    blsplotGG::plot_first_difference(blsplotGG::shoes2008,
        main_title = "First Differences Plot for Shoe Sales",
        remove_y_axis = FALSE,
        x_title_size = 10,
        add_zero_line = TRUE)

# Display ggplot2 object
print(p_shoes_sa_diff)
```

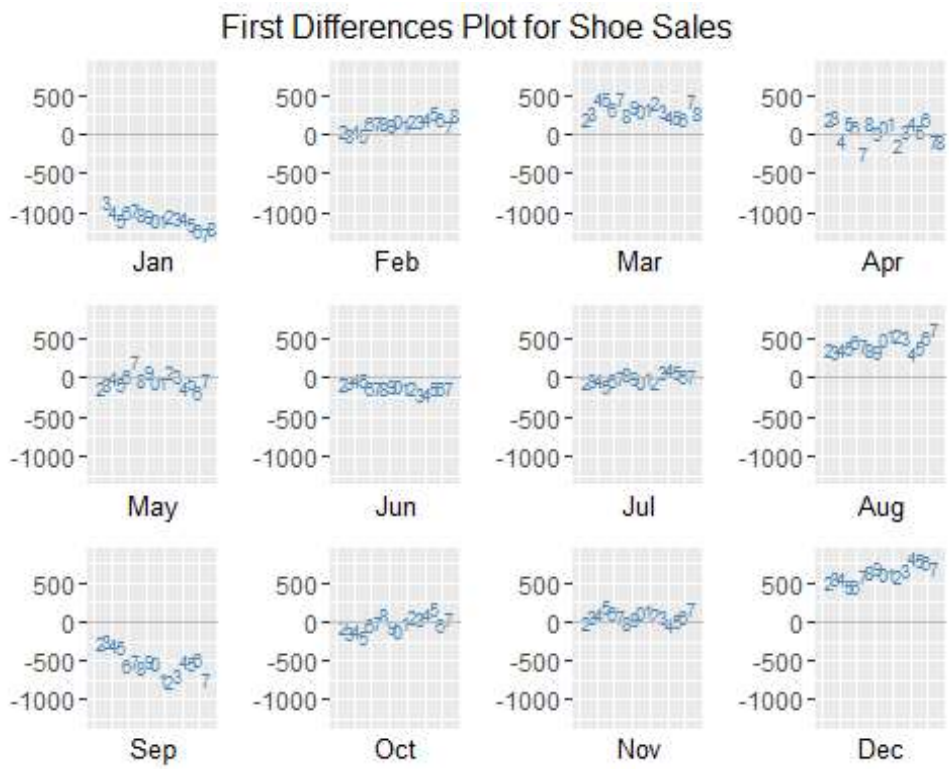

Figure 25: First Difference Plot Example

3.8 plot_fts

The `plot_fts` function generates a plot of the final t-statstics for the automatic outlier identification procedure. The critical value used to flag outliers is included in the plot.

Figure 26 shows a default plot of the t-statistics. Each type of outlier is plotted with a different color, generated by the `Dark2` `RColorBrewer` palette (Neuwirth 2022). The critical value used in the outlier identification procedure is given in the subtitle and is shown by a horizonal line in the plot.

```r
# Generate seas object with automatic outlier information
# for Airline Passenger series
air_seas_outlier <-
    seasonal::seas(AirPassengers, arima.model = "(0 1 1)(0 1
1)",
                   outlier.save = "fts", outlier.types = "all")

# Generate plot of final outlier t-values
plot_fts_air <-
    blsplotGG::plot_fts(air_seas_outlier,
        main_title = "Outlier T-Values for Airline Passengers")

# Display ggplot2 object
print(plot_fts_air)
```

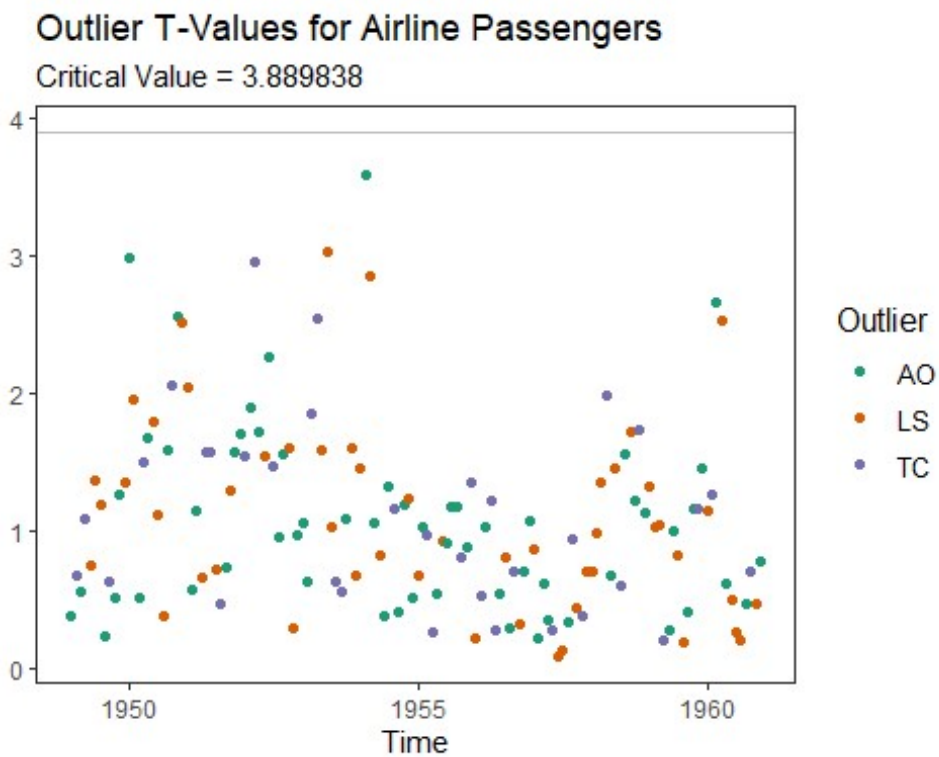

Figure 26: Outlier T-test Plot Example

Figure 27 shows a plot of the t-statistics where the t-statistic for the automatically identified outlier is included in the plot.

```
# Generate plot of final outlier t-values with identified
# outlier t-statistic included
plot_fts_air <-
    blsplotGG::plot_fts(air_seas_outlier,
        main_title = "Outlier T-Values for Airline Passengers",
        add_identified_outliers = TRUE)

# Display ggplot2 object
print(plot_fts_air)
```

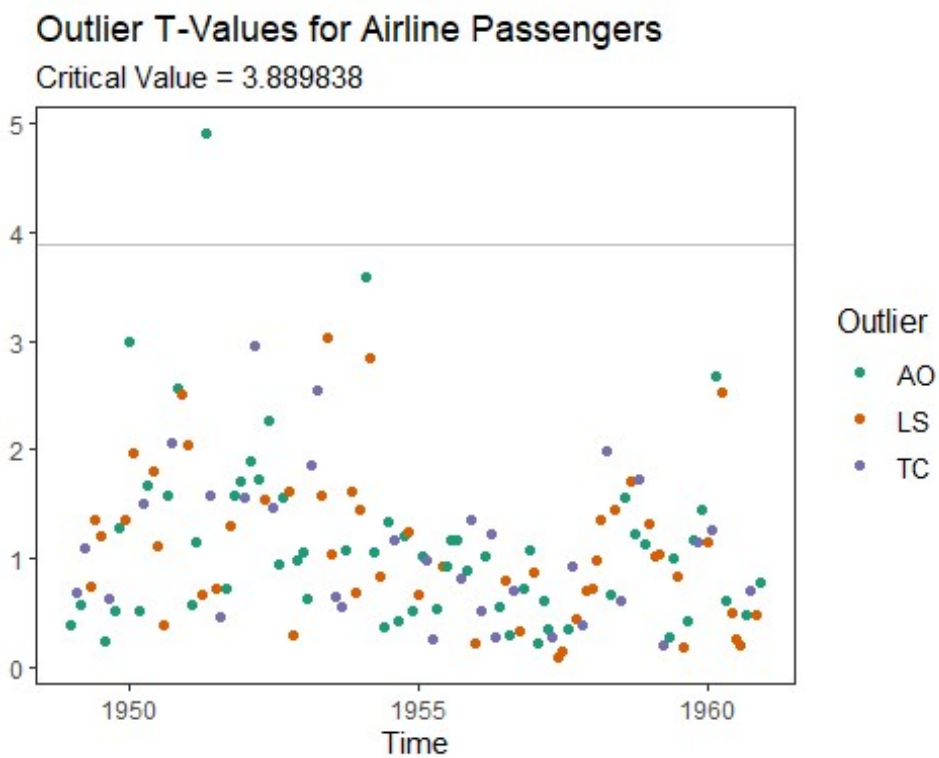

Figure 27: Outlier T-test Plot Example, With Identified Outliers

3.9 plot_resid

The `plot_resid` function produces a plot of the regARIMA residuals.

Figure 28 shows a high-density plot regARIMA residuals for the Airline Passenger series, similar to the `plot_ratio` function detailed in Section 2.3. Normality diagnostics appear in the plot subheader.

```r
# Generate seas object for Airline Passenger series
air_seas <-
    seasonal::seas(AirPassengers,
                   arima.model = "(0 1 1)(0 1 1)")

# Generate ratio plot of regARIMA model residuals,
# with normality statistics
plot_resid_air <-
   blsplotGG::plot_resid(air_seas,
      main_title = "regARIMA Residuals for Airline Passengers",
      use_ratio = TRUE,
      line_color="darkblue")

# Display ggplot2 object
print(plot_resid_air)
```

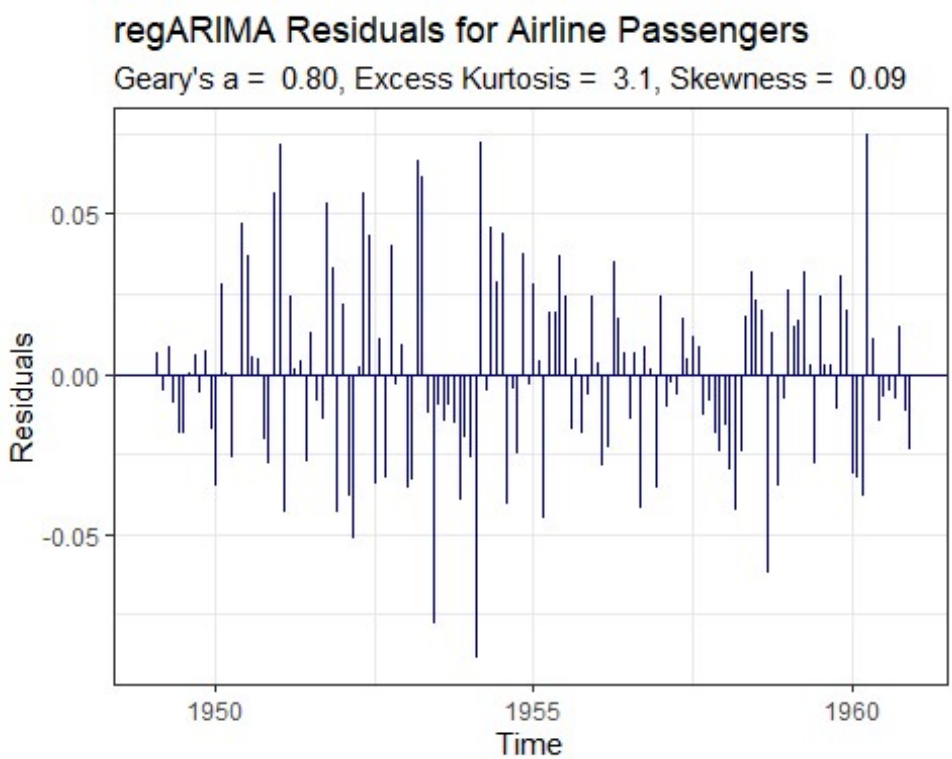

Figure 28: regARIMA Residual Plot Example

Figure 29 shows a residual plot for the Employed Males 20+ series from the US Bureau of Labor Statistics. This series is included in the `blsplotGG` package. This version of the plot is not a ratio plot - the `use_ratio` argument allows the user to specify whether a ratio plot is produced.

```r
# Generate seas object for BLS Employed Males aged 20+ series
n2000025_seas <-
   seasonal::seas(blsplotGG::employment_list$n2000025,
      x11 = "", transform.function = "log",
      check.print = c("none", "+acf", "+acfplot",
                      "+normalitytest"),
      regression.aictest = NULL,
      outlier.types = "all",
      arima.model = "(0 1 1)(0 1 1)",
      forecast.maxlead = 60
)

# Generate line plot of regARIMA model residuals,
# with normality statistics
n2000025_resid_plot <-
    blsplotGG::plot_resid(n2000025_seas,
        main_title = 'ARIMA Residuals for Employed Males 20+',
        use_ratio = FALSE,
        line_color='steelblue')

# Display ggplot2 object
print(n2000025_resid_plot)
```

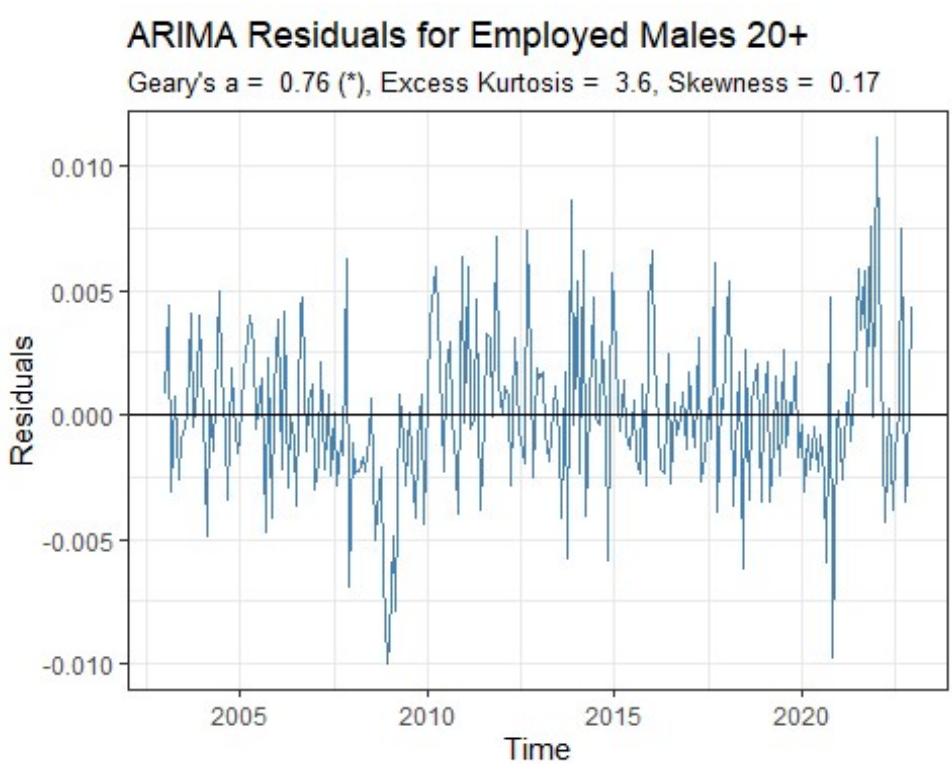

Figure 29: regARIMA Residual Plot Example (Line Plot)

Note that the sub header has a smaller font than Figure 28, and there is an asterisk next to the value for Geary's `a`. This shows that the diagnostic is significant at the one percent level, which indicates the residuals do not follow a normal distribution.

4 Seasonal Adjustment Plots

This section shows 16 functions that produce plots that are useful for analysts in comparing seasonal adjustments, either between different options or different adjustment methods or components of a given seasonal adjustment.

4.1 plot_component_facet

The `plot_component_facet` function generates a facet plot of the trend, seasonal, and irregular components from the seasonal adjustment encapsulated in a `seas` object.

Figure 30 shows the basic components of an X-11 seasonal adjustment of the US Employed Males aged 20 and over series.

```
# Generate seas object for BLS Employed Males aged 20+ series
n2000025_x11_seas <-
    seasonal::seas(blsplotGG::employment_list$n2000025,
        arima.model = "(0 1 3)(0 1 1)",
        transform.function = "log",
        forecast.maxlead = 36,
        regression.aictest = NULL,
        regression.save = c("td", "hol"),
        x11.seasonalma = "s3x9",
        check.print = c( 'pacf', 'pacfplot' ))

# Generate facet plot for the components from the seasonal
# adjustment of BLS Employed Males aged 20+ series
n2000025_comp_plot <-
    blsplotGG::plot_component_facet(n2000025_x11_seas,
        main_title = "US Employed Males 20+",
        sub_title = "X-11 (3x9 Seasonal filter)")

# Display ggplot2 object
print(n2000025_comp_plot)
```

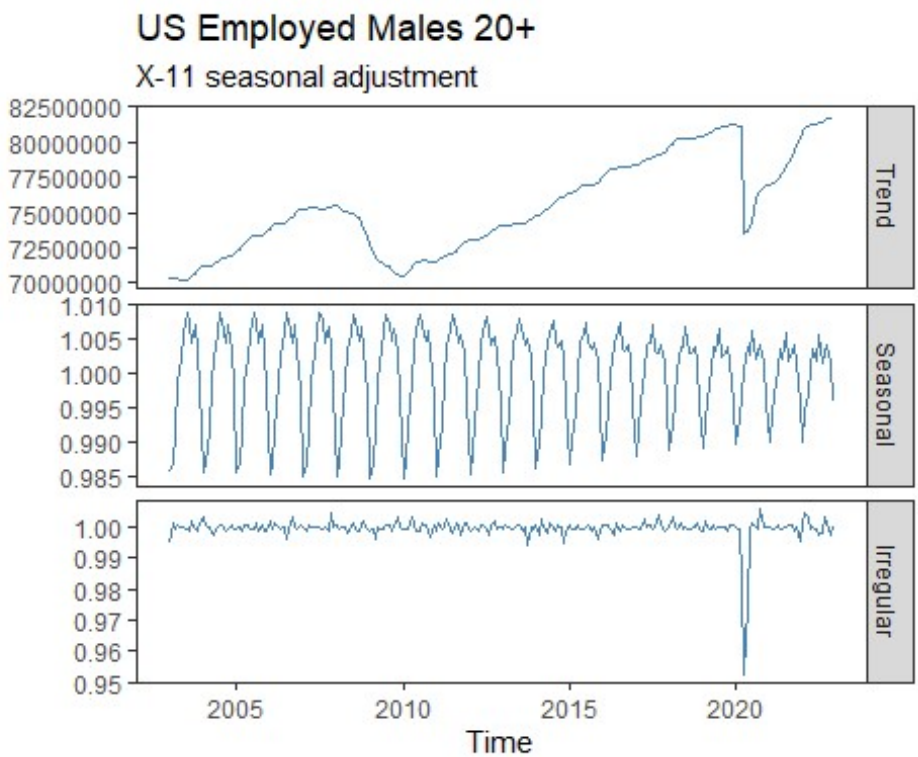

Figure 30: Seasonal Adjustment Component Facet Plot Example

For this function, the components are limited to the three main components (trend, seasonal, and irregular); you can generate plots with other factors like trading day, holiday, or outlier factors with the `plot_ratio_facet` function detailed in Section 4.3.

4.2 plot_irregular_boxplot

The `plot_irregular_boxplot` function produces a boxplot of the irregular component extracted from a 'seas' object. The user specifies which table from the X-13 output is used; possible tables include `"irregular"`, `"d13"`, `"s13"`, `"modirregular"`, `"e3"`, `"irregularoutlieradj"`, and "`se3`".

Figure 31 shows the boxplot for the X-11 irregular component for the US Retail Shoe Sales series.

```r
# Generate seas object for US Retail Shoe Sales, saving the
# X-11 irregular component
shoes_seas <-
   seasonal::seas(blsplotGG::shoes2008,
      x11.save = "d13", x11 = "",
      transform.function = "log",
      regression.aictest = c("td", "easter"),
      regression.save = c("td", "hol"),
      outlier.types = "all",
      arima.model = "(0 1 1)(0 1 1)",
      forecast.maxlead = 60)

# Generate irregular box plots for Shoe Sales series
p_bp1 <-
   blsplotGG::plot_irregular_boxplot(shoes_seas, "d13",
      main_title = "X-11 Irregular by Month",
      sub_title = "US Shoe Sales",
      line_color = "steelblue")

# Display ggplot2 object
print(p_bp1)
```

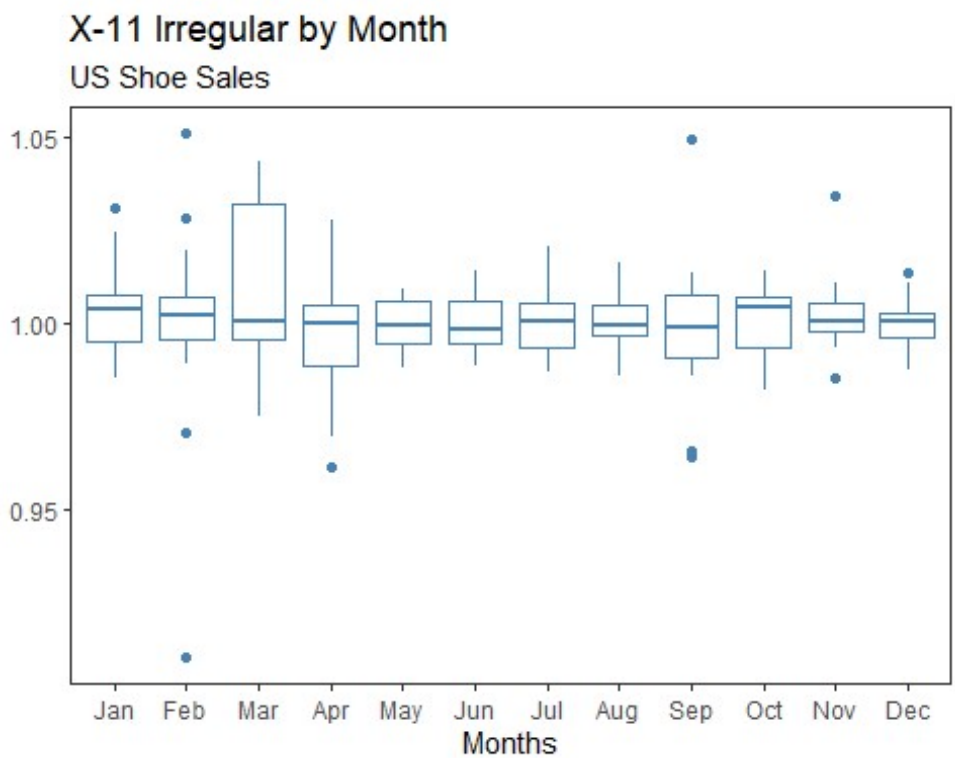

Figure 31: X-11 Irregular Boxplot Example

There are options to generate a jittered boxplot - this is a boxplot with the randomly displaced observations overlaid. Using jittered points within a boxplot can reveal the density and spread of the individual observations.

Figure 32 uses the `add_jitter` option to create a jittered boxplot of the modified irregular from a SEATS seasonal adjustment of the US Employed Males 20+ series. In this case, the `jitter_degree` argument is used to set the degree of jittering.

```
# Generate seas object for US Employed Males aged 20+ with
# SEATS seasonal adjustment
n2000025_seats_seas <-
    seasonal::seas(blsplotGG::employment_list$n2000025,
        arima.model = "(0 1 1)(0 1 1)",
        transform.function = "log",
        forecast.maxlead = 36,
        regression.aictest = NULL,
        seats.save = "se3",
        check.print = c( 'pacf', 'pacfplot' ))


# Generate jittered box plot of the SEATS Outlier Adjusted
# irregular
p_bp2_jitter <-
   blsplotGG::plot_irregular_boxplot(n2000025_seats_seas,
        this_series = "se3",
        main_title = "SEATS Outlier Adj Irregular by Month",
        sub_title = "US Employed Males 20+",
        line_color = "steelblue",
        add_jitter = TRUE,
        jitter_degree = 0.25)

# Display ggplot2 object
print(p_bp2_jitter)
```

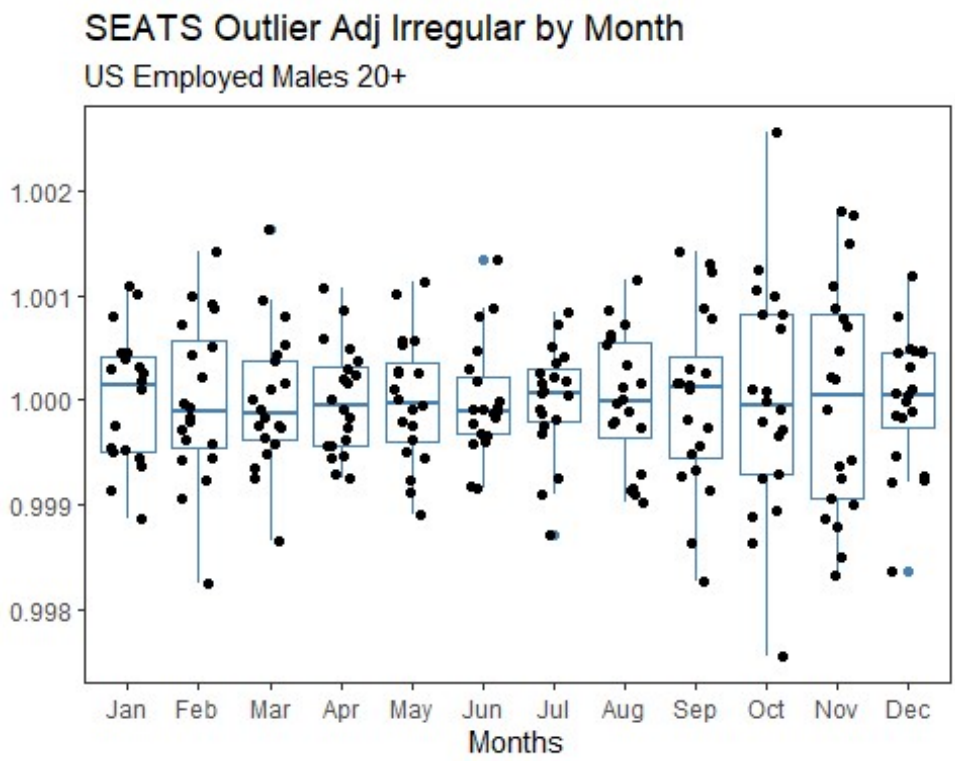

Figure 32: SEATS Irregular Boxplot, With Jitter, Example

4.3 plot_ratio_facet

The `plot_ratio_facet` function generates a facet plot of a user-selected list of factors, which are extracted from a `seas` object provided by the user. Users can choose between the following possible entries: `"sf"` (seasonal factor), `"td"` (trading day factor), `"hol"` (holiday factors), and `"irr"` (irregular).

The next two plots show plots of components from an X-11 seasonal adjustment of the US Retail Shoe Sales series.

Figure 33 shows the main seasonal adjustment components from the adjustment of the Shoe Sales series. Each facet has the same range for the y-axis, which is the function default. This shows that the range of the seasonal factors is much larger than for the other components (trading day and holiday factors).

Note that in the input for the `seas` function, the trading day and holiday factors are stored using the `regression.save` command to allow them to be easily extracted from the seas object. The seasonal factors are stored automatically.

```r
# Generate seas object for US Shoe Sales with X-11 seasonal
# adjustment
shoes_x11_seas <-
    seasonal::seas(blsplotGG::shoes2008,
        arima.model = "(0 1 1)(0 1 1)",
        transform.function = "log",
        forecast.maxlead = 36,
        regression.aictest = c("td", "easter"),
        regression.save = c("td", "hol"),
        x11.seasonalma = "s3x5",
        check.print = c( 'pacf', 'pacfplot' ))

# Generate facet plot of seasonal, trading day, and holiday
# factors
shoes_factor_ratio_one   <-
    blsplotGG::plot_ratio_facet(shoes_x11_seas,
        ratio_tables = c("sf", "td", "hol"),
        main_title = "US Retail Shoe Sales",
        sub_title = "X-11 Seasonal Adjustment Components")

# Display ggplot2 object
print(shoes_factor_ratio_one)
```

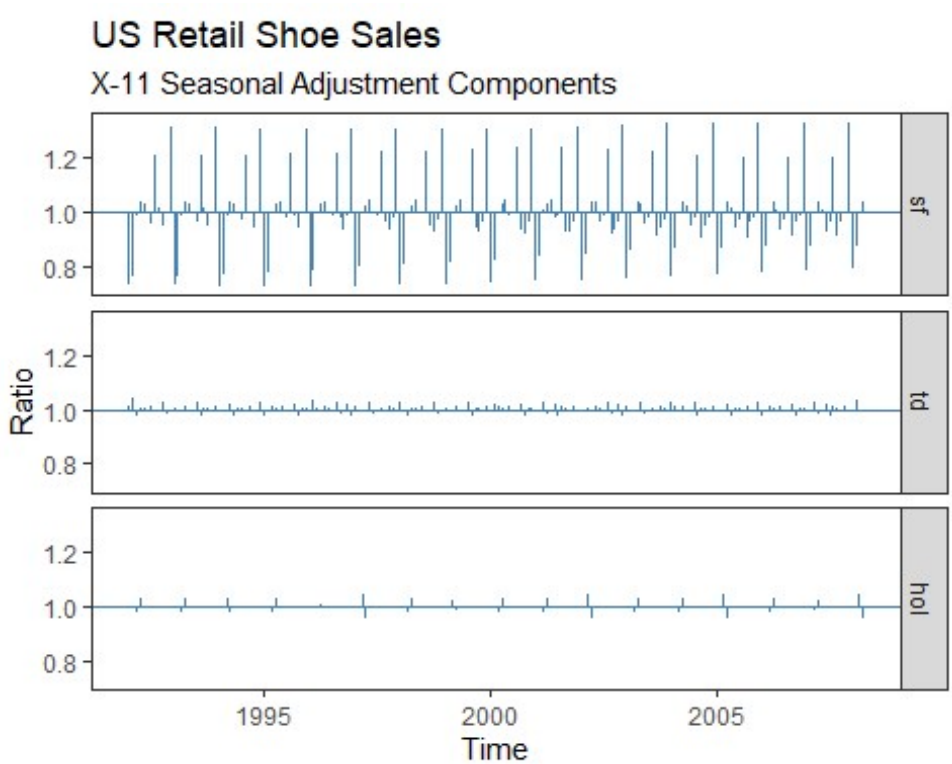

Figure 33: Ratio Facet Plot Example, Part 1

Figure 34 plots the same components, with each y-axis set to be different for the different facets by setting `this_scale = "free_y"`.

```
# Generate facet plot of seasonal, trading day, and holiday
# factors with each y-axis set to show its own scale
shoes_factor_ratio_two   <-
    plot_ratio_facet(shoes_x11_seas, c("sf", "td", "hol"),
        main_title = "US Retail Shoe Sales",
        sub_title = "X-11 Seasonal Adjustment Components",
        this_scale = "free_y")

# Display ggplot2 object
print(shoes_factor_ratio_two)
```

Figure 34: Ratio Facet Plot Example, Part 2

With this plot, we can see the structure of the holiday and trading day factors more clearly than in Figure 33.

4.4 plot_sadj_and_ori

The `plot_sadj_and_ori` is a function that produces a plot of the original series and the seasonally adjusted series. This is one of a few functions that produce plots of specific components of the seasonal adjustment.

Figure 35 shows a plot of the UK gas series with its X-11 seasonal adjustment. The user provides the time series (`ts`) vector for the original series and seasonally adjusted series.

```
# Generate seas object for UK Gas series with X-11 seasonal
# adjustment
ukgas_x11_seas   <-
    seasonal::seas(UKgas,
        series.period = 4,
        arima.model = "(0 1 1)(0 1 1)",
        transform.function = "log",
        forecast.maxlead = 20,
        x11.seasonalma = "s3x5",
        check.print = c( 'pacf', 'pacfplot' ))

# Extract seasonally adjusted series
ukgas_x11_sa     <- seasonal::final(ukgas_x11_seas)

# Generate plot of X-11 seasonal adjustment and the original
# series
ukgas_sa_ori_p <-
    blsplotGG::plot_sadj_and_ori(
        this_sa = ukgas_x11_sa,
        this_ori = UKgas,
        this_sa_type = "X-11",
        main_title = "UK Gas Seasonal Adjustment",
        line_color = c("lightgrey", "forestgreen"))

# Display ggplot2 object
print(ukgas_sa_ori_p)
```

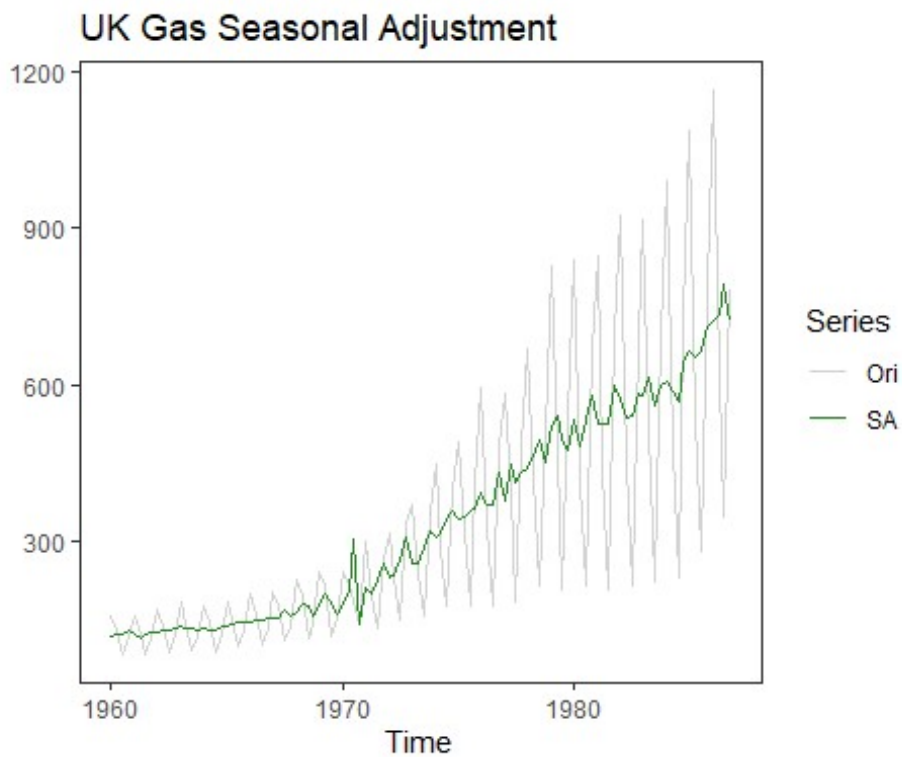

Figure 35: Original vs Seasonal Adjustment Plot Example

4.5 plot_sadj_and_trend

The `plot_sadj_and_trend` is a function that produces a plot of the trend component and the seasonally adjusted series extracted from a `seas` object. There is also an option to include the original series in the plot.

Figure 36 shows a plot of the trend component for the UK gas series with its X-11 seasonal adjustment. The original series is included in the plot by setting the `plot_ori` argument to `TRUE`. The colors used for the different lines are specified by the `line_color` argument.

```
# Generate seas object for UK Gas series with X-11 seasonal
# adjustment, with trend saved
ukgas_x11_seas   <-
    seasonal::seas(UKgas, series.period = 4,
        arima.model = "(0 1 1)(0 1 1)",
        transform.function = "log",
        forecast.maxlead = 20,
        x11.seasonalma = "s3x5",
        x11.save = "d12",
        check.print = c( 'pacf', 'pacfplot' ))
```

```r
# Generate plot with X-11 seasonal adjustment and trend, with
# UK Gas series included
ukgas_x11_sadj_and_trend_p <-
    blsplotGG::plot_sadj_and_trend(ukgas_x11_seas,
        plot_ori = TRUE,
        main_title = "UK Gas",
        sub_title = "X-11 Seasonal Adjustment",
        line_color = c("lightgrey", "steelblue", "forestgreen"))

# Display ggplot2 object
print(ukgas_x11_sadj_and_trend_p)
```

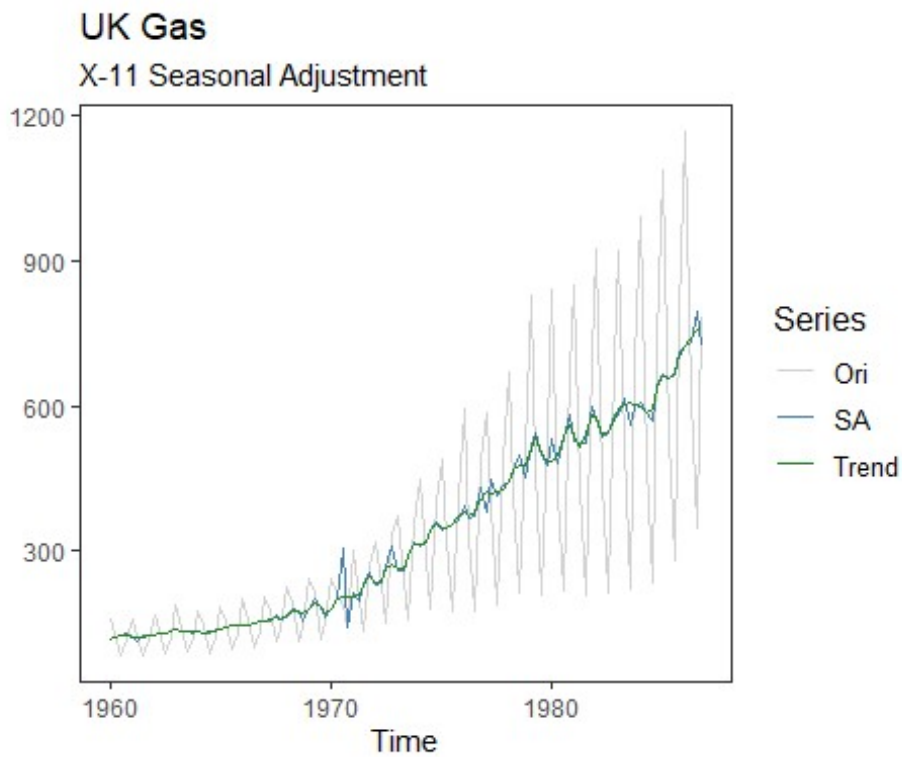

Figure 36: Seasonal Adjustment vs Trend (X-11) Plot Example

Figure 37 shows a plot of the trend component for the UK gas series with its SEATS seasonal adjustment. The original series is not included in the plot by setting the `plot_ori` argument to `FALSE`.

```r
# Generate seas object for UK Gas series with SEATS seasonal
# adjustment, with trend saved
ukgas_seats_seas   <-
   seasonal::seas(UKgas, series.period = 4,
        arima.model = "(0 1 1)(0 1 1)",
        transform.function = "log",
        forecast.maxlead = 20,
        seats.save = "s12",
        check.print = c( 'pacf', 'pacfplot' )
)
# Generate plot with SEATS seasonal adjustment and trend,
# without UK Gas series included
ukgas_seats_sadj_and_trend_p <-
    blsplotGG::plot_sadj_and_trend(ukgas_seats_seas,
        plot_ori = FALSE,
        main_title = "UK Gas Series",
        sub_title = "SEATS Seasonal Adjustment",
        line_color = c("steelblue", "forestgreen")
)
# Display ggplot2 object
print(ukgas_seats_sadj_and_trend_p)
```

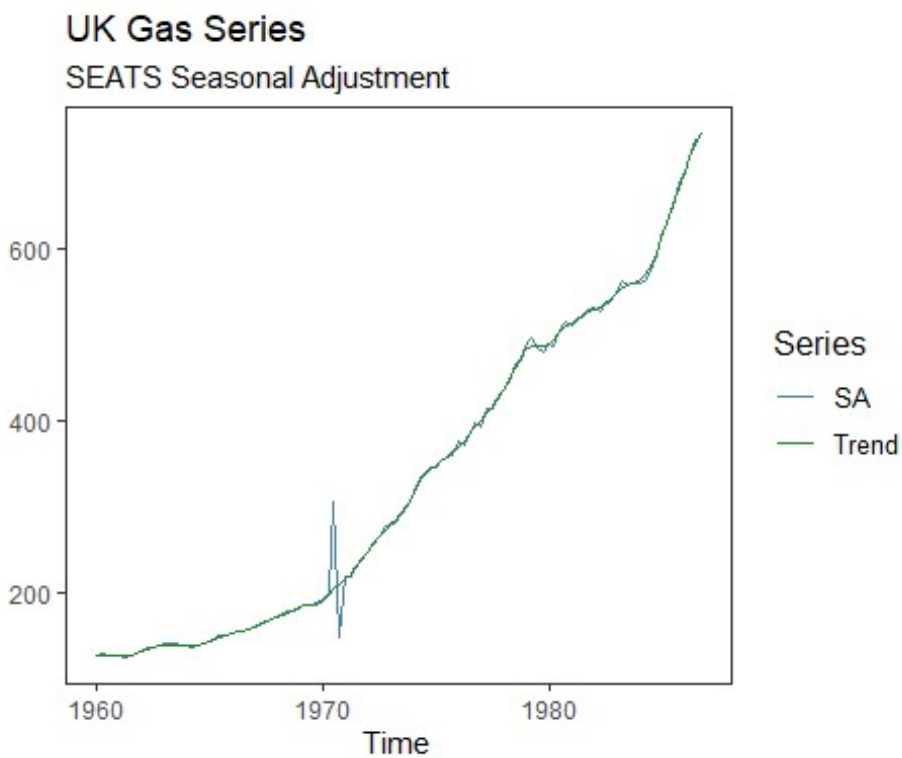

Figure 37: Seasonal Adjustment vs Trend (SEATS) Plot Example

4.6 plot_sadj_and_trend_facet

The `plot_sadj_and_trend_facet` is a function that produces a facet plot of the trend component and the seasonally adjusted series extracted from a `seas` object. There is also an option to include the original series in the plot.

Figure 38 shows a facet plot of the trend component for the UK gas series with its X-11 seasonal adjustment. The original series is included in the plot by setting the `plot_ori` argument to TRUE.

```
# Generate seas object for UK Gas series with X-11 seasonal
# adjustment, with trend saved
ukgas_x11_seas   <-
    seasonal::seas(UKgas, series.period = 4,
        arima.model = "(0 1 1)(0 1 1)",
        transform.function = "log",
        forecast.maxlead = 20,
        x11.seasonalma = "s3x5",
        x11.save = "d12",
        check.print = c( 'pacf', 'pacfplot' )
)

# Generate facet plot with X-11 seasonal adjustment and trend,
# with UK Gas series included
ukgas_x11_sadj_and_trend_facet_p <-
    plot_sadj_and_trend_facet(ukgas_x11_seas,
        plot_ori = TRUE,
        main_title = "UK Gas",
        sub_title = "X-11 Seasonal Adjustment",
        line_color = c("steelblue", "forestgreen", "grey")
)

# Display ggplot2 object
print(ukgas_x11_sadj_and_trend_facet_p)
```

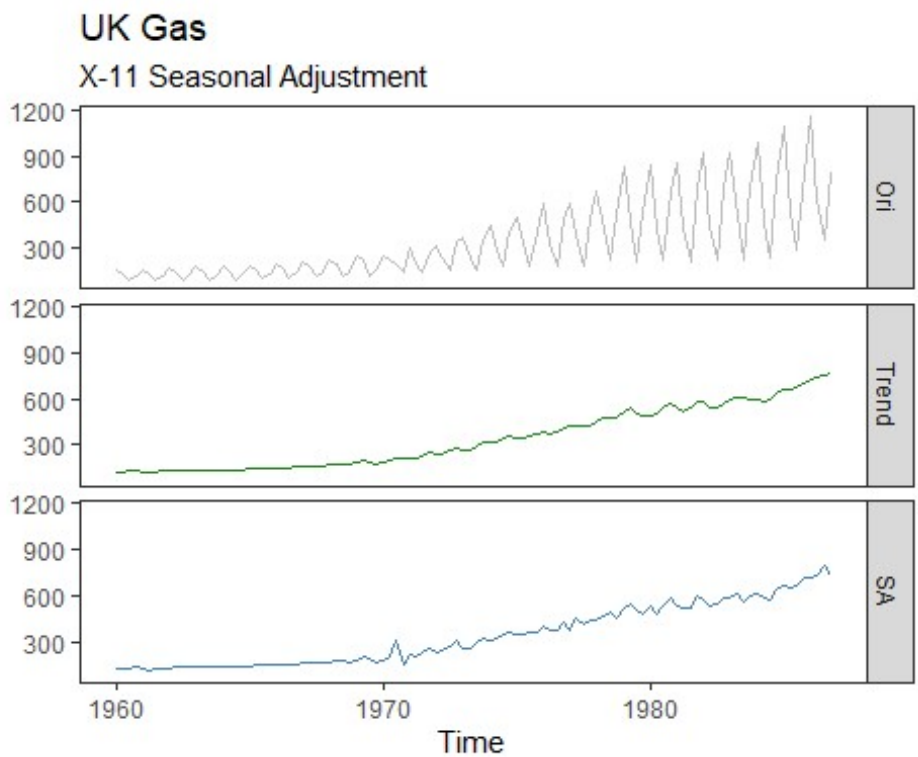

Figure 38: Seasonal Adjustment vs Trend (X-11) Plot Example

Figure 39 shows a facet plot of the trend component for the UK gas series with its SEATS seasonal adjustment. The original series is not included in the plot by setting the `plot_ori` argument to `FALSE`.

```
# Generate seas object for UK Gas series with SEATS seasonal
# adjustment, with trend saved
ukgas_seats_seas <-
    seasonal::seas(UKgas, series.period = 4,
        arima.model = "(0 1 1)(0 1 1)",
        transform.function = "log",
        forecast.maxlead = 20,
        seats.save = "s12",
        check.print = c( 'pacf', 'pacfplot' )
)
```

```
# Generate facet plot with SEATS seasonal adjustment and trend,
# without UK Gas series included
ukgas_seats_sadj_and_trend_facet_p <-
    blsplotGG::plot_sadj_and_trend_facet(ukgas_seats_seas,
        plot_ori = FALSE,
        main_title = "UK Gas Series",
        sub_title = "SEATS Seasonal Adjustment",
        line_color = c("steelblue", "forestgreen")
)

# Display ggplot2 object
print(ukgas_seats_sadj_and_trend_facet_p)
```

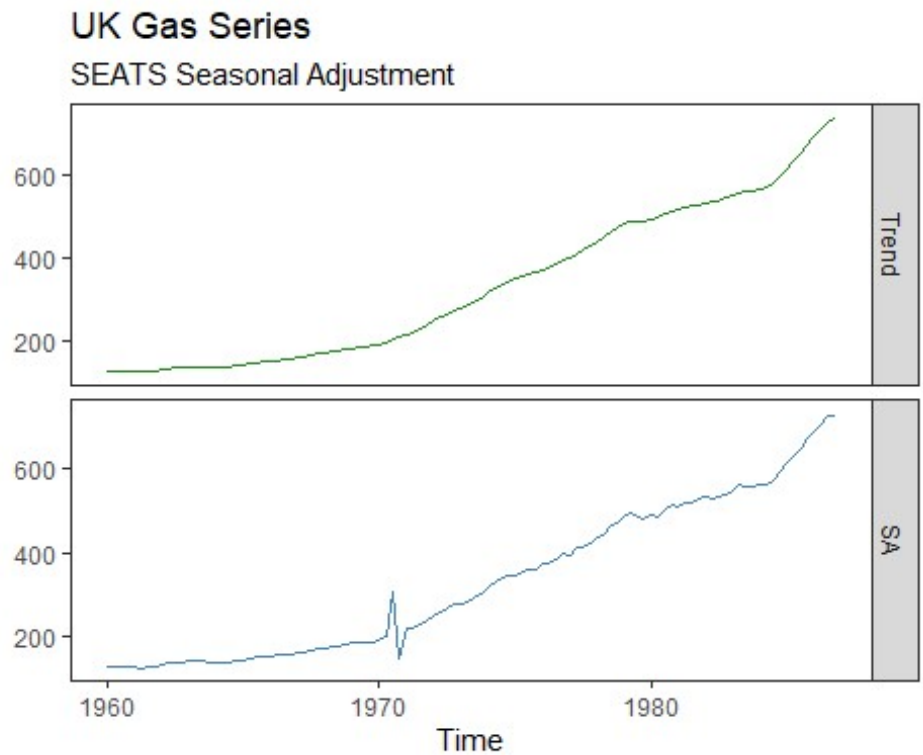

Figure 39: Seasonal Adjustment vs Trend (SEATS) Plot Example

4.7 plot_sf_mean

The `plot_sf_mean` function generates the monthly (or quarterly) means of the seasonal factors for a few seasonal adjustments. The plot is generated from a time series matrix with columns containing the seasonal factors from either the seasonal adjustments of several series or several seasonal adjustments of the same series.

Figure 40 shows the monthly means from the major components for US Employment. The seasonal factors are extracted from several `seas` objects for the US Employment series.

A note about the example shown: the four employment series are run using the same modeling and adjustment options using a list object with `ts` objects. More options for adjusting multiple time series with the `seasonal` package are given in Sax (2024).

```
# Generate list of seas objects for US Employment Series
EM_seas <-
    seasonal::seas(blsplotGG::employment_list,
        x11 = "", transform.function = "log",
        check.print = c("none", "+acf", "+acfplot",
                        "+normalitytest"),
        regression.aictest = NULL,
        outlier.types = "all",
        arima.model = "(0 1 1)(0 1 1)",
        forecast.maxlead = 60
)
EM_names <- names(employment_list)

# Use Filter function to grab seas objects
EM_seas_only <-
   Filter(function(x) inherits(x, "seas"), EM_seas)
names(EM_seas_only) <- EM_names

# Create time series matrix from extracted seasonal factors
EM_Comp_Sf <-
    cbind(seasonal::series(EM_seas_only$n2000013, "d10"),
          seasonal::series(EM_seas_only$n2000014, "d10"),
          seasonal::series(EM_seas_only$n2000025, "d10"),
          seasonal::series(EM_seas_only$n2000026, "d10"))
colnames(EM_Comp_Sf) <- EM_names

# Generate plot of monthly means of seasonal factors from
# employment data
em_plot <-
    blsplotGG::plot_sf_mean(EM_Comp_Sf,
        main_title = 'US Employment Seasonal Means',
        sub_title = 'X-11 Seasonals',
        forecast = 60,
        this_legend_title = "SF Means",
        this_legend_entry =
            c("M 16-19", "F 16-19", "M 20+", "F 20+"))

# Display ggplot2 object
print(em_plot)
```

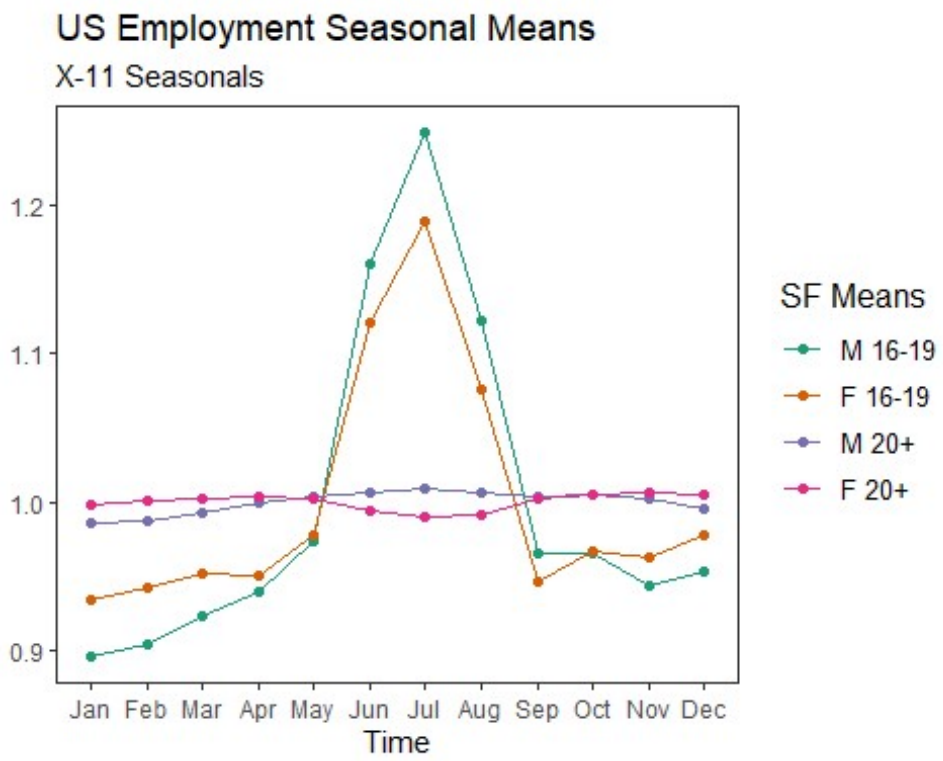

Figure 40: Seasonal Means Plot Example

Through this plot, you can see immediately the difference between the seasonality of the four series. The seasonal patterns for the two series with employment for Males and Females for workers between 16–19-year-olds are very different than those for the Male and Female employment for workers with an age of 20 plus.

Another way of showing this is in a facet plot. Using the `do_facet` argument creates a facet plot of the monthly (or quarterly) seasonal means, as in Figure 41.

```
# Generate a facet plot of monthly means of seasonal factors
# from employment data
em_plot_facet <-
    blsplotGG::plot_sf_mean(EM_Comp_Sf,
        main_title = 'US Employment Seasonal Means',
        sub_title = 'X-11 Seasonals',
        forecast = 60,
        do_facet = TRUE,
        this_scale = "fixed",
        this_legend_entry =
            c("M 16-19", "F 16-19", "M 20+", "F 20+"))

# Display ggplot2 object
print(em_plot_facet)
```

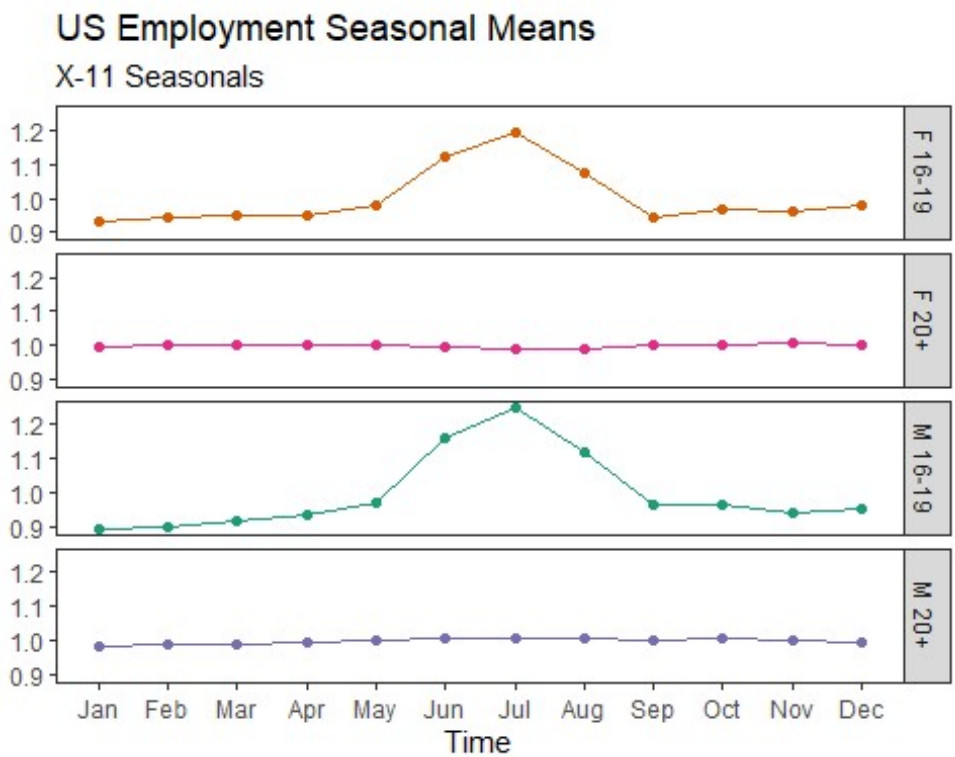

Figure 41: Sesasonal Means Facet Plot Example

4.8 plot_sf_series

The `plot_sf_series` function generates seasonal subplots of the seasonal factors. Here, the plot takes in the time series (`ts`) object of the seasonal factors, usually extracted from a `seas` object.

Figure 42 shows an example of a seasonal subplot for the Airline Passenger series. This shows monthly subplots of the seasonal factors, with lines for the mean of each month imposed on the individual subplots.

```
# Generate seas object for the Airline Passenger Series
air_seas    <-
    seasonal::seas(AirPassengers,
        arima.model = '(0 1 1)(0 1 1)',
        x11 = '')

# Extract seasonal factors for the Airline Passenger Series
air_sf      <- seasonal::series(air_seas, 'd10')
```

```r
# Generate seasonal factor plot
plot_air_sf <-
    blsplotGG::plot_sf_series(air_sf,
        add_mean_line = TRUE,
        main_title = 'Air Passengers Seasonal Sub-Plots',
        this_color_sf = 'darkgreen',
        this_color_mean = 'lightgreen',
        this_legend_title = 'X-11 Seasonal',
        this_legend_text = c('sf', 'mean'),
        this_legend_position = "bottom",
        this_legend_space = 0)

# Display ggplot2 object
print(plot_air_sf)
```

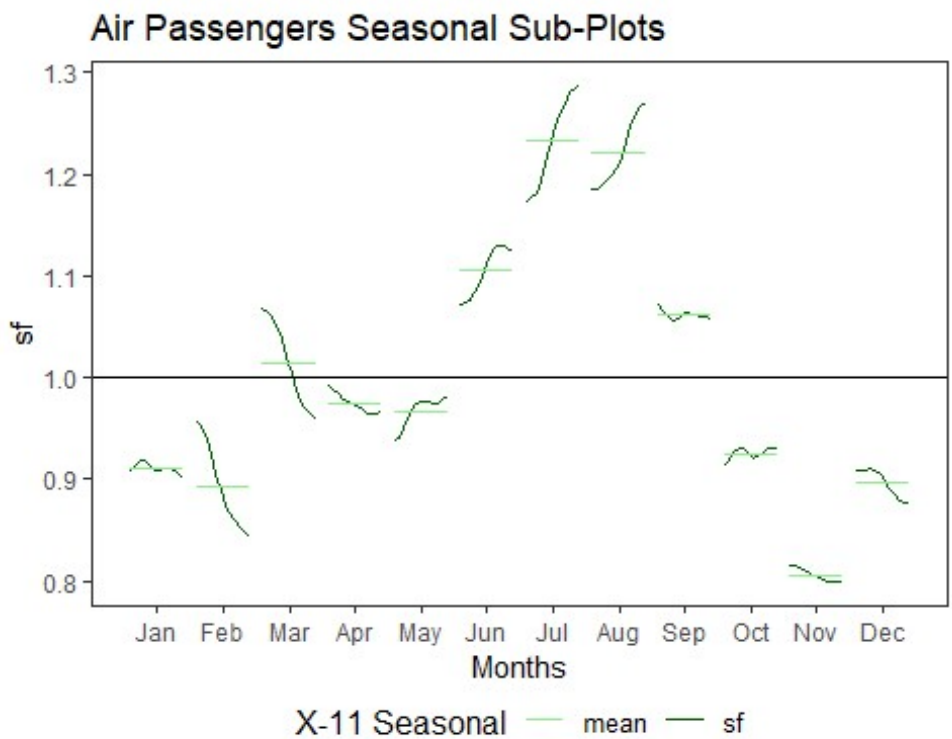

Figure 42: Seasonal Plot Example

4.9 plot_sf_two

The `plot_sf_two` function generates a comparison seasonal subplot of two sets of seasonal factors plotted for each month (or quarter). These are assumed to be seasonal factors generated from the same time series.

These seasonal factors could be from two different seasonal adjustment methods applied to the same series, or different estimates from the same method.

Figure 43 shows an example of a seasonal comparison subplot for the Airline Passenger series between factors from the X-11 and SEATS seasonal adjustment methods. This shows monthly subplots of both sets of seasonal factors, with lines for the mean of each month imposed on the individual subplots for each seasonal factor.

```
# Generate seas object for the X-11 adjustment of the
# Airline Passenger Series
air_seas_x11    <-
    seasonal::seas(AirPassengers,
        arima.model = '(0 1 1)(0 1 1)',
        x11 = '')

# Generate seas object for the SEATS adjustment of the
# Airline Passenger Series
air_seas_seats  <-
    seasonal::seas(AirPassengers,
        arima.model = '(0 1 1)(0 1 1)',
        seats.save = "s10")

# Extract seasonal factors
air_sf_x11      <- seasonal::series(air_seas_x11, 'd10')
air_sf_seats    <- seasonal::series(air_seas_seats, 's10')

# Generate seasonal factor plot comparing X-11 and SEATS
# factors
plot_air_sf_two <-
    blsplotGG::plot_sf_two(air_sf_x11, air_sf_seats,
        main_title = 'Air Passengers Seasonal Sub-Plots',
        this_legend_text = c('X-11 sf', 'X-11 mean',
                             'SEATS sf', 'SEATS mean'),
        this_legend_title_size = 10,
        this_legend_text_size = 8)

# Display ggplot2 object
print(plot_air_sf_two)
```

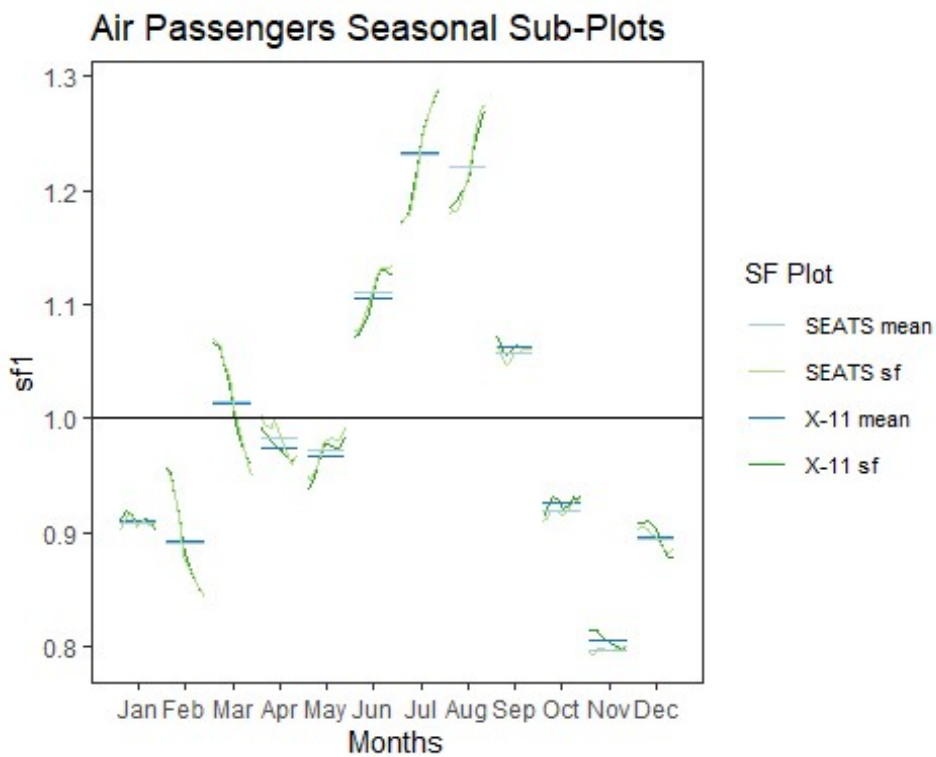

Figure 43: Seasonal Comparison Plot Example

4.10 plot_two_sa

The `plot_two_sa` function compares two seasonal adjustments, usually from the same time series. These can be from different adjustment methods, or different options or models with the same adjustment method.

The user can optionally include the original series in the plot.

Figure 44 gives an example of such a comparison plot, in this case comparing X-11 and SEATS seasonal adjustments for the UK Gas series.

```r
# Generate seas object for the X-11 adjustment of the
# UK Gas Series
ukgas_x11_seas   <-
    seasonal::seas(UKgas,
        series.period = 4,
        arima.model = "(0 1 1)(0 1 1)",
        transform.function = "log",
        forecast.maxlead = 20,
        x11.seasonalma = "s3x5",
        check.print = c( 'pacf', 'pacfplot' )
)

# Generate seas object for the X-11 adjustment of the
# UK Gas Series
ukgas_seats_seas <-
    seasonal::seas(UKgas,
        series.period = 4,
        arima.model = "(0 1 1)(0 1 1)",
        transform.function = "log",
        forecast.maxlead = 20,
        check.print = c( 'pacf', 'pacfplot' ))

# Extract seasonally adjusted series
ukgas_x11_sa     <- seasonal::final(ukgas_x11_seas)
ukgas_seats_sa   <- seasonal::final(ukgas_seats_seas)

# Generate plot comparing X-11 and SEATS seasonal adjustments
ukgas_seats_v_x11_p <-
    blsplotGG::plot_two_sa(this_sa_one = ukgas_x11_sa,
        this_sa_two = ukgas_seats_sa,
        main_title = "UK Gas Seasonal Adjustments",
        sub_title = "X-11 - Blue, SEATS - Green",
        line_color = c("steelblue", "forestgreen")
)

# Display ggplot2 object
print(ukgas_seats_v_x11_p)
```

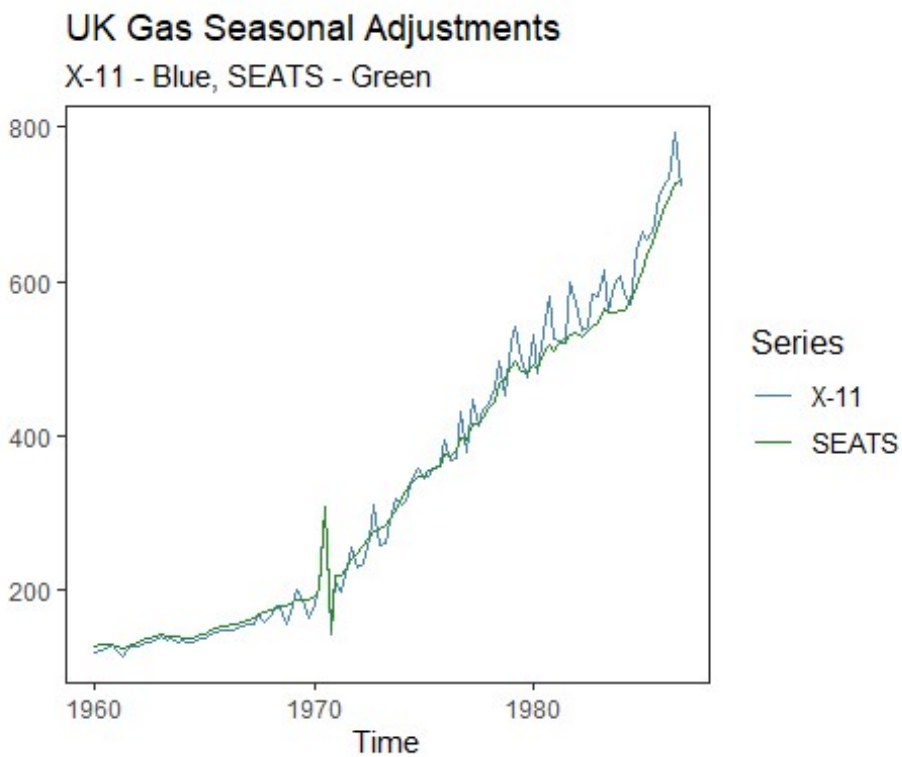

Figure 44: Seasonal Adjustment Comparison Plot Example

Note how toward the end of the series the SEATS adjustment is below that of the X-11 adjustment. The SEATS adjustment appears much smoother than the X-11 adjustment as well.

Figure 45 gives an example of another comparison plot, comparing X-11 and SEATS seasonal adjustments for the US Employment for Males 20 and over series. The original series is included in this plot in grey.

```
# Generate seas object for the SEATS adjustment of the
# US Employment for Males 20 and over
n2000025_seats_seas <-
    seasonal::seas(blsplotGG::employment_list$n2000025,
        arima.model = "(0 1 1)(0 1 1)",
        transform.function = "log",
        forecast.maxlead = 36,
        regression.aictest = NULL,
        seats.save = "s11",
        check.print = c( 'pacf', 'pacfplot' )
)
```

```r
# Generate seas object for the X-11 adjustment of the
# US Employment for Males 20 and over
n2000025_x11_seas <-
    seasonal::seas(blsplotGG::employment_list$n2000025,
        arima.model = "(0 1 1)(0 1 1)",
        transform.function = "log",
        forecast.maxlead = 36,
        regression.aictest = NULL,
        x11.save = "d11",
        check.print = c( 'pacf', 'pacfplot' )
)

# Extract seasonally adjusted series
n2000025_x11_sa     <-
   seasonal::final(n2000025_x11_seas)
n2000025_seats_sa   <-
   seasonal::final(n2000025_seats_seas)

# Generate plot comparing X-11 and SEATS seasonal adjustments
n2000025_seats_v_x11_p <-
    blsplotGG::plot_two_sa(
        this_sa_one = n2000025_x11_sa,
        this_sa_two = n2000025_seats_sa,
        this_ori = blsplotGG::employment_list$n2000025,
        main_title = "US Employment Male 20+",
        this_legend_title_size = 10,
        this_legend_text_size = 8,
        this_legend_position = "bottom",
        this_legend_space = 0,
        line_color = c("grey", "blue", "green")
)

# Display ggplot2 object
print(n2000025_seats_v_x11_p)
```

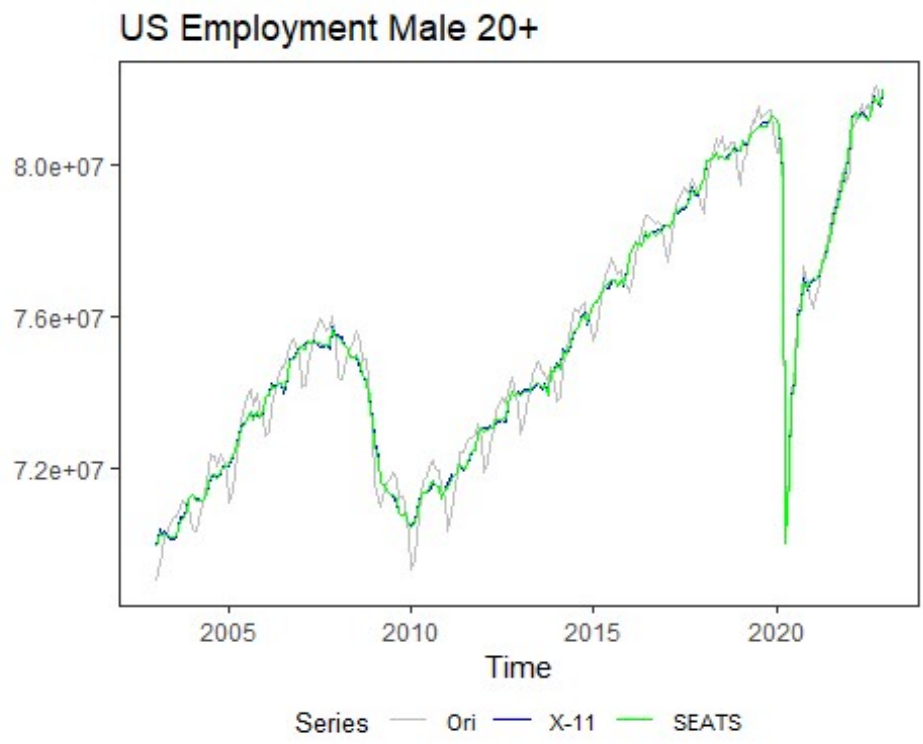

Figure 45: Seasonal Adjustment Comparison Plot, With Original Series, Example

Note that it is very difficult to make comparisons during the Covid pandemic around 2020.

4.11 plot_two_sa_comparison

the `sec-plot-two-sa-comparison` function is one of two functions that generate facet style plots highlighting two seasonal adjustments. In this case, a separate ratio/difference plot is produced to show a direct comparison between the two seasonal adjustments.

Figure 46 shows a comparison between the X-11 and SEATS seasonal adjustments for the UK Gas Series.

```
# Generate seas object for the X-11 adjustment of the
# UK Gas Series
ukgas_x11_seas   <-
    seasonal::seas(UKgas,
        series.period = 4,
        arima.model = "(0 1 1)(0 1 1)",
        transform.function = "log",
        forecast.maxlead = 20,
        x11.seasonalma = "s3x5",
        check.print = c( 'pacf', 'pacfplot' ))
```

```r
# Generate seas object for the SEATS adjustment of the
# UK Gas Series
ukgas_seats_seas <-
    seasonal::seas(UKgas,
        series.period = 4,
        arima.model = "(0 1 1)(0 1 1)",
        transform.function = "log",
        forecast.maxlead = 20,
        check.print = c( 'pacf', 'pacfplot' )
)

# Extract seasonally adjusted series
ukgas_x11_sa     <-
   seasonal::final(ukgas_x11_seas)
ukgas_seats_sa   <-
   seasonal::final(ukgas_seats_seas)

# Generate a facet-style plot comparing SEATS and X-11 seasonal
# adjustments, with a comparison panel showing the ratio of the
# two adjustments.
ukgas_seats_v_x11_p <-
    plot_two_sa_comparison(
         this_sa_one = ukgas_x11_sa,
         this_sa_two = ukgas_seats_sa,
         overall_title = "UK Gas Production",
         sa_one_title = "X-11 Seasonal Adjustment",
         sa_two_title = "SEATS Seasonal Adjustment",
         comparison_title =
            "Ratio of Seasonal Adjustments (X11 / SEATS)")

# Display ggplot2 object
print(ukgas_seats_v_x11_p)
```

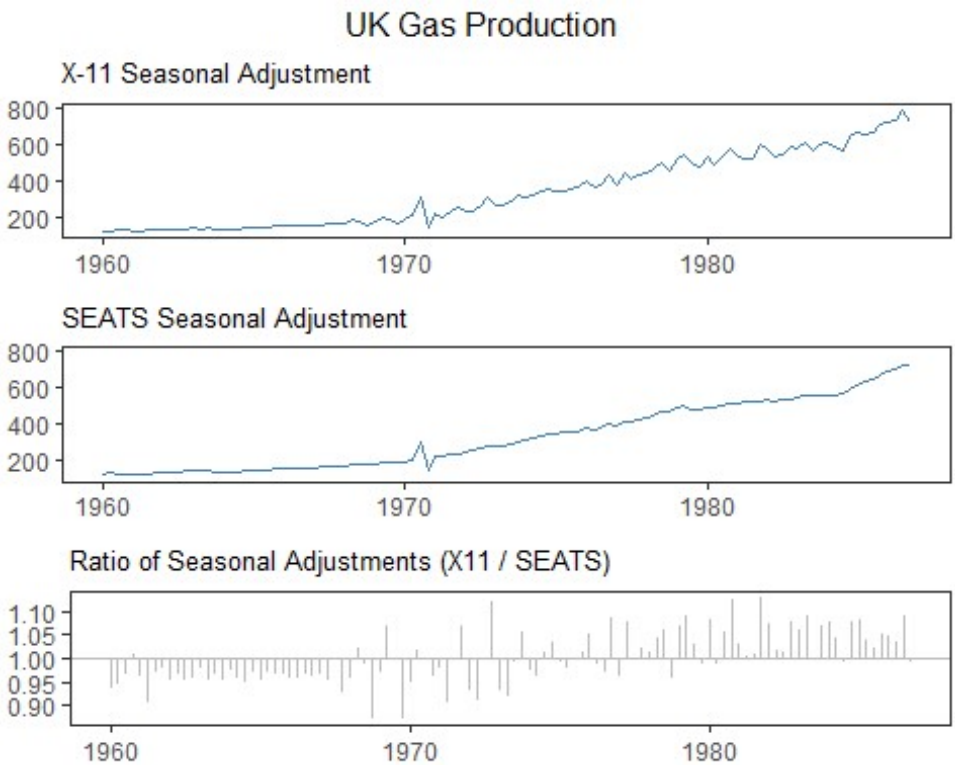

Figure 46: Seasonal Adjustment Comparison Plot Example, With Comparison Panel

4.12 plot_two_sa_facet

The `plot_two_sa_facet` function is like the `plot_two_sa_comparison` function in Section 4.11, except this is a true facet plot. You can add the original series as a panel of the facet plot, but you cannot add a comparison panel as in Figure 46.

Figure 47 generates a facet plot for the UK Gas series comparing the X-11 and SEATS seasonal adjustment.

```
# Generate seas object for the X-11 adjustment of the
# UK Gas Series
ukgas_x11_seas   <-
    seasonal::seas(UKgas,
        series.period = 4,
        arima.model = "(0 1 1)(0 1 1)",
        transform.function = "log",
        forecast.maxlead = 20,
        x11.seasonalma = "s3x5",
        check.print = c( 'pacf', 'pacfplot' )
)
```

```r
# Generate seas object for the SEATS adjustment of the
# UK Gas Series
ukgas_seats_seas <-
    seasonal::seas(UKgas,
        series.period = 4,
        arima.model = "(0 1 1)(0 1 1)",
        transform.function = "log",
        forecast.maxlead = 20,
        check.print = c( 'pacf', 'pacfplot' )
)

# Extract seasonally adjusted series
ukgas_x11_sa     <-
   seasonal::final(ukgas_x11_seas)
ukgas_seats_sa   <-
   seasonal:final(ukgas_seats_seas)

# Generate a facet plot comparing SEATS and X-11 seasonal
# adjustments, without a comparison panel.
ukgas_two_sa_facet_p <-
   blsplotGG::plot_two_sa_facet(this_sa_one = ukgas_x11_sa,
      this_sa_two = ukgas_seats_sa,
      this_ori = UKgas,
      main_title = "UK Gas Seasonal Adjustments",
      line_color = "forestgreen",
      this_sa_text = c("X-11", "Seats")
)

# Display ggplot2 object
print(ukgas_two_sa_facet_p)
```

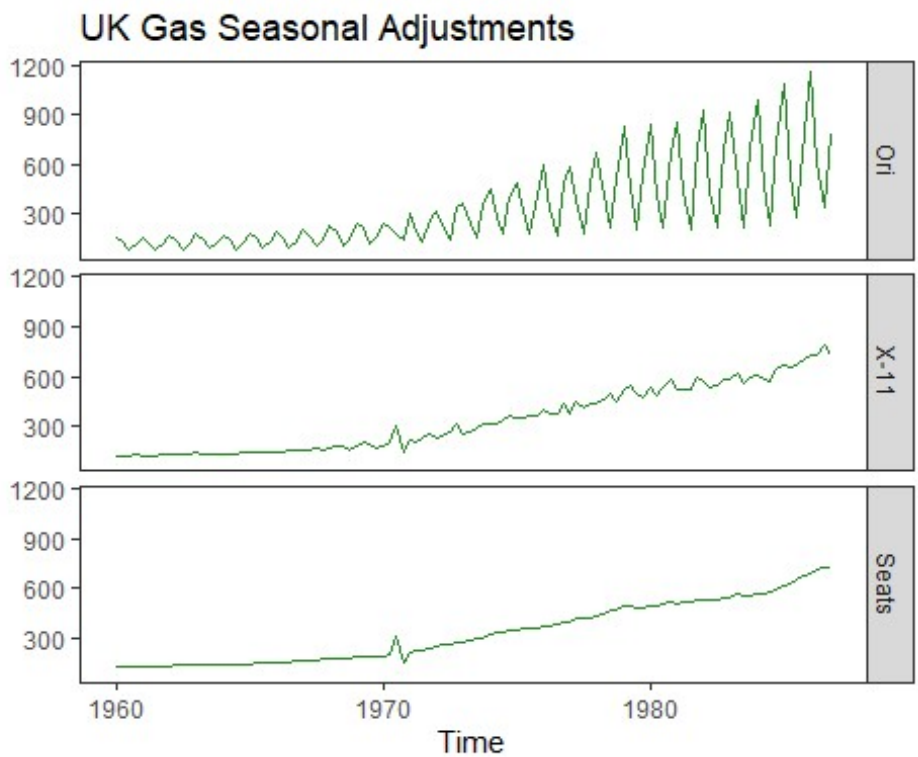

Figure 47: Seasonal Adjustment Facet Plot Example

5 SEATS Plots

There are three types of plots associated with the SEATS seasonal adjustment method within X-13ARIMA-SEATS - difference plots (Section 5.1 and Section 5.2), seasonal sum plots (Section 5.3), and plots related to the filters used by SEATS to filter the data (Section 5.4 to Section 5.7).

Note that these plots are NOT available for X-11 seasonal adjustments.

To create many of these SEATS graphs, the series must be run in X-13ARIMA-SEATS with the option `finite=yes` in the `seats` spec, or `seats.finite = "yes"` for input to the `seas` function of the `seasonal` package. For more information on input options for the `seats` spec of the X-13ARIMA-SEATS program, refer to Section 7.14 of U. S. Census Bureau (2025).

5.1 plot_fully_differenced_transformed

The `plot_fully_differenced_transformed` function produces a line plot of a time series that has been transformed and differenced based on the regARIMA model used in an X-13ARIMA-SEATS run. This series is generated by the `seats` spec and needs to be saved in the `seas` object generated by the `seasonal` package.

This function can produce this plot for three series - the original series, the SEATS seasonally adjusted series, or the SEATS trend cycle. Only one run of the seasonal package is needed to set up the `seas` object needed for each plot by setting `seats.save = c("dor", "dsa", "dtr")` in the input to the `seas` function.

The series plotted is controlled by the `this_series` argument. The user specifies an individual series to plot (`dor` for the original series, `dsa` for the SEATS seasonally adjusted series, or `dtr` for the SEATS trend).

Figure 48 is an example of this plot for the fully difference transformed original series.

```
# Generate seas object for the SEATS adjustment of the
# US Shoe Sales series
shoes_seats_seas <-
   seasonal::seas(blsplotGG::shoes2008,
                  arima.model = "(0 1 1)(0 1 1)",
                  transform.function = "log",
                  forecast.maxlead = 36,
                  check.print = c( 'pacf', 'pacfplot' ),
                  seats.finite = "yes",
                  seats.save = c( 'dor', 'dsa', 'dtr' )
)

# Generate plot of the fully differenced transformed original
# series for the US Shoe Sales series
p_ori_full_diff <-
   blsplotGG::plot_fully_differenced_transformed(
      shoes_seats_seas,
      this_series = "dor",
      sub_title = "US Shoe Sales",
      add_zero_line = TRUE)

# Display ggplot2 object
print(p_ori_full_diff)
```

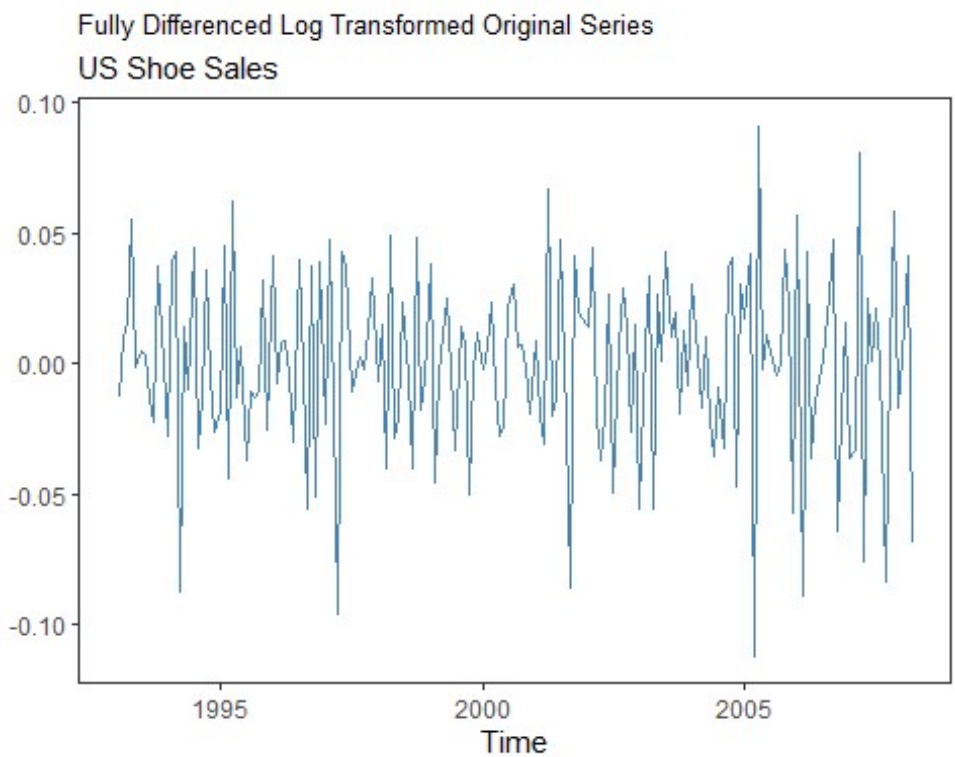

Figure 48: Fully Differenced Transformed Plot (All Series) Example

Note that this function uses the `plot_table` function to generate the plots. For more details on that function, see Section 2.6.

5.2 plot_fully_differenced_transformed_all

The `plot_fully_differenced_transformed_all` function is similar to the `plot_fully_differenced_transformed` function showed in Section 5.1, except that the original, seasonally adjusted, and trend fully differenced transformed series are plotted together in a panel plot.

Figure 49 is an example of this plot that shows the three fully differenced transformed series plots.

```
# Generate a panel plot of fully differenced transformed series
# for the original, seasonally adjusted, and trend series
p_all_full_diff <-
   blsplotGG::plot_fully_differenced_transformed_all(
      shoes_seats_seas,
      overall_title = "US Shoe Sales",
      title_size = 8, add_zero_line = TRUE)

# Display ggplot2 object
print(p_all_full_diff)
```

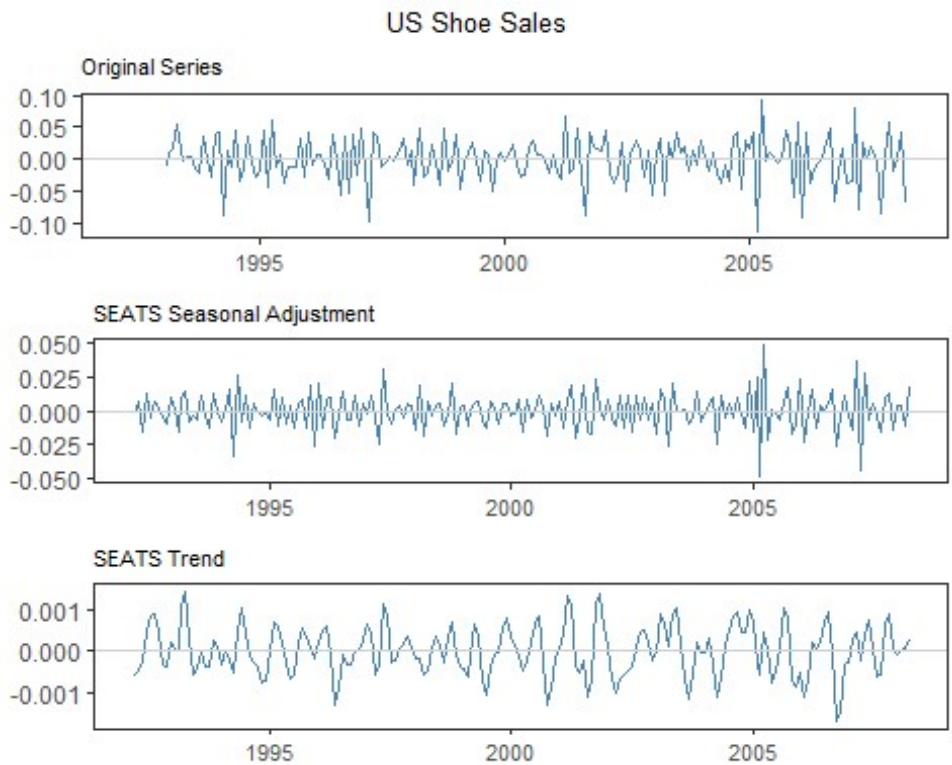

Figure 49: Fully Differenced Transformed Plot (All Series) Example

The `plot_fully_differenced_transformed` function is used to generate the plots in the individual panels. The individual plots are put into a facet format using the `ggarrange` function of the `ggpubr` package (Kassambara 2025).

5.3 plot_seasonal_sums

The function `plot_seasonal_sums` generates a plot of the seasonal period length moving sums of the SEATS seasonal factors from a SEATS adjustment extracted from a `seas` object generated by the `seasonal` package.

The filter coefficients are extracted from a `seas` object from a SEATS seasonal adjustment generated by the `seas` function of the `seasonal` package. To facilitate generating this plot, the input to `seas` should include an argument to save tables related to the adjustment and trend filters.

For example, a user could set `seats.save = 'ssm'`. In order to save these files, the user should also set `seats.finite = "yes"`.

Figure 50 shows such a plot for the US Shoe Sales series.

```r
# Generate seas object for the SEATS adjustment of the
# US Shoe Sales series
shoes_seats_seas <-
   seasonal::seas(blsplotGG::shoes2008,
                  arima.model = "(0 1 1)(0 1 1)",
                  transform.function = "log",
                  forecast.maxlead = 36,
                  check.print = c( 'pacf', 'pacfplot' ),
                  seats.finite = "yes",
                  seats.save = 'ssm' )

# Generate a plot of the seasonal period length moving
# sums of the SEATS seasonal factors
p_seasonal_sum <-
   blsplotGG::plot_seasonal_sums(shoes_seats_seas,
      sub_title = "US Shoe Sales",
      do_grid = TRUE,
      line_color = "darkgreen")

# Display ggplot2 object
print(p_seasonal_sum)
```

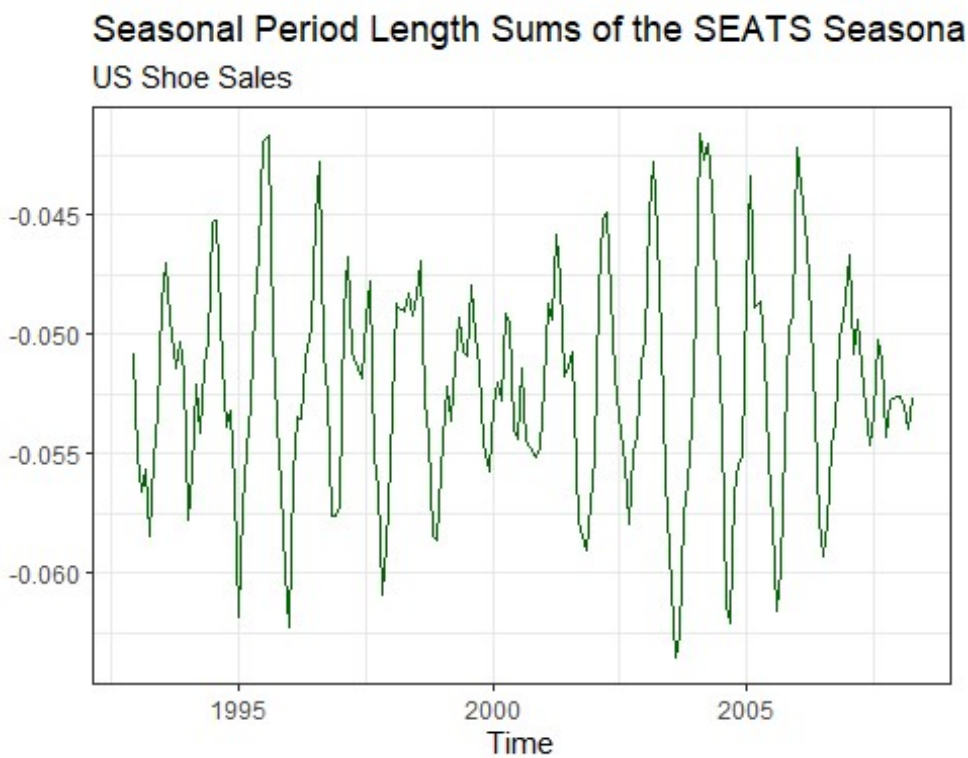

Figure 50: Seasonal Sums Plot Example

Note that this function uses the `plot_table` function from Section 2.6 to generate the plots.

5.4 plot_seats_filter

The `plot_seats_filter` function generates plots for the filters used in the SEATS seasonal adjustment process. There are four filter plots generated:

- the seasonal adjustment concurrent filter,
- the seasonal adjustment symmetric filter,
- the trend concurrent filter, and
- the trend symmetric filter.

The filter coefficients are extracted from a `seas` object from a SEATS seasonal adjustment generated by the `seas` function of the `seasonal` package. To facilitate generating this plot, the input to `seas` should include an argument to save tables related to the adjustment and trend filters.

For example, a user could set `seats.save = c( 'fac', 'faf', 'ftc', 'ftf' )`. To save these files, the user should also set `seats.finite = "yes"`.

Figure 51 shows the SEATS filter plot for the symmetric seasonal adjustment filter for the US Shoe Store series.

```
# Generate seas object for the SEATS adjustment of the
# US Shoe Sales series, saving the filter coefficients
shoes_seats_seas <-
    seasonal::seas(blsplotGG::shoes2008,
        arima.model = "(0 1 1)(0 1 1)",
        transform.function = "log",
        forecast.maxlead = 36,
        check.print = c( 'pacf', 'pacfplot' ),
        seats.finite = "yes",
        seats.save = c( 'fac', 'faf', 'ftc', 'ftf' ) )

# Generate plot of the SEATS symmetric seasonal adjustment
# filter
p_sa_sym_filter <-
    blsplotGG::plot_seats_filter(shoes_seats_seas,
        this_series = "sa",
        this_filter = "symmetric",
        sub_title = "US Shoe Sales",
        point_color = "darkgreen")

# Display ggplot2 object
print(p_sa_sym_filter)
```

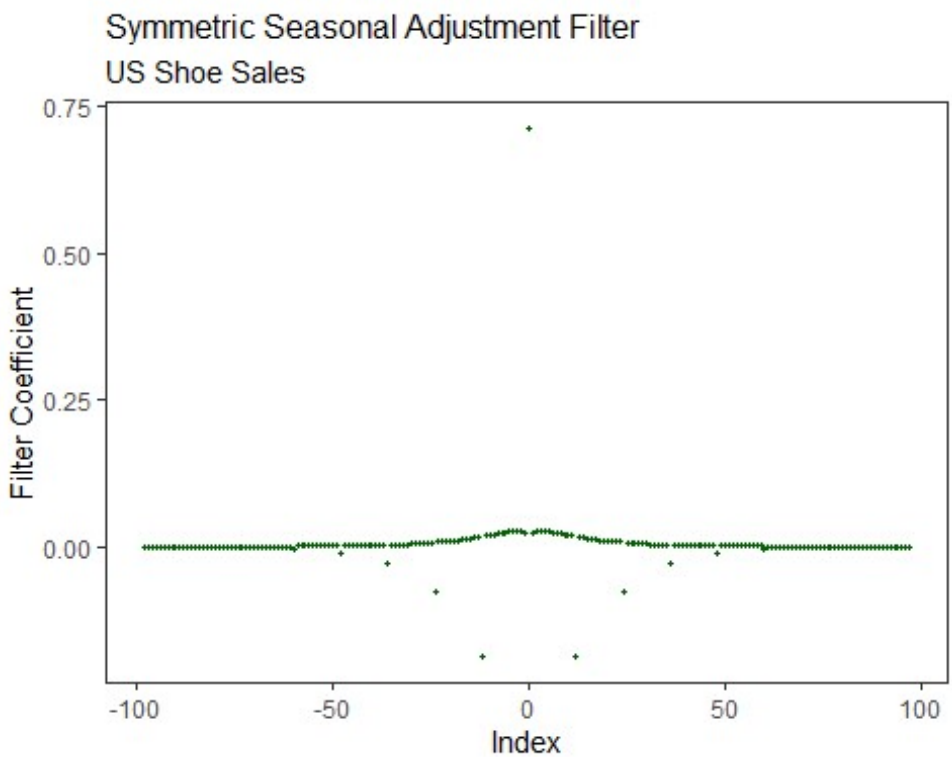

Figure 51: SEATS Filter Plot (SA, Symmetric) Example

Figure 52 shows the SEATS filter plot for the concurrent seasonal adjustment filter for the US Shoe Store series.

```
# Generate plot of the SEATS concurrent seasonal adjustment
# filter
p_sa_conc_filter <-
    blsplotGG::plot_seats_filter(shoes_seats_seas,
        this_series = "sa",
        this_filter = "concurrent",
        sub_title = "US Shoe Sales",
        point_color = "darkgreen")

# Display ggplot2 object
print(p_sa_conc_filter)
```

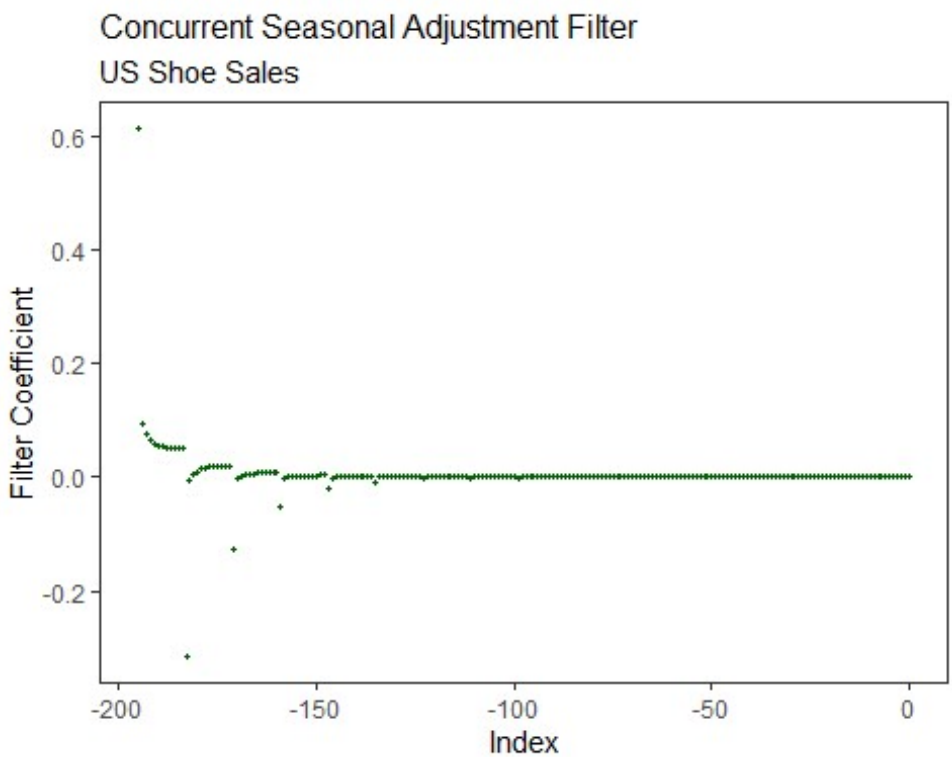

Figure 52: SEATS Filter Plot (SA, Concurrent) Example

5.5 plot_seats_filter_all

The `plot_seats_filter_all` function is similar to the `plot_seats_filter` function, except that all the plots that are listed in Section 5.4 are arranged in one plot.

Figure 53 show the four possible filter plots for the US Shoe Store series in one plot.

```
# Generate plot of the SEATS concurrent and symmetric seasonal
# adjustment and trend filters
p_all_filter <-
    blsplotGG::plot_seats_filter_all(shoes_seats_seas,
       overall_title = "US Shoe Sales",
       this_x_symmetric_breaks = seq(-96,96,24),
       point_color = "darkgreen")

# Display ggplot2 object
print(p_all_filter)
```

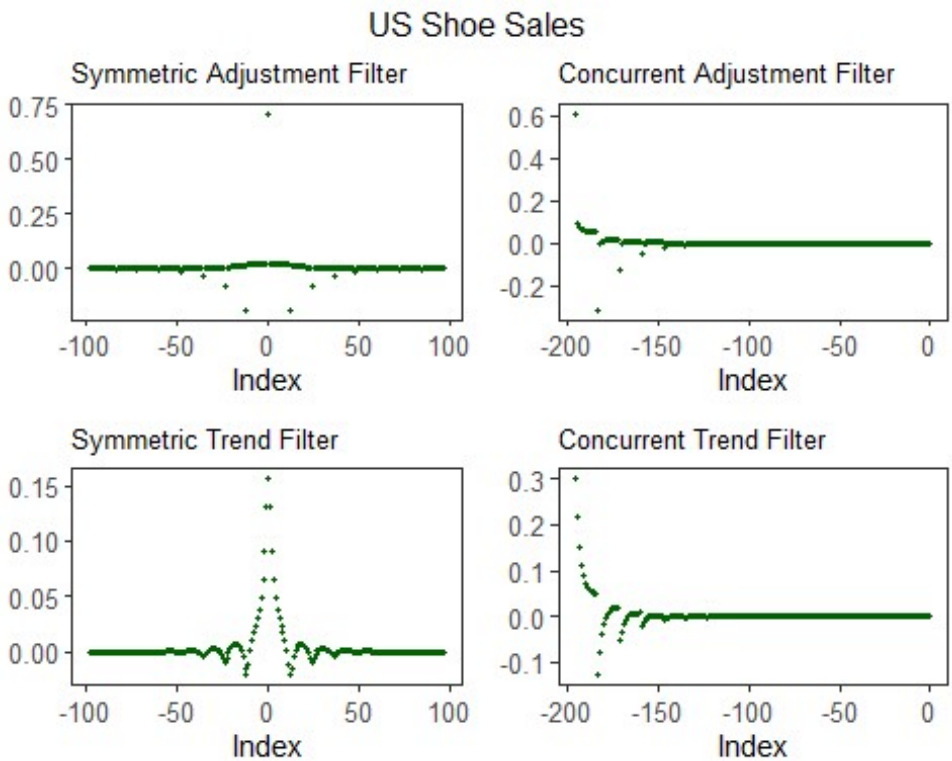

Figure 53: All SEATS Filter Plots Example

The `plot_seats_filter` function is used to generate the plots in the individual panels. The individual plots are put into a 2x2 grid format using the `ggarrange` function of the `ggpubr` package (Kassambara 2025).

5.6 plot_squared_gain

The `plot_squared_gain` function plots the squared gains of eiter the seasonal adjustment or trend filters for a given SEATS adjustment.

The squared gain for the concurrent and symmetric filters is plotted on the same graph.

The squared gains for the filters are extracted from a `seas` object from a SEATS seasonal adjustment generated by the `seas` function of the `seasonal` package. To facilitate generating this plot, the input to `seas` should include an argument to save tables related to the squared gains of the filters.

For example, a user can set `seats.save = c( 'gac', 'gaf', 'gtc', 'gtf' )`. To save these files, the user should also set `seats.finite = "yes"`.

Figure 54 shows the squared gain of the concurrent and symmetric seasonal adjustment filters for the US Shoe Sales series.

```r
# Generate seas object for the SEATS adjustment of the
# US Shoe Sales series, saving squared gains coefficients
# for seasonal adjustments and trends
shoes_seats_seas <-
    seasonal::seas(blsplotGG::shoes2008,
        arima.model = "(0 1 1)(0 1 1)",
        transform.function = "log",
        forecast.maxlead = 36,
        check.print = c( 'pacf', 'pacfplot' ),
        seats.finite = "yes",
        seats.save = c( 'gac', 'gaf', 'gtc', 'gtf' ) )

# Generate squared gain plot for the SEATS seasonal adjustment
p_sa_squared_gain <-
   blsplotGG::plot_squared_gain(shoes_seats_seas,
      main_title = “Squared Gains of SA Filter”,
      sub_title = "US Shoe Sales",
      this_palette = "Set2")

# Display ggplot2 object
print(p_sa_squared_gain)
```

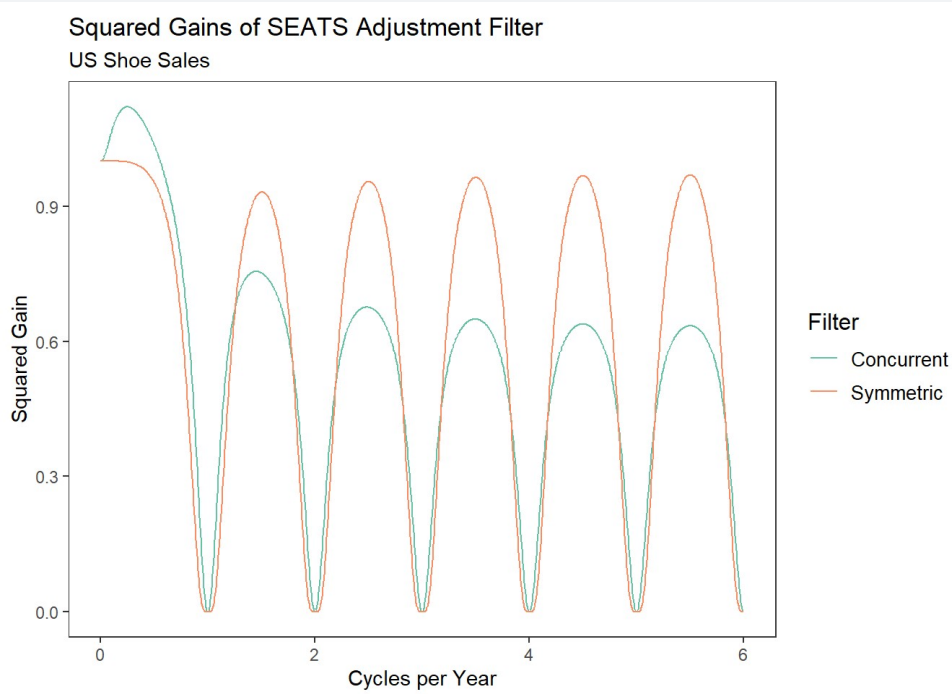

Figure 54: SEATS Squared Gain Plot Example

Figure 55 shows the squared gain of the concurrent and symmetric trend filters for the US Shoe Sales series.

```
# Generate squared gain plot for the SEATS trend
p_trend_squared_gain <-
  blsplotGG::plot_squared_gain(shoes_seats_seas,
                    sub_title = "US Shoe Sales",
                    this_series = "trend",
                    this_palette = "Set2")

# Display ggplot2 object
print(p_trend_squared_gain)
```

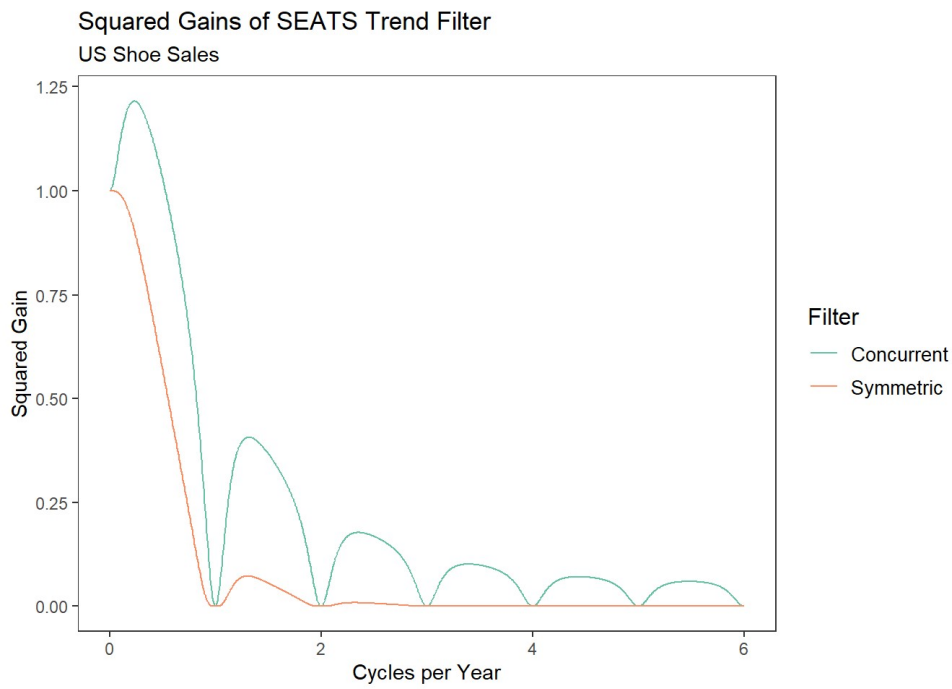

Figure 55: SEATS Squared Gain Plot Example

5.7 plot_time_shift

The `plot_time_shift` function plots the time shift for the finite concurrent seasonal adjustment and trend filters for a given SEATS adjustment.

The time shift for both the trend and seasonal adjustment filters is plotted on the same graph.

The time shift for the filters is extracted from a `seas` object from a SEATS seasonal adjustment generated by the `seas` function of the `seasonal` package. To facilitate generating this plot, the input to `seas` should include an argument to save tables related to the squared gains of the filters.

For example, a user could set `seats.save = c( 'tac', 'ttc' )`. In order to save these files, the user should also set `seats.finite = "yes"`.

Figure 56 shows the time shift for the finite concurrent trend and seasonal adjustment filters for the US Shoe Sales series. The colors for the plot are set by the `this_palette` argument to the '`Set2`' palette from the `RColorBrewer` package (Neuwirth 2022).

```
# Generate seas object for the SEATS adjustment of the
# US Shoe Sales series, saving time shift coefficients
# for the concurrent seasonal adjustments and trend filters
shoes_time_shift <-
   seasonal::seas(blsplotGG::shoes2008,
      arima.model = "(0 1 1)(0 1 1)",
      transform.function = "log",
      forecast.maxlead = 36,
      check.print = c( 'pacf', 'pacfplot' ),
      seats.finite = "yes",
      seats.save = c( 'tac', 'ttc' )
)

# Generate time shift plot for SA and trend filter
p_time_shift <-
   blsplotGG::plot_time_shift(shoes_time_shift,
      sub_title = "US Shoe Sales",
      this_palette = "Set2")

# Display ggplot2 object
print(p_time_shift)
```

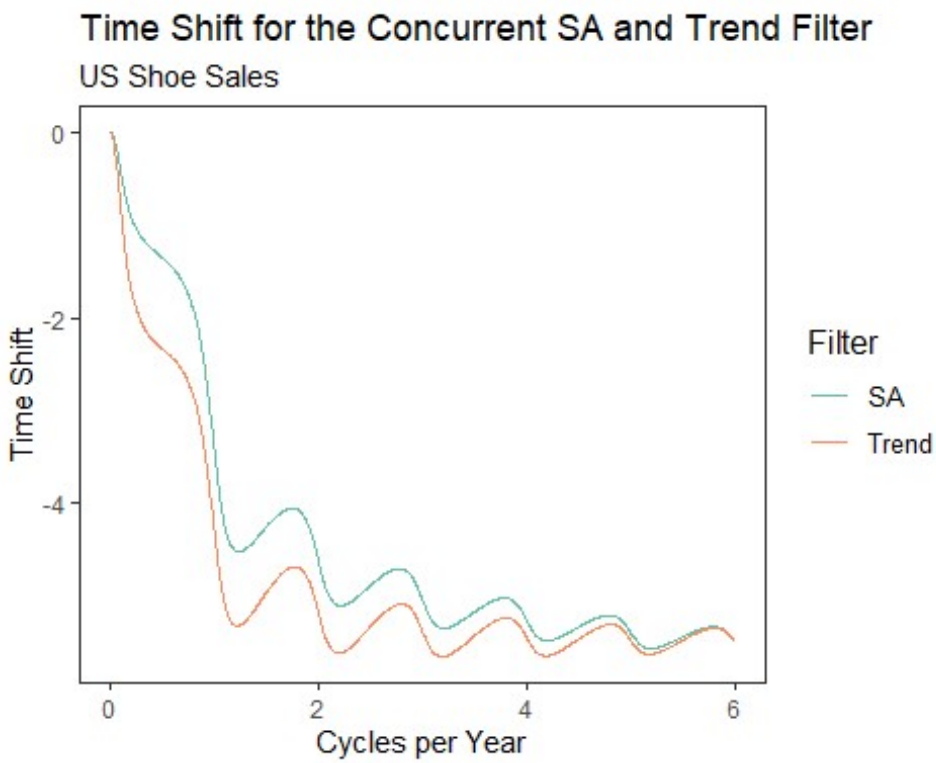

Figure 56: Time Shift Plot (SA and Trend) Example

6 Spectral Plots

Spectral plots are used as a diagnostic by the X-13ARIMA-SEATS program. This package produces two kinds of spectral plots - a plot that compares the spectrum of the original series with the seasonal adjusted series (Section 6.1) and a spectrum of a single time series (Section 6.2).

Section 6.1 of U. S. Census Bureau (2025) gives more information about the spectrum diagnostic as it appears in X-13ARIMA-SEATS. The research that established the procedure for flagging spectral peaks can be found in Soukup and Findley (1999).

6.1 plot_double_spectrum

The `plot_double_spectrum` function produces a plot of the the spectrum of the original series and the seasonal adjusted series. The seasonal and trading day frequencies are labelled with vertical dashed lines, and if a peak is considered visually significant as determined by Soukup and Findley (1999), It is labelled by a star at the top of the peak.

The spectral estimates are extracted from a `seas` object from a SEATS seasonal adjustment generated by the `seas` function of the `seasonal` package. To facilitate generating this plot, the input to `seas` should include an argument to save tables related to the spectrum.

For example, a user could set `spectrum.save = c("sp0", "sp1")` if they wanted to save the spectrum of the original series modified for extremes and the X-11 seasonally adjusted series modified for extremes. A list of these codes can be found in Table 1 in Section 6.2.

Figure 57 shows the double spectrum plot for the Airline Passenger series. All of the seasonal peaks for the original series are visually significant, and the first seasonal peak for the seasonally adjusted series is visually significant, as well as the second trading day frequency.

```
# Generate seas object for the X-11 adjustment of the
# Airline Passenger series, saving spectral estimates
# for the seasonal adjustment and original series
air_seas <-
    seasonal::seas(AirPassengers,
                   arima.model = "(0 1 1)(0 1 1)",
                   x11="",
                   spectrum.save = c("sp0", "sp1"))

# Generate spectrum of the original series and seasonally
# adjusted series
plot_double_spectrum_air <-
    blsplotGG::plot_double_spectrum(air_seas,
        series_name = "AirPassengers",
        this_color = c("steelblue", "forestgreen"),
        this_median_color = c("blue", "green"),
        this_freq_color = c("darkblue", "darkgreen"),
        this_peak_color = c("red", "orange"))

# Display ggplot2 object
print(plot_double_spectrum_air)
```

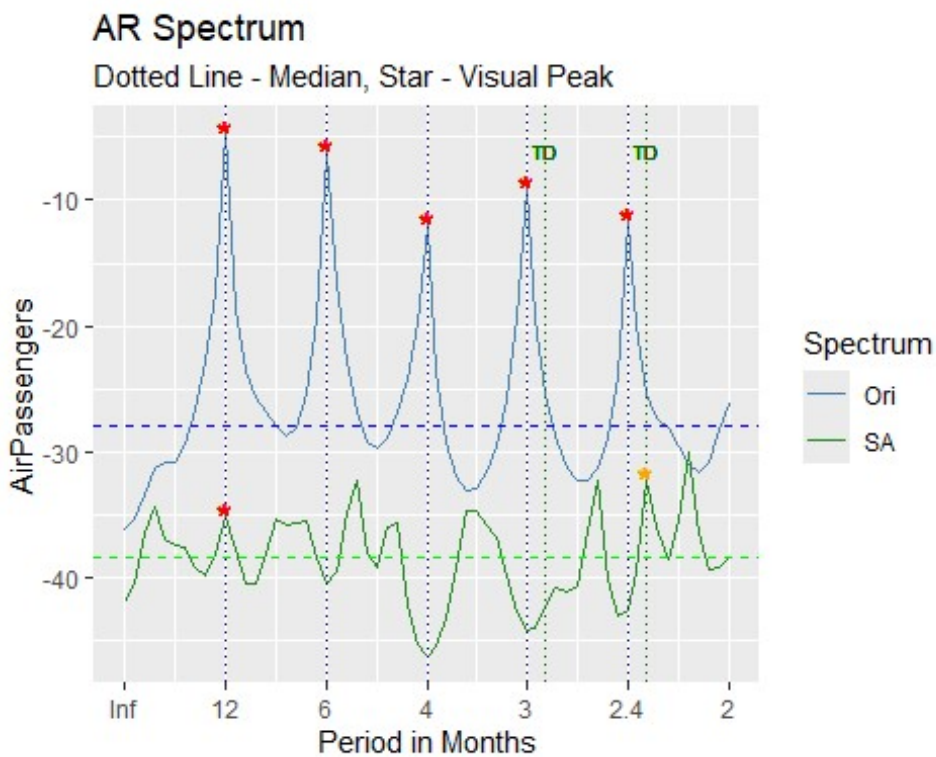

Figure 57: Double Spectrum Example

6.2 plot_spectrum

The `plot_spectrum` function generates one of the spectrums generated by the X-13ARIMA-SEATS program. This function extracts the spectral estimates from a seas object generated by the user and input the spectral estimates into the function.

In the input to the `seas` function of the `seasonal` package, users should specify a `spectrum.save` argument specifying the spectral plot they want to generate. The object saved will be a matrix, with columns for the frequency and spectral estimate being plotted.

Table 1 shows possible entries for that `spectrum.save` argument.

Table 1: Spectral Graph Table Names in X-13ARIMA-SEATS

Code	Description
sp0	Extreme modified original series
sp1	Extreme modified X-11 seasonally adjusted series)
sp2	Extreme modified X-11 irregular
s1s	Extreme modified SEATS seasonally adjusted series)
s2s	Extreme modified SEATS irregular
is0	Extreme modified composite series
is1	Extreme modified indirect seasonally adjusted series)
is2	Extreme modified indirect irregular
spr	Model residuals
ser	Extended residusal

Note that if you doing an X-11 seasonal adjustment, you should not specify a spectrum that requires a SEATS seasonal adjustment, and vice versa.

The next three plots show different spectrums for an X-11 seasonal adjustment of the Airline Passenger series. Figure 58 shows the spectrum of the extreme modified Airline Passenger series. As in Figure 57, all the first five seasonal peaks are flagged as being visually significant. None of the trading day peaks are flagged as visually significant.

```
# Generate seas object for the X-11 adjustment of the
# Airline Passenger series, saving spectral estimates
# for the original series, seasonal adjustment, and irregular.
air_seas <-
    seasonal::seas(AirPassengers, x11="",
                   arima.model = "(0 1 1)(0 1 1)",
                   spectrum.save = c("sp0", "sp1", "sp2"))

# Generate spectrum of the original series
plot_air_spectrum <-
    blsplotGG::plot_spectrum(air_seas, this_spectrum = "sp0",
        series_name = "AirPassengers",
        sub_title = "Airline Passengers",
        this_color = "steelblue",
        this_median_color = "blue",
        this_freq_color = c("darkblue", "darkgreen"),
        this_peak_color = c("red", "orange"))

# Display ggplot2 object
print(plot_air_spectrum)
```

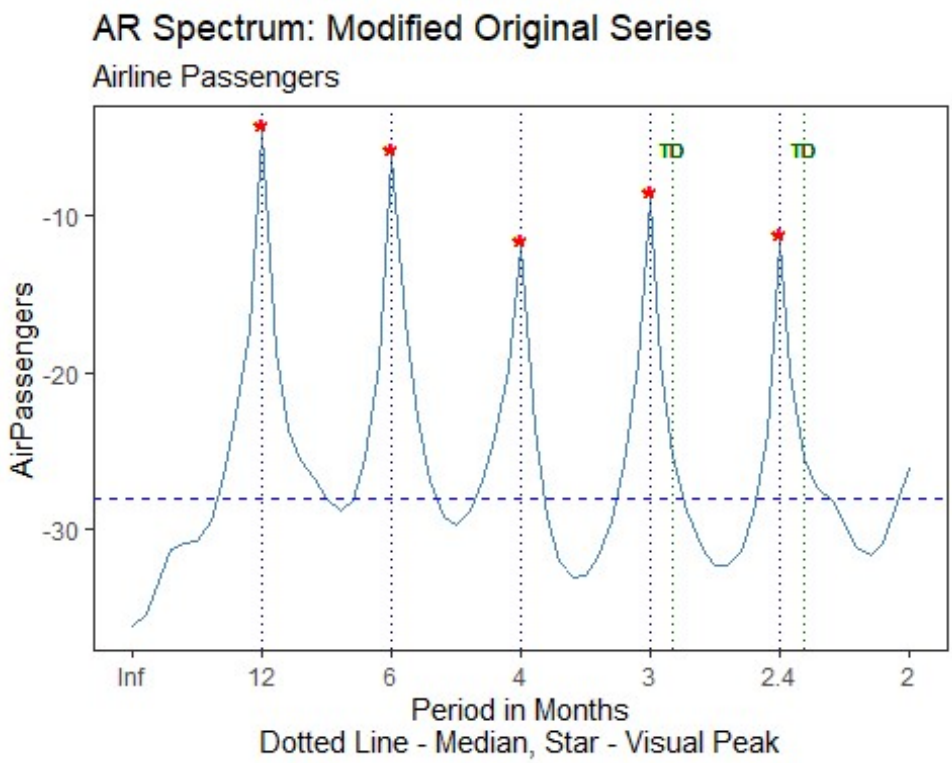

Figure 58: Spectrum Example (Modified original)

Figure 59 shows the spectrum of the modified seasonally adjusted Airline Passenger Series. As in Figure 57, the first seasonal peak is flagged as visually significant, as is the second trading day peak.

```
# Generate spectrum of the seasonally adjusted series
plot_air_spectrum <-
    blsplotGG::plot_spectrum(air_seas,
        this_spectrum = "sp1",
        series_name = "AirPassengers",
        sub_title = "Airline Passengers",
        this_color = "steelblue",
        this_median_color = "blue",
        this_freq_color = c("darkblue", "darkgreen"),
        this_peak_color = c("red", "orange"))

# Display ggplot2 object
print(plot_air_spectrum)
```

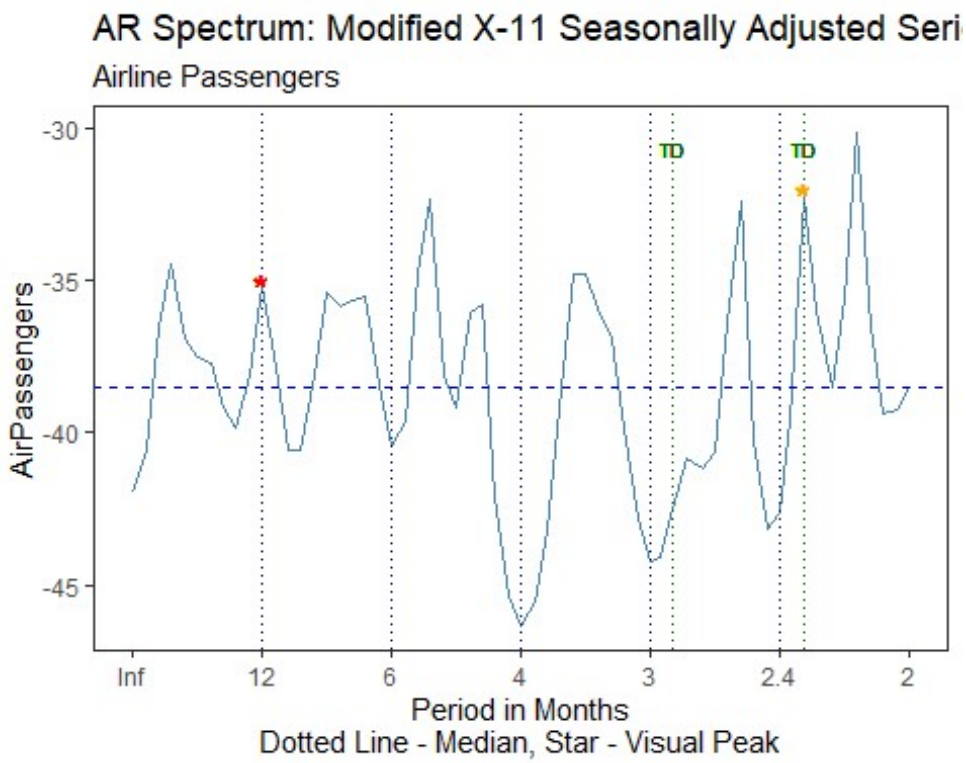

Figure 59: Spectrum Example (Modified X-11 SA)

Figure 60 shows the spectrum of the modified irregular from the X-11 adjustment of the Airline Passenger Series. None of the spectral peaks are visually significant, but as in Figure 59, the second trading day peak is visually signficant.

```
# Generate spectrum of the irregular series
plot_air_spectrum <-
    blsplotGG::plot_spectrum(air_seas,
       this_spectrum = "sp2",
       series_name = "AirPassengers",
       sub_title = "Airline Passengers",
       this_color = "steelblue",
       this_median_color = "blue",
       this_freq_color = c("darkblue", "darkgreen"),
       this_peak_color = c("red", "orange"))

# Display ggplot2 object
print(plot_air_spectrum)
```

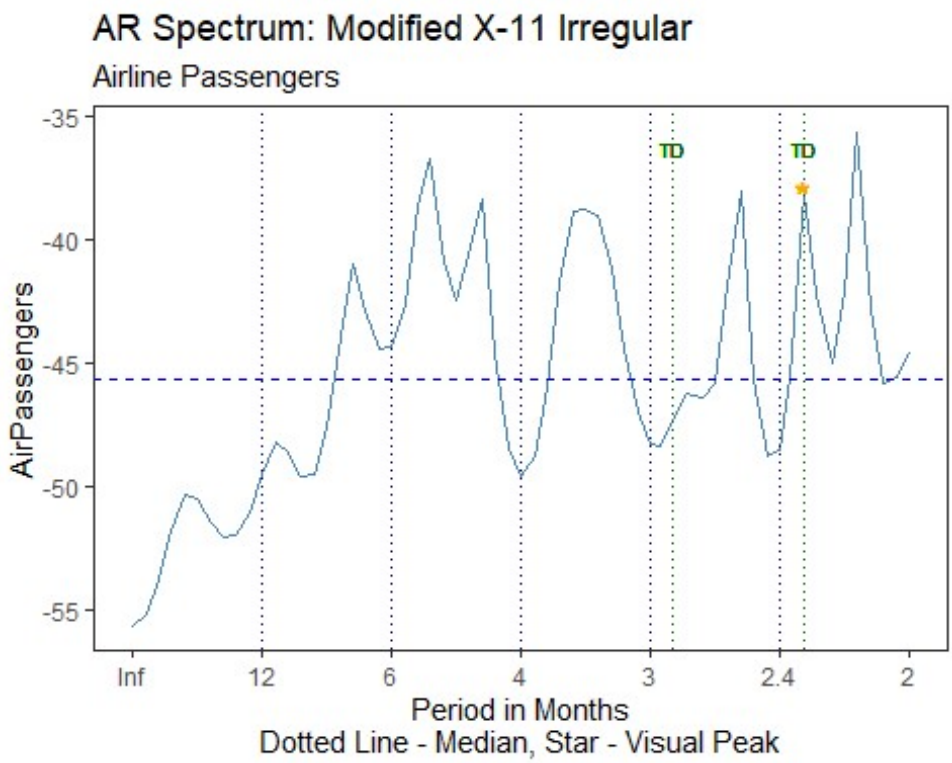

Figure 60: Spectrum Example (Modified X-11 Irregular)

7 History Plots

The `history` spec of X-13ARIMA-SEATS generates revisions between the initial estimate and the most recent estimate for several quantities derived from seasonally adjusting a time series. This spec can also produce an analysis of the revisions history for different lags past the concurrent observation. Usually, analysts look at the first and seasonal lags.

There are functions that examine the revisions of seasonal adjustment (Section 7.4), month-to-month or quarter-to-quarter changes in the seasonal adjustment (Section 7.2), or the trend component (Section 7.1).

In addition, this spec can also generate historical out-of-sample forecast errors (Section 7.3) and likelihood statistics derived from regARIMA model estimation. For some supporting theory for out-of-sample squared forecast error diagnostic output, see Findley (2005).

More details on history analysis in X-13ARIMA-SEATS can be found in Sections 6.3 and 7.8 of the X-13 Reference Manual (U. S. Census Bureau (2025)).

7.1 plot_all_trend_lags

The `plot_all_trend_lags` function generates a plot of multiple lags of revisions of a trend component. The main input is a seas object from a sesonal adjustment where a history analysis of the trend cycle is specified in the history spec (using the argument `history.estimates`) with a number of different revisions lags specified (using the argument `history.trendlags`).

In Figure 61, the final trend estimate for every observation is shown in a continuous line, while the lagged revisions for each observation are shown as shorter lines radiating from the final trend estimate.

```
# Generate seas object for the X-11 adjustment of the
# US Shoe Sales series, saving trend history estimates and
# setting trend lags to be plotted.
shoes_seas <-
    seasonal::seas(blsplotGG::shoes2008,
        x11 = "",
        x11.save = "d13",
        transform.function = "log",
        check.print = c("none", "+acf", "+acfplot",
                        "+normalitytest"),
        regression.aictest = c("td", "easter"),
        regression.save = c("td", "hol"),
        outlier.types = "all",
        arima.model = "(0 1 1)(0 1 1)",
        forecast.maxlead = 60,
        history.estimates = c("sadj", "sadjchng", "trend"),
        history.trendlags = 1:4,
        history.print = "all",
        history.save = c("tre", "trr"))

# Generate trend history plot
p_shoes_trend_history <-
    blsplotGG::plot_all_trend_lags(shoes_seas,
        this_trend_name = "shoes",
        main_title = "All Trend Revisions, Lags 1 to 4",
        sub_title = "US Retail Sales of Shoes")

# Display ggplot2 object
print(p_shoes_trend_history)
```

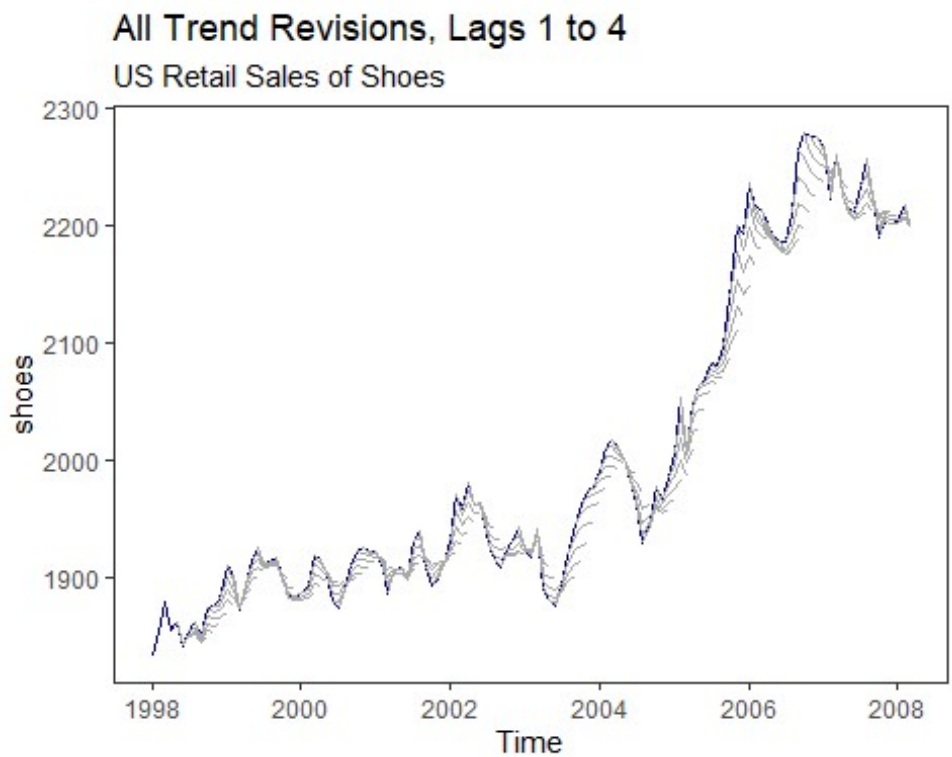

Figure 61: Trend Lag History Plot Example

7.2 plot_changes_history

The function `plot_changes_history` generates a plot of the percent change (from the previous observation) of the final seasonally adjusted estimate and the percent change of the initial seasonally adjusted estimate estimate. Each is plotted as a circle of different colors, with a vertical line connecting them.

The hoistory estimates are extracted from a `seas` object from a SEATS seasonal adjustment generated by the `seas` function of the `seasonal` package. To facilitate generating this plot, the input to `seas` should include an argument to save tables related to the revisions history being plotted.

For example, a user could set `history.save = c("che", "tce")` if they wanted to save the revisions history of the seasonal adjustment and trend changes.

Figure 62 shows a history plot of the seasonally adjusted series changes for the US Shoe Sales series. The initial changes are plotted in light blue, and the final changes are plotted in a darker blue.

```r
# Generate seas object for the X-11 adjustment of the
# US Shoe Sales series, saving history estimates for the
# month-to-month changes of the seasonally adjusted and
# trend series.
shoes_seas <-
    seasonal::seas(blsplotGG::shoes2008,
        x11 = "",
        x11.save = "d13",
        transform.function = "log",
        check.print = c("none", "+acf", "+acfplot",
                        "+normalitytest"),
        regression.aictest = c("td", "easter"),
        regression.save = c("td", "hol"),
        outlier.types = "all",
        arima.model = "(0 1 1)(0 1 1)",
        forecast.maxlead = 60,
        history.estimates =
          c("sadj", "sadjchng", "trend", "trendchng"),
        history.sadjlags = c(1,12),
        history.print = "all",
        history.save = c("che", "tce"))

# Generate seasonally adjustment changes history plot
p_shoes_sadjchng_history <-
    blsplotGG::plot_changes_history(shoes_seas,
        plot_start = c(2003, 1),
        main_title = "Seasonal Adjustment Change History Graph",
        sub_title = "US Retail Sales of Shoes")

# Display ggplot2 object
print(p_shoes_sadjchng_history)
```

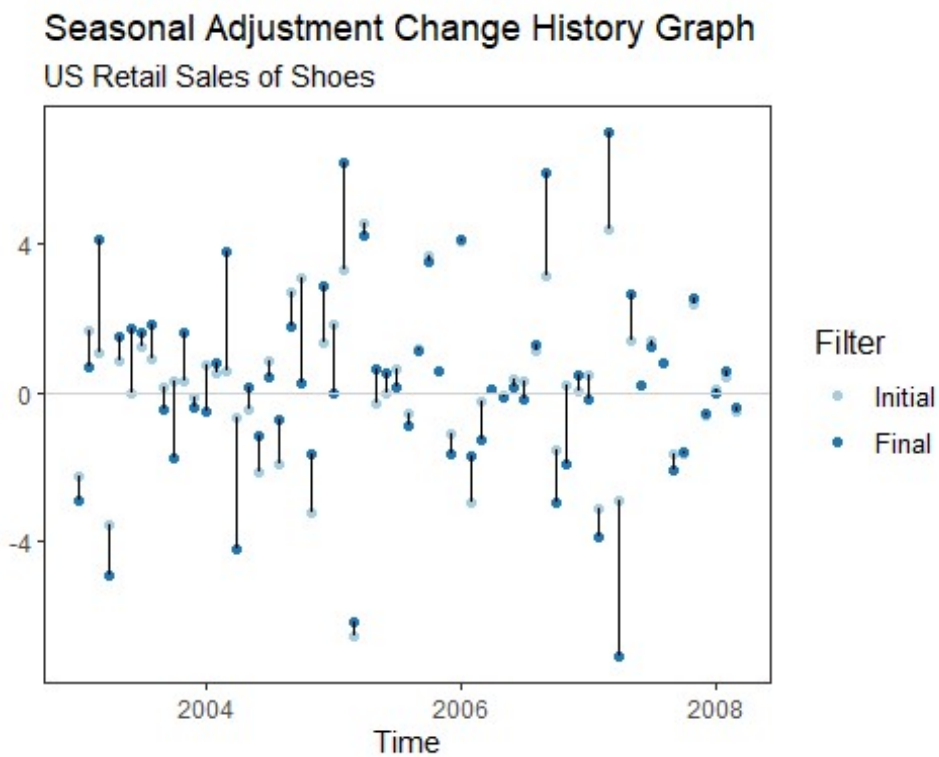

Figure 62: Changes Revision History Plot Example

Figure 63 shows a history plot of the trend change history for the US Shoe Sales series. In this plot, the legend appears at the bottom of the plot, due to the setting of the `this_legend_position` argument.

```
# Generate trend changes history plot
p_shoes_trendchng_history <-
    blsplotGG::plot_changes_history(shoes_seas,
        this_change = "trend",
        plot_start = c(2003, 1),
        main_title = "Trend Change History Graph",
        sub_title = "US Retail Sales of Shoes",
        this_legend_position = "bottom",
        legend_title_size = 10,
        legend_text_size = 8,
        this_legend_space = 0)

# Display ggplot2 object
print(p_shoes_trendchng_history)
```

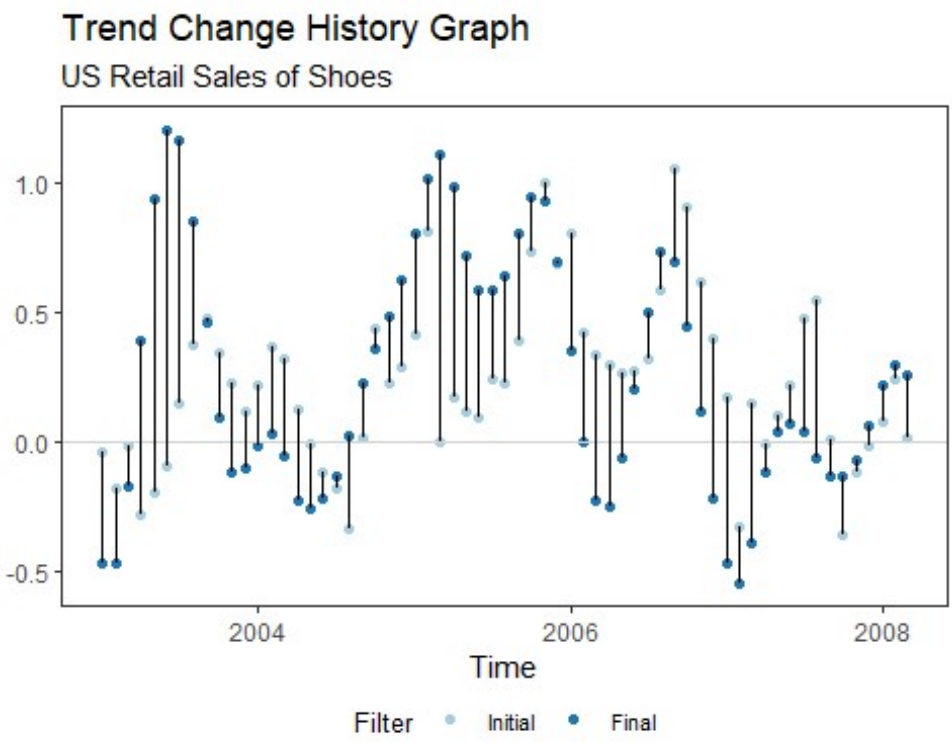

Figure 63: Trend Changes Revision History Plot Example

7.3 plot_fcst_history

The `plot_fcst_history` function produces a plot of the differences in the sum of squared forecast errors between forecasts from two models. This plot can be useful diagnostic to determine which model produces better forecasts for the time series.

Findley (2005) established the theory that justified generating model selection plots to compare squared forecast error for different forecast leads.

By examining how the difference between the squared forecast error changed over time, these graphs can be used as a model selection tool in circumstances where likelihood statistics cannot be used.

Before calling this function, users need to generate two `seas` objects for the same series with different regARIMA models. In the input for both runs of the `seasonal` packages, users also need to have several options from the history spec; users must set `history.estimates = 'fcst'` and `history.save = 'fcsterrors'`. Users may also wish to explicitly set the forecast lags used in the forecast history analysis - `history.fstep = c(1, 12)` for monthly series and `history.fstep = c(1, 4)` for quarterly series

Figure 64 is a forecast history plot for the Airline Passengers series. Here we're comparing a regARIMA model with no regressors to a model with trading day, Easter, and outlier regressors.

```
# Generate seas object for the Airline Passengers series,
# saving history estimates for the one step and 12 step
# ahead forecasts. The regARIMA model has no regressors.
air_seas_mdl <-
    seasonal::seas(AirPassengers, x11="",
        slidingspans = "",
        transform.function = "log",
        arima.model = "(0 1 1)(0 1 1)",
        regression.aictest = NULL, outlier = NULL,
        forecast.maxlead = 36,
        check.print = c( 'pacf', 'pacfplot' ),
        history.fstep = c(1, 12),
        history.estimates = 'fcst',
        history.save = 'fcsterrors')

# Generate seas object for the Airline Passengers series,
# saving history estimates for the one step and 12 step
# ahead forecasts. The regARIMA model has trading day and
# other regressors.
air_seas_mdl2 <-
    seasonal::seas(AirPassengers, x11="",
        slidingspans = "",
        transform.function = "log",
        arima.model = "(0 1 1)(0 1 1)",
        regression.variables = c("td"),
        forecast.maxlead = 36,
        check.print = c( 'pacf', 'pacfplot' ),
        history.fstep = c(1, 12),
        history.estimates = 'fcst',
        history.save = 'fcsterrors')

# Generate forecast history plot
plot_fcst_history_air <-
    blsplotGG::plot_fcst_history(air_seas_mdl, air_seas_mdl2,
        main_title =
            'Differences in the Sum of Squared Forecast',
        name_mdl1 = 'Airline model',
        name_mdl2 = 'Airline model + regressors')
```

```
# Display ggplot2 object, using the suppressWarnings function
# to suppress warning messages.
suppressWarnings(print(plot_fcst_history_air))
```

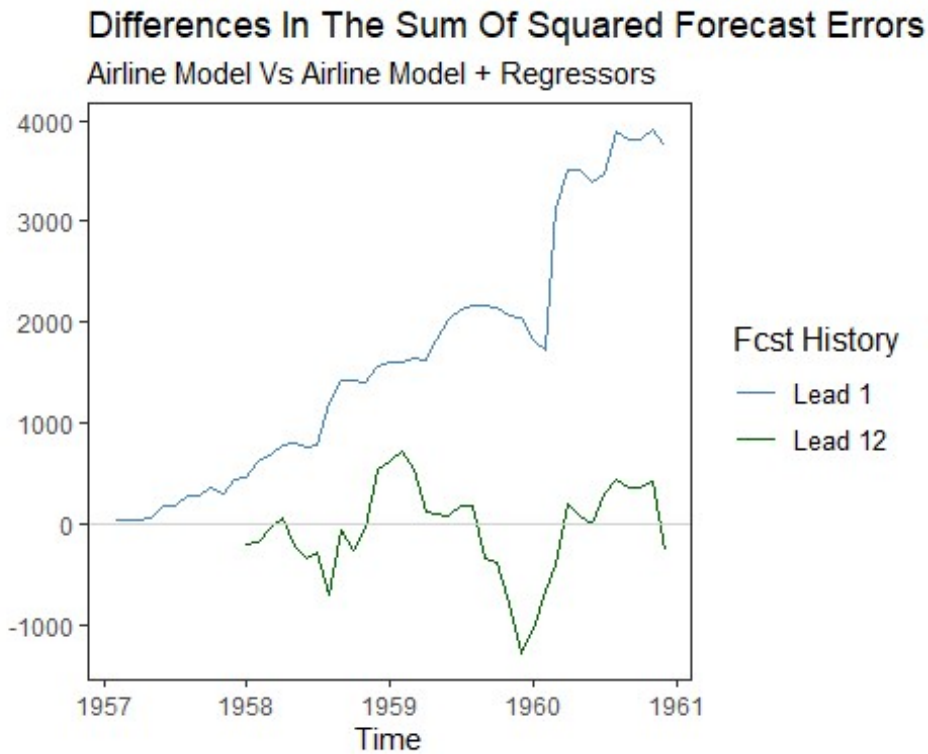

Figure 64: Forecast History Plot Example

There are two forecast lags being displayed here - one-step ahead forecasts and 12-step ahead forcasts. The line for the one-step ahead forecasts is rising consistently above the zero line. This implies that the forecast error for the first model is consistently higher than that of the second model, showing a consistent preference for the second model with the additional regressors. The results for the seasonal forecast lag (12 step ahead) are different, with not consistent preferences between the two models.

7.4 plot_sa_history

The `plot_sa_history` function shows a number of estimates from the history analysis of seasonal adjustments. The initial and final seasonal adjustments are plotted as solid lines of different colors. Individual points for the lag 1 (initial) and the seasonal lag (12 for monthly series, 4 for quarterly) revisions are also included in the plot.

The first argument for the `plot_sa_history` is a `seas` object. The input for this object needs to include some arguments from the `history` spec. The first required argument is setting `history.estimates = "sadj"` - there can be other entries for this argument, but `"sadj"` needs to be one of them.

Another required argument is `history.save = "sae"`. Again, there can be other entries for this argument, but `"sae"` needs to be one of them.

Finally, you should have `history.sadjlags = c(1, 12)` for monthly series or `history.sadjlags = c(1, 4)` for quarterly series.

Figure 65 shows a seasonal adjustment history plot for the US Shoe Sales series.

```
# Generate seas object for the X-11 adjustment of the
# US Shoe Sales series, saving history estimates for the
# seasonally adjusted series.
shoes_seas <-
    seasonal::seas(blsplotGG::shoes2008,
        x11 = "",
        x11.save = "d13",
        transform.function = "log",
        check.print = c("none", "+acf", "+acfplot",
                        "+normalitytest"),
        regression.aictest = c("td", "easter"),
        regression.save = c("td", "hol"),
        outlier.types = "all",
        arima.model = "(0 1 1)(0 1 1)",
        forecast.maxlead = 60,
        history.estimates = c("sadj", "sadjchng", "trend"),
        history.sadjlags = c(1, 12),
        history.print = "all",
        history.save = c("sae", "sar"))

# Generate seasonal adjustment history graph for US Shoe Sales
p_shoes_sa_history <-
    blsplotGG::plot_sa_history(shoes_seas,
        add_ori = FALSE,
        main_title = "SA History Graph, Lag 1 and 12",
        sub_title = "US Retail Sales of Shoes",
        this_palette = "Paired")

# Display ggplot2 object, using the suppressWarnings function
# to suppress warning messages.
suppressWarnings(print(p_shoes_sa_history))
```

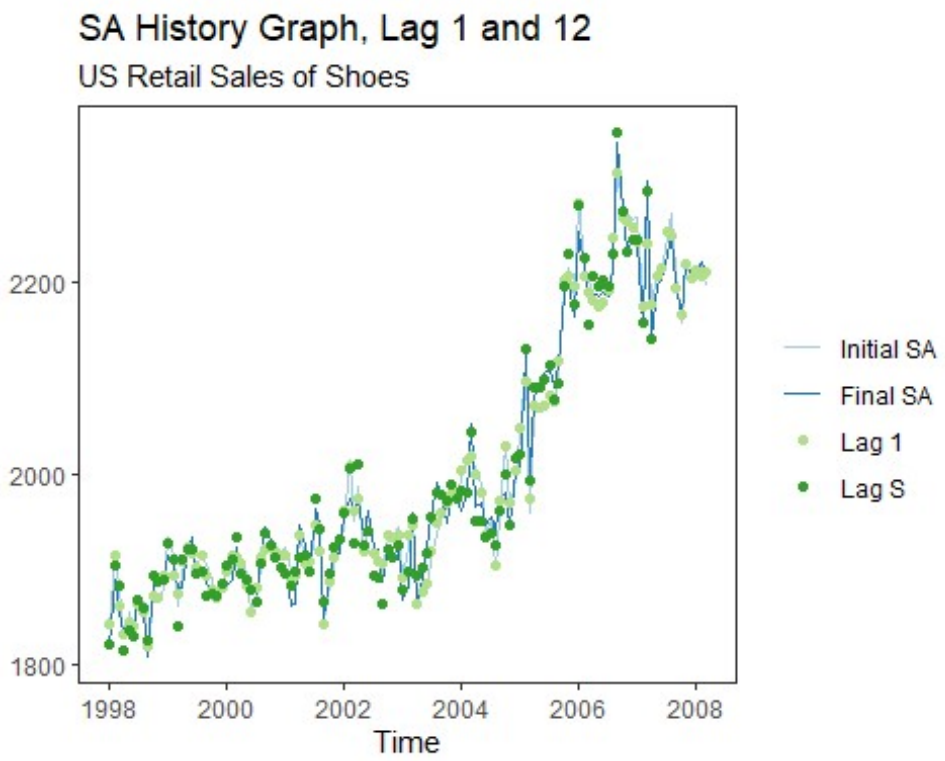

Figure 65: Seasonal Adjustment History Plot Example

8 Sliding Spans Plots

The `slidingspans` spec of X-13ARIMA-SEATS generates the sliding spans diagnostics. This method would form up to four sliding spans of data from the original series, with each span starting a year after the last span. The length of the span would be based on the seasonal filter used in the seasonal adjustment.

The maximum absolute difference for every observation that has more than one estimate was generated, and those with an absolute difference more than 3 percent were deemed to be too unstable to be reliable.

This method is available for seasonal factors, seasonally adjusted series, month-to-month and year-to-year changes, and trading day factors. This method worked best for multiplicative seasonal adjustments.

There are two types of plots that are generated by `blsplotGG`. There are three plots that show the maximum percentage difference generated by the sliding spans (both a ratio plot, a plot by period, and an overlay from two sliding spans analysis). The other plot shows the estimates from the sliding spans themselves.

8.1 plot_maximum_percent_difference

The `plot_maximum_percent_difference` function generates a ratio plot of the maximum percent difference generated over the sliding spans for a given type of estimate. The plot resembles a ratio plot as shown in Section 2.3. There is a horizontal line set to the cut off for the maximum difference that determines which observations are flagged as unstable.

Figure 66 shows a plot of the maximum percentage differences of the SEATS seasonal factors for the sliding spans of US Shoe Sales series.

```
# Generate seas object for the SEATS adjustment of the
# US Shoe Sales series, saving estimates from the sliding spans
# for the seasonal factors.
shoes_sspan_seas <-
   seasonal::seas(blsplotGG::shoes2008,
                  arima.model = "(0 1 1)(0 1 1)",
                  transform.function = "log",
                  forecast.maxlead = 36,
                  slidingspans.save = c('sfs', 'chs')
)

# Generate a maximum percent difference plot of seasonal factors
p_shoes_maxpct_sf <-
   blsplotGG::plot_maximum_percent_difference(
      seas_obj = shoes_sspan_seas,
      this_series = "sfs",
      main_title =
         "Maximum Percent Difference Plot of Seasonal Factors",
      sub_title = "US Shoe Sales",
      line_color = "blue",
      cut_color = "purple")

# Display ggplot2 object
print(p_shoes_maxpct_sf)
```

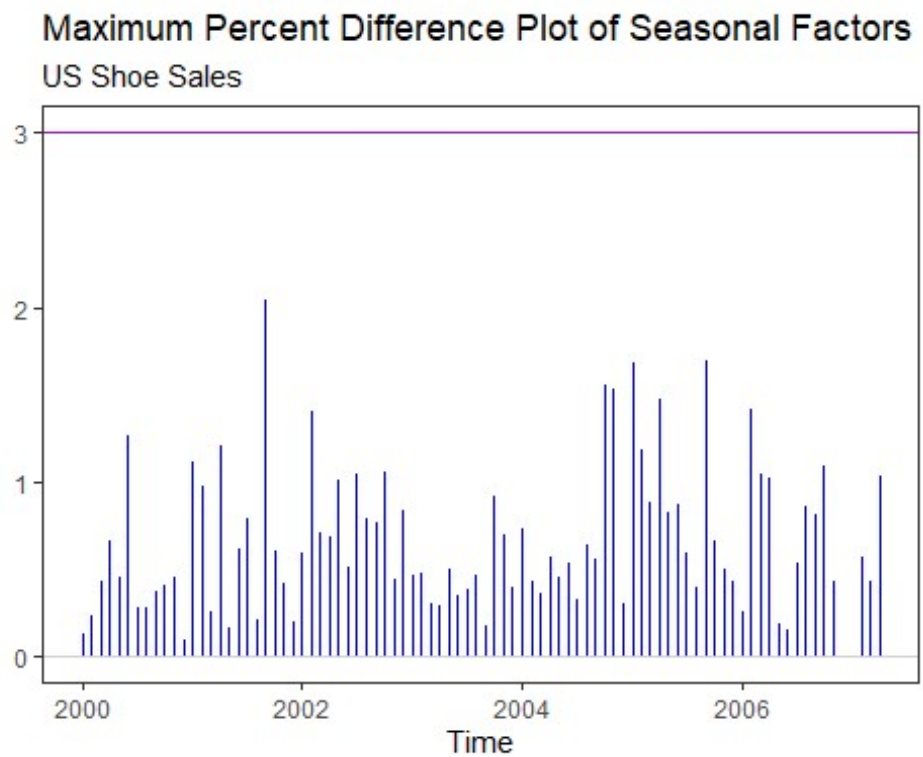

Figure 66: Max Percentage Difference Plot Example

8.2 plot_maximum_percent_difference_by_period

The `plot_maximum_percent_difference_by_period` function generates a plot of the maximum percent difference generated over the sliding spans for a given type of estimate grouped by period (months or quarters). The plot is not a ratio plot as shown in Section 8.1. Each period has points for each year of data, with a different color for each year. There is a horizontal line set to the cut off for the maximum difference that determines which observations are flagged as unstable.

Figure 67 shows a plot of the maximum percentage differences for the sliding spans of the SEATS seasonal factors grouped by month of the US Shoe Sales series. Note that the `seas` object generated for Figure 66 is used in this section.

```r
# Generate a maximum percent difference plot of seasonal factors
# grouped by month
p_shoes_maxpct_sf <-
   blsplotGG::plot_maximum_percent_difference_by_period(
      seas_obj = shoes_sspan_seas,
      this_series = "sfs",
      main_title =
         "Maximum Percent Difference Plot of Seasonal Factors",
      sub_title = "US Shoe Sales",
      this_palette = "Paired"
)

# Display ggplot2 object
print(p_shoes_maxpct_sf)
```

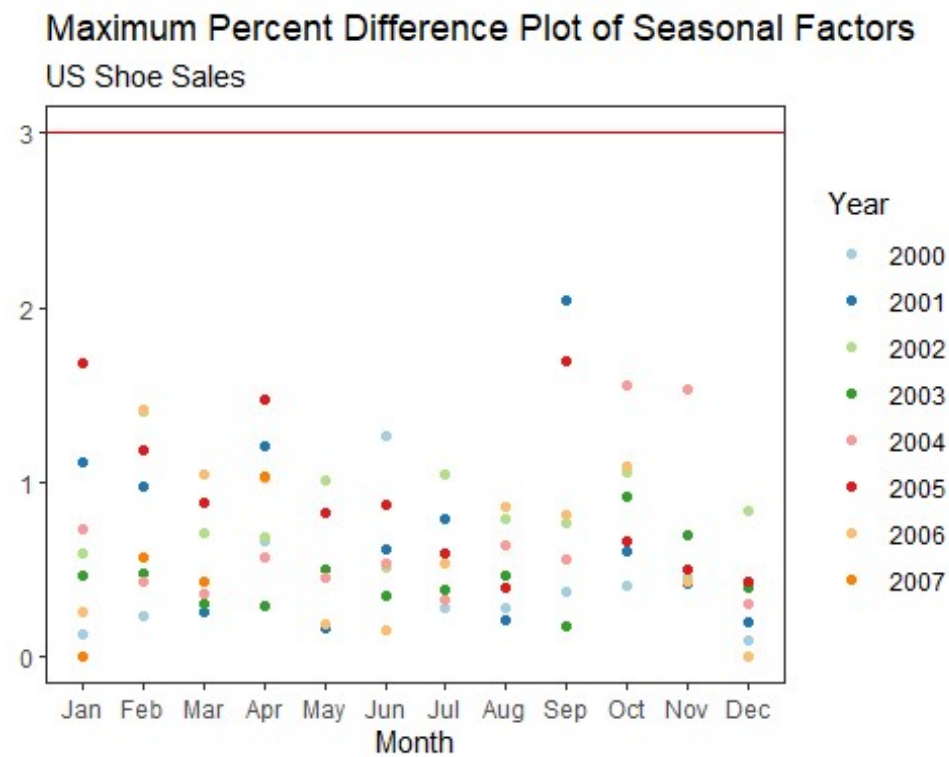

Figure 67: Max Percentage Difference By Month Plot Example

Figure 68 shows the same plot as Figure 67 except the legend has been moved to the bottom of the plot using the `this_legend_position` agrument.

```r
# Generate vector to use in legend
legend_order <-
    as.character(as.vector(matrix(2000:2007,
                                  ncol = 4, byrow = TRUE)))

# Generate a maximum percent difference plot of seasonal factors
# grouped by month, with the legend on the bottom of the plot
p_shoes_maxpct_sf <-
   blsplotGG::plot_maximum_percent_difference_by_period(
      seas_obj = shoes_sspan_seas,
      this_series = "sfs",
      main_title =
         "Maximum Percent Difference Plot of Seasonal Factors",
      sub_title = "US Shoe Sales",
      this_palette = "Paired",
      this_legend_position = "bottom",
      this_legend_order = legend_order,
      this_legend_size = 0.25,
      this_legend_space = 0)

# Display ggplot2 object
print(p_shoes_maxpct_sf)
```

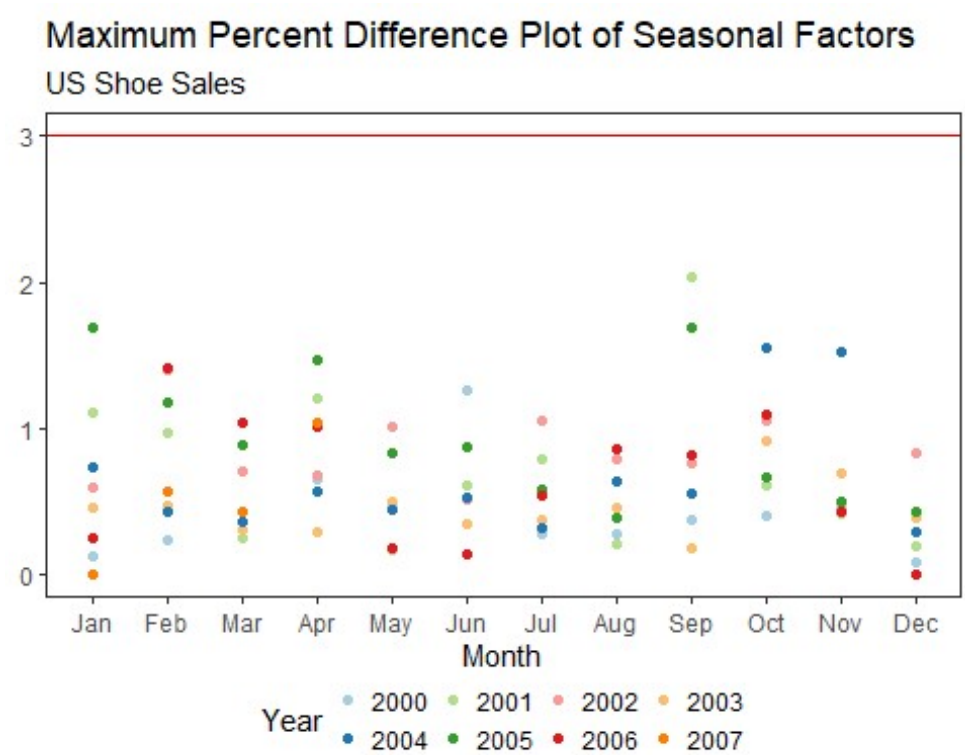

Figure 68: Max Percentage Difference By Month Plot (Bottom Legend) Example

8.3 plot_sliding_spans

The `plot_sliding_spans` function generates a plot of the estimates generated by the sliding spans analysis plotted on the same axis. Each span is plotted in a different color.

Figure 69 shows a plot of the sliding spans of the SEATS seasonal factors estimated for the US Shoe Sales series. Note that the `seas` object generated for Figure 66 is used in this section.

```
# Generate a plot of the sliding spans of seasonal factors
# grouped by month
p_shoes_spans_sf <-
    blsplotGG::plot_sliding_spans(
        seas_obj = shoes_sspan_seas,
        this_series = "sfs",
        main_title = "Spans of Seasonal Factors",
        sub_title = "US Shoe Sales",
        this_palette = "Set1")

# Display ggplot2 object
print(p_shoes_spans_sf)
```

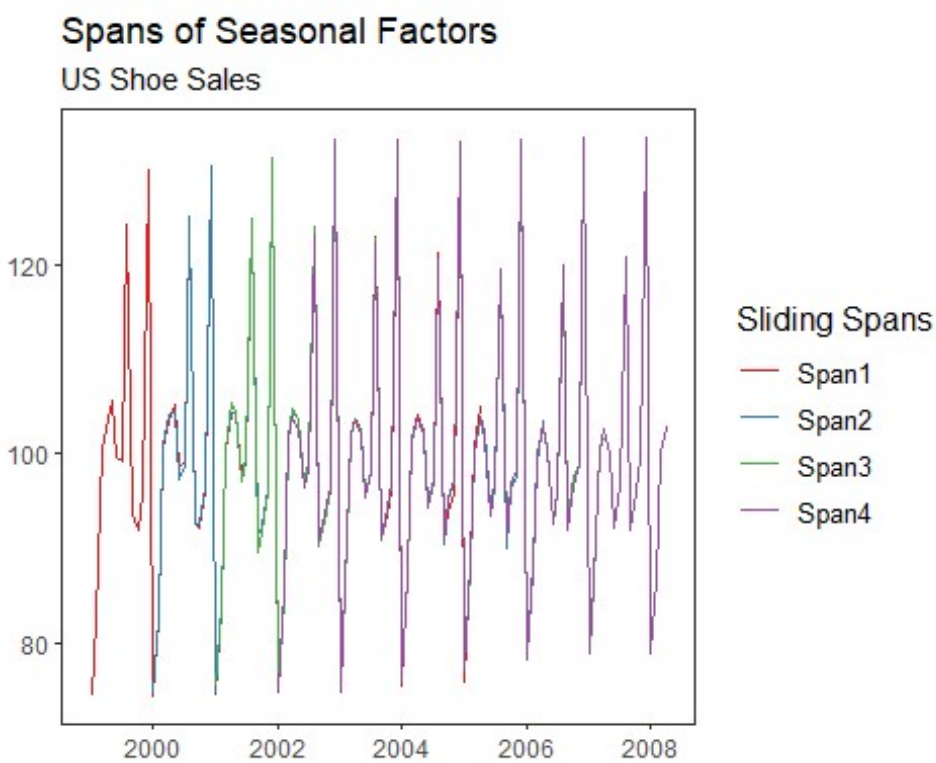

Figure 69: Sliding Spans Plot Example

8.4 plot_sliding_spans_two

The `plot_sliding_spans_two` is another plot that compares the maximum percent differences as in Section 8.1 and Section 8.2. In this case, the sliding spans results are plotted for seasonal factors or changes from two sliding spans runs.

The sliding spans analysis for both series should have the same length and number of spans (which can be set in the input for `seas` with the `slidingspans.length` and `slidingspans.numspans` arguments).

Figure 70 shows a comparison between the maximum percentage differences in the seasonal factors in the individual sliding spans for SEATS and X-11 adjustments for US Shoe Sales series. The maximum percent difference for the SEATS seasonal factors are a different colored point than for the X-11 adjustment, and a verticle line connects the SEATS seasonal factors with the X-11 seasonal factors.

```
# Generate seas object for the SEATS adjustment of the
# US Shoe Sales series, saving estimates from the sliding spans
# for the seasonal factors.
shoes_sspan_seas <-
   seasonal::seas(blsplotGG::shoes2008,
      arima.model = "(0 1 1)(0 1 1)",
      transform.function = "log",
      forecast.maxlead = 36,
      slidingspans.save = c('sfs', 'chs'))

# Generate seas object for the X-11 adjustment of the
# US Shoe Sales series, saving estimates from the sliding spans
# for the seasonal factors and setting the length and number
# of the sliding spans to match those for the SEATS adjustment.
shoes_sspan_x11_seas <-
   seasonal::seas(blsplotGG::shoes2008,
      arima.model = "(0 1 1)(0 1 1)",
      transform.function = "log",
      forecast.maxlead = 36,
      x11.seasonalma = "s3x3",
      slidingspans.length = 76,
      slidingspans.numspans = 4,
      slidingspans.print = 'sfs',
      slidingspans.save = c('sfs', 'chs'))
```

```r
# Generate plot comparing the sliding spans between the SEATS
# and X-11 seasonal factors.
p_shoes_maxpct_sf <-
   blsplotGG::plot_sliding_spans_two(
      seas_obj_one = shoes_sspan_seas,
      seas_obj_two = shoes_sspan_x11_seas,
      this_series = "sfs",
      main_title = "Maximum Difference of the Seasonal Factors",
      sub_title = "US Retail Shoe Sales",
      cut_color = "purple",
      this_legend_entry = c("Seats", "X-11"))

# Display ggplot2 object
print(p_shoes_maxpct_sf)
```

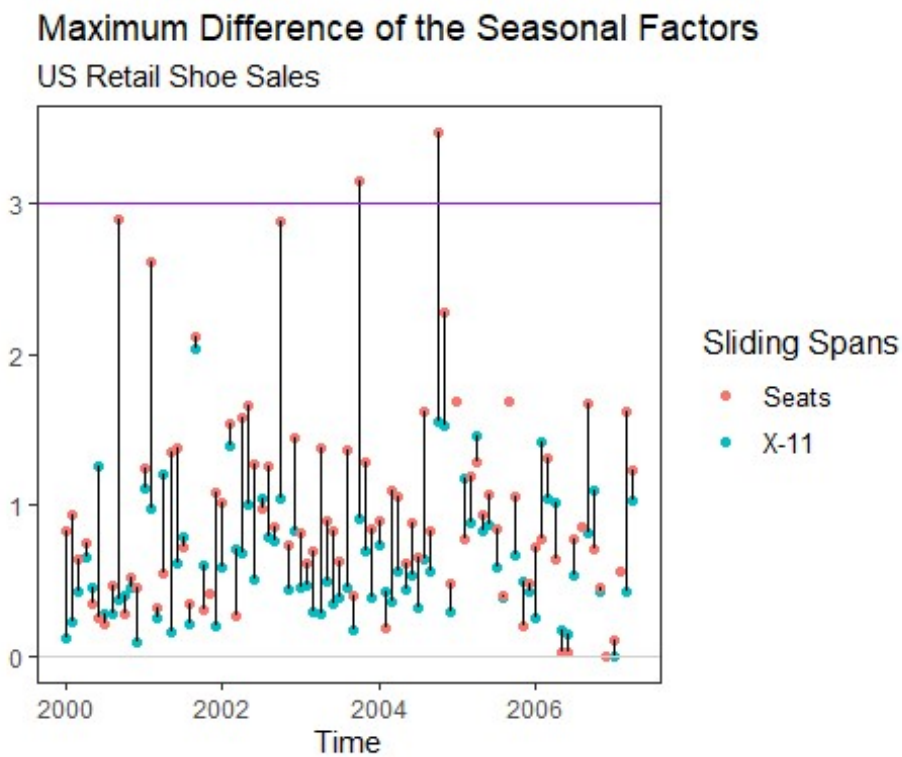

Figure 70: Sliding Spans Comparison Plot Example

9 References

Box, G. E. P., G. M. Jenkins, and G. C. Reinsel. 1976. *Time Series Analysis, Forecasting and Control*. 3rd ed. Holden-Day.

Buszuwski, James S., and Stuart Scott. 1988. "On the Use of Intervention Analysis in Seasonal Adjustment." *Proceeding of the American Statistical Association, Business and Economic Statistics Section*, 337–42.

Cleveland, Robert B., William S. Cleveland, Jean E. McRae, and Irma Terpenning. 1990. "STL: A Seasonal-Trend Decomposition Procedure Based on Loess." *Journal of Official Statistics* 6: 3–73.

Cleveland, William S., and Irma Terpenning. 1982. "Graphical Methods for Seasonal Adjustment." *Journal of the American Statistical Association* 77: 52–72.

Durbin, James, and Siem Jan Koopman. 2012. *Time Series Analysis by State Space Methods*. Oxford University Press. https://doi.org/10.1093/acprof:oso/9780199641178.001.0001.

Findley, D. F. 2005. "Asymptotic Stationarity Properties of Out-of-Sample Forecast Errors of Misspecified RegARIMA Models and the Optimality of GLS for One-Step-Ahead Forecasting." *Statistica Sinica* 15 (2): 447–76. https://www3.stat.sinica.edu.tw/statistica/oldpdf/A15n28.pdf.

Findley, D. F., B. C. Monsell, W. R. Bell, M. C. Otto, and B. C. Chen. 1998. "New Capabilities of the X-12-ARIMA Seasonal Adjustment Program (with Discussion)." *Journal of Business and Economic Statistics* 16: 127–77. {https://www.census.gov/content/dam/Census/library/working-papers/1998/adrm/jbes98.pdf}.

Hood, C. C. 2002. "X-12-Graph: A SAS/GRAPH Program for X-12-ARIMA Output, User's Guide for the X-12-Graph Batch for PC/Windows, Version 1.2."

Hyndman, Rob J, and Yeasmin Khandakar. 2008. "Automatic Time Series Forecasting: The Forecast Package for R." *Journal of Statistical Software* 27 (3): 1–22. https://doi.org/10.18637/jss.v027.i03.

Hyndman, Rob, George Athanasopoulos, Christoph Bergmeir, Gabriel Caceres, Leanne Chhay, Mitchell O'Hara-Wild, Fotios Petropoulos, Slava Razbash, Earo Wang, and Farah Yasmeen. 2025. *forecast: Forecasting Functions for Time Series and Linear Models*. https://pkg.robjhyndman.com/forecast/.

Kassambara, Alboukadel. 2025. *Ggpubr: 'Ggplot2' Based Publication Ready Plots*. https://CRAN.R-project.org/package=ggpubr.

Lytras, Demetra. 2022a. *X-13-Graph Batch, Version 3.0: A SAS/GRAPH® Program for x-13ARIMA-SEATS Output User's Guide for the Windows Interface to x-13-Graph, Version 3.1*. Washington, DC: U.S. Census Bureau, U.S. Department of Commerce. https://www2.census.gov/software/x-13arima-seats/x-13-graph/batch/x13gb-interface-doc.pdf.

———. 2022b. *X-13-Graph Java Documentation Version 3.1*. Washington, DC: U.S. Census Bureau, U.S. Department of Commerce. https://www2.census.gov/software/x-13arima-seats/x-13-graph/java/x13graphjavadoc.pdf.

Monsell, Brian C. 2025. *blsplotGG: Plots for Seasonal Adjustment Analysts Using Ggplot2*.

Neuwirth, Erich. 2022. *RColorBrewer: ColorBrewer Palettes*. https://CRAN.R-project.org/package=RColorBrewer.

R Core Team. 2024. *R: A Language and Environment for Statistical Computing*. Vienna, Austria: R Foundation for Statistical Computing. https://www.R-project.org/.

Sax, Christoph. 2021. *Tsbox: Class-Agnostic Time Series in R*. https://docs.ropensci.org/tsbox/.

———. 2018. *Seasonal: R Interface to X-13ARIMA-SEATS. R Package Version 1.7*. https://CRAN.R-project.org/package=seasonal.

———. 2024. "Adjusting Multiple Series — Cran.r-Project.org." https://cran.r-project.org/web/packages/seasonal/vignettes/multiple.html. [Accessed 20-02-2026]

Sax, Christoph, and Dirk Eddelbuettel. 2018. "Seasonal Adjustment by X-13ARIMA-SEATS in R." *Journal of Statistical Software* 87 (11): 1–17. https://doi.org/10.18637/jss.v087.i11.

Scott, Stuart. 1987. "On the Impact of Outliers on Seasonal Adjustment." *Proceeding of the American Statistical Association, Business and Economic Statistics Section*, 469–74.

Soukup, Raymond J., and David F. Findley. 1999. "On the Spectrum Diagnostics Used by X-12-ARIMA to Indicate the Presence of Trading Day Effects After Modeling or Adjustment." *Proceeding of the American Statistical Association, Business and Economic Statistics Section*, 144–49.

Stoffer, David. 2025. *Astsa: Applied Statistical Time Series Analysis*. https://CRAN.R-project.org/package=astsa.

U. S. Census Bureau. 2025. "X-13ARIMA-SEATS Reference Manual, Version 1.1." https://www2.census.gov/software/x-13arima-seats/x13as/windows/documentation/docx13ashtml.pdf.

Wickham, Hadley. 2016. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York. https://ggplot2.tidyverse.org.